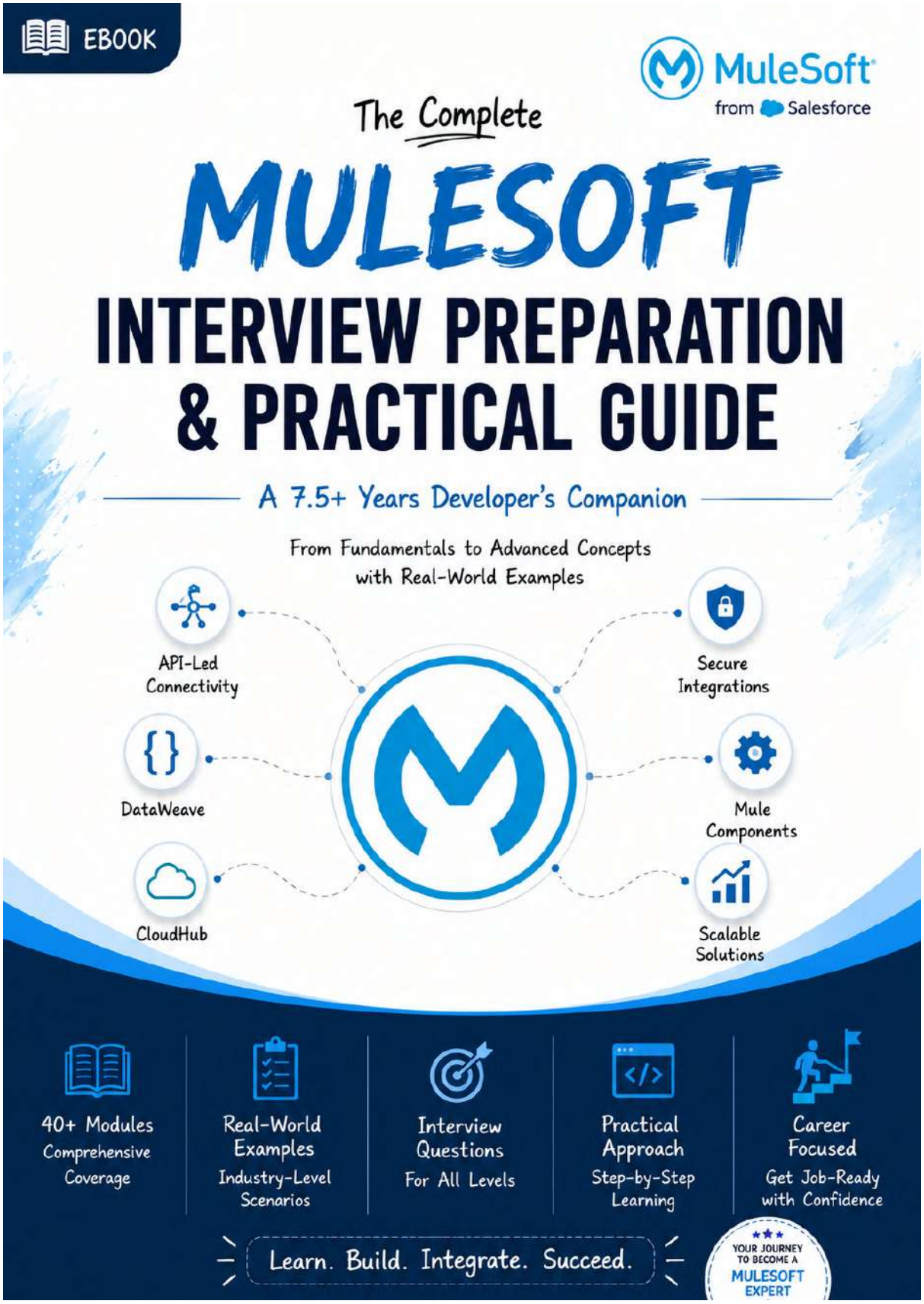

EBOOK
MuleSoft
from Salesforce
The Complete
MULESOFT
INTERVIEW PREPARATION
& PRACTICAL GUIDE
A 7.5+ Years Developer's Companion
From Fundamentals to Advanced Concepts
with Real-World Examples
API-Led Connectivity
DataWeave
CloudHub
Secure Integrations
Mule Components
Scalable Solutions
40+ Modules
Comprehensive Coverage
Real-World Examples
Industry-Level Scenarios
Interview Questions
For All Levels
Practical Approach
Step-by-Step Learning
Career Focused
Get Job-Ready with Confidence
Learn. Build. Integrate. Succeed.
YOUR JOURNEY TO BECOME A MULESOFT EXPERT

11.1 OAUTH 2.0

MuleSoft from salesforce

OAuth 2.0 is an industry-standard authorization framework that enables secure access to resources without sharing user credentials. It is an authorization framework, not authentication.

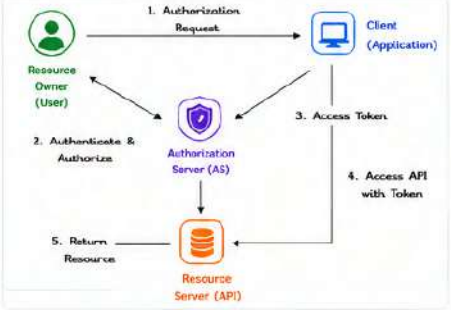
1. WHAT IS OAUTH 2.0?

- OAuth 2.0 allows a user to grant limited access to their resources on one site to another site without sharing the password.
- It is an authorization framework, not an authentication protocol.
- It works on top of HTTPS.
- It uses Access Tokens instead of username/password.
- Commonly used with APIs.

2. KEY ROLES

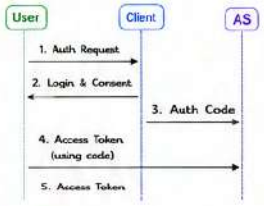
Resource Owner (User)	The end user who owns the resource.
Client (Application)	Application requesting access to the resource.
Authorization Server (AS)	Server that authenticates the user and issues tokens.
Resource Server (API)	Server that hosts the protected resources.

3. OAUTH 2.0 FLOW (ABSTRACT)



4. OAUTH 2.0 GRANT TYPES

1. Authorization Code Grant (Used by Web Applications)

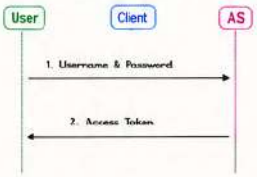


Best For: Server-side web apps
Flow: Involves redirect, auth code exchange.

2. Client Credentials Grant (Used by Machine-to-Machine)

Best For: APIs, integrations, back-end services.
Flow: No user involved.

3. Resource Owner Password Credentials Grant (Used by Trusted Applications)



Best For: Trusted apps where user trusts the client.
Flow: User credentials sent to client.

4. Refresh Token Grant (Used to get new access token)

Best For: Getting new tokens when access token expires.
Flow: Uses refresh token.

5. TOKEN TYPES

Access Token

- Represents the permission to access resources.
- Short-lived.
- Sent in Authorization header Bearer <access_token>.
- Used to call APIs.

Refresh Token

- Used to obtain a new access token.
- Long-lived.
- Not sent to APIs.
- Should be stored securely.

6. TOKEN RESPONSE (EXAMPLE)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "def50200b3a1e1...",
  "scope": "read write",
  "issued_token_type": "access_token"
}
```

- access_token: The token to access APIs
- token_type: Always "Bearer" in OAuth 2.0
- expires_in: Token validity in seconds
- refresh_token: Token to get a new access token
- scope: Permissions granted
- issued_token_type: Type of token issued

7. AUTHORIZATION HEADER

Use the access token in API calls:

```
GET /api/orders HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Accept: application/json
```

8. SCOPES

Scopes define what the access token can do.

Example: scope=read write delete

- read -> Can read data
- write -> Can create/update data
- delete -> Can delete data

9. BEST PRACTICES

- Always use HTTPS.
- Use Authorization Code Grant for user-based apps.
- Store tokens securely. Never expose in frontend.
- Use short-lived access tokens and refresh tokens.
- Validate token (signature, expiry, audience, issuer).
- Revoke tokens when no longer needed.

10. OAUTH 2.0 VS OAUTH 1.0

Feature	OAuth 1.0	OAuth 2.0
Complexity	Complex (Signature based)	Simple
Transport	Can work over HTTP	Must use HTTPS
Token Types	Single Token	Access Token + Refresh Token
Grant Types	Limited	Multiple (4 grant types)
Use Case	Legacy APIs	Modern APIs, Mobile, Web, Microservices

11. COMMON ENDPOINTS

Endpoint	Purpose
/authorize	User authorization (login & consent)
/token	Issue access token / refresh token
/userinfo	Get user information
/introspect	Validate token
/revoke	Revoke a token

12. OAUTH 2.0 IN MULESOFT

- Use HTTP Request connector to get tokens.
- Store client_id, client_secret in Secure Properties.
- Store tokens in Object Store or Cache.
- Use token in Authorization header for API calls.
- Validate tokens using JWT Validation policy (if JWT token).

KEY TAKEAWAYS

- OAuth 2.0 is an authorization framework, not an authentication protocol.
- It enables secure delegated access to resources.
- Use the right grant type for your use case.
- Access tokens are short-lived; refresh tokens are long-lived.
- Always follow security best practices to protect tokens and user data.

Secure access. Protect data. Build resilient integrations with OAuth 2.0!

11.1 OAUTH 2.0

MuleSoft from salesforce

OAuth 2.0 is an industry-standard authorization framework that enables secure access to resources without sharing user credentials. It is an authorization framework, not authentication.

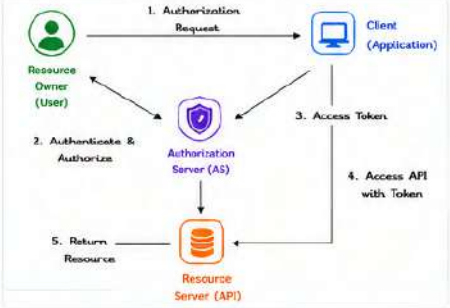
1. WHAT IS OAUTH 2.0?

- OAuth 2.0 allows a user to grant limited access to their resources on one site to another site without sharing the password.
- It is an authorization framework, not an authentication protocol.
- It works on top of HTTPS.
- It uses Access Tokens instead of username/password.
- Commonly used with APIs.

2. KEY ROLES

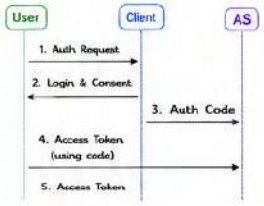
Resource Owner (User)	The end user who owns the resource.
Client (Application)	Application requesting access to the resource.
Authorization Server (AS)	Server that authenticates the user and issues tokens.
Resource Server (API)	Server that hosts the protected resources.

3. OAUTH 2.0 FLOW (ABSTRACT)



4. OAUTH 2.0 GRANT TYPES

1. Authorization Code Grant (Used by Web Applications)

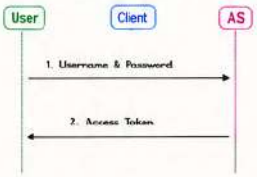


Best For: Server-side web apps
Flow: Involves redirect, auth code exchange.

2. Client Credentials Grant (Used by Machine-to-Machine)

Best For: APIs, integrations, back-end services.
Flow: No user involved.

3. Resource Owner Password Credentials Grant (Used by Trusted Applications)



Best For: Trusted apps where user trusts the client.
Flow: User credentials sent to client.

4. Refresh Token Grant (Used to get new access token)

Best For: Getting new tokens when access token expires.
Flow: Uses refresh token.

5. TOKEN TYPES

Access Token

- Represents the permission to access resources.
- Short-lived.
- Sent in Authorization header Bearer <access_token>.
- Used to call APIs.

Refresh Token

- Used to obtain a new access token.
- Long-lived.
- Not sent to APIs.
- Should be stored securely.

6. TOKEN RESPONSE (EXAMPLE)

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "def50200b3a1e1...",
  "scope": "read write",
  "issued_token_type": "access_token"
}
```

- access_token: The token to access APIs
- token_type: Always "Bearer" in OAuth 2.0
- expires_in: Token validity in seconds
- refresh_token: Token to get a new access token
- scope: Permissions granted
- issued_token_type: Type of token issued

7. AUTHORIZATION HEADER

Use the access token in API calls:

```
GET /api/orders HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Accept: application/json
```

8. SCOPES

Scopes define what the access token can do.

Example: scope=read write delete

- read -> Can read data
- write -> Can create/update data
- delete -> Can delete data

9. BEST PRACTICES

- Always use HTTPS.
- Use Authorization Code Grant for user-based apps.
- Store tokens securely. Never expose in frontend.
- Use short-lived access tokens and refresh tokens.
- Validate token (signature, expiry, audience, issuer).
- Revoke tokens when no longer needed.

10. OAUTH 2.0 VS OAUTH 1.0

Feature	OAuth 1.0	OAuth 2.0
Complexity	Complex (Signature based)	Simple
Transport	Can work over HTTP	Must use HTTPS
Token Types	Single Token	Access Token + Refresh Token
Grant Types	Limited	Multiple (4 grant types)
Use Case	Legacy APIs	Modern APIs, Mobile, Web, Microservices

11. COMMON ENDPOINTS

Endpoint	Purpose
/authorize	User authorization (login & consent)
/token	Issue access token / refresh token
/userinfo	Get user information
/introspect	Validate token
/revoke	Revoke a token

12. OAUTH 2.0 IN MULESOFT

- Use HTTP Request connector to get tokens.
- Store client_id, client_secret in Secure Properties.
- Store tokens in Object Store or Cache.
- Use token in Authorization header for API calls.
- Validate tokens using JWT Validation policy (if JWT token).

KEY TAKEAWAYS

- OAuth 2.0 is an authorization framework, not an authentication protocol.
- It enables secure delegated access to resources.
- Use the right grant type for your use case.
- Access tokens are short-lived; refresh tokens are long-lived.
- Always follow security best practices to protect tokens and user data.

Secure access. Protect data. Build resilient integrations with OAuth 2.0!

11.1.1 OAUTH 2.0 – HOW TO APPLY

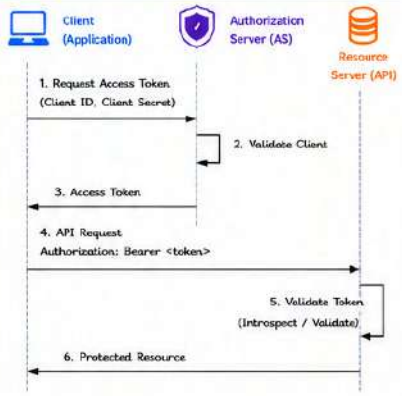
MuleSoft from salesforce

OAuth 2.0 is an authorization framework. To apply it, an API (Resource Server) validates access tokens issued by an Authorization Server. Clients obtain tokens and call the API with those tokens.

1. HIGH LEVEL STEPS

1. User authenticates with Authorization Server (AS) via Client (Application).
2. Client receives Access Token from AS.
3. Client calls API (Resource Server) with Access Token in Authorization header.
4. API validates token with AS (if required) and allows access to protected resource.
5. API returns the requested resource to the client.

2. SEQUENCE DIAGRAM (CLIENT CREDENTIALS GRANT)



3. WHERE TO APPLY IN MULESOFT

A. AS A CLIENT (CALLING AN API)

- Register your application with the Authorization Server and get Client ID & Secret.
- Request an access token using OAuth 2.0 grant type.
- Store token securely (in memory / cache).
- Add token in Authorization header while calling the API.

B. AS A RESOURCE SERVER (PROTECTING AN API)

- Configure API to accept OAuth 2.0 tokens.
- Validate the token (signature or introspection).
- Extract claims/scopes from token.
- Allow/deny access based on scopes/roles.

4. APPLYING OAUTH 2.0 AS A CLIENT IN MULESOFT

1. Configure HTTP Listener (If needed for token flow)

```
<http:listener-config name="HTTP_Listener_config"
 host="0.0.0.0" port="8081" />

<http:listener config-ref="HTTP_Listener_config"
 path="/token" allowedMethods="POST" />
```

2. Request Access Token (Example: Client Credentials Grant)

```
<http:request config-ref="HTTP_Request_config"
 method="POST" path="/oauth2/token">
  <http:request-builder>
    <http:header headerName="Content-Type"
      value="application/x-www-form-urlencoded" />
  </http:request-builder>
  <http:body>
   <![CDATA[
   grant_type=client_credentials&
   client_id=YOUR_CLIENT_ID&
   client_secret=YOUR_CLIENT_SECRET&
   scope=read write
   ]]>
  </http:body>
 <http:request-builder>
</http:request>
```

Store access_token from response.

3. Call the Protected API with Access Token

```
<http:request config-ref="HTTP_Request_config"
 method="GET" path="/api/customers">

 <http:request-builder>
  <http:header headerName="Authorization"
    value="Bearer #[vars.accessToken]" />

 </http:request-builder>
</http:request>
```

Use token in Authorization header

4. Handle Token Expiry (Refresh if needed)

If token is expired (401):

- Request new token using refresh_token.
- Update stored token.
- Retry original API request

Best Practice: Cache token and reuse until it expires.

5. APPLYING OAUTH 2.0 AS A RESOURCE SERVER IN MULESOFT

1. Accept Token in Request

```
<http:listener config-ref="HTTP_Listener_config"
path="/api/customers" allowedMethods="GET" />
```

Client must send:
Authorization: Bearer <access_token>

2. Validate Token

Options:

- Validate JWT (signature & claims)
- Introspect token with AS
- Use scopes/roles for authorization

Example: Validate JWT

```
<mule:jwt-validate config-ref="JWT_Config"
 doc:name="Validate JWT" />
```

3. Check Scopes / Claims

Example (Check scope):

```
<expression-filter expression="#[
attributes.jwt.claims.scope contains 'read:customers'
]" doc:name="Check Scope" />
```

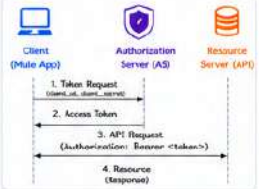
4. Process & Return Resource

If valid → process request and return data.
If invalid → return 401/403

```
<http:error-response statusCode="401"
 reasonPhrase="Unauthorized" />
```

6. EXAMPLE – CLIENT CREDENTIALS FLOW (END TO END)

1. Mule application requests token from AS using client_id & client_secret.
2. AS validates client and returns access_token.
3. Mule application calls API with Authorization: Bearer <token>.
4. API validates token (introspect or verify JWT).
5. API checks scope and processes request.
6. API returns the resource to the client.



7. COMMON OAUTH 2.0 ENDPOINTS

Endpoint	Purpose	Method	Example
/oauth/authorize	User authorization (Auth Code)	GET	/oauth/authorize
/oauth/token	Get access/refresh token	POST	/oauth/token
/oauth/introspect	Validate token	POST	/oauth/introspect
/oauth/revoke	Revoke token	POST	/oauth/revoke
/oauth/userinfo	Get user info (OIDC)	GET	/oauth/userinfo

8. BEST PRACTICES

- Always use HTTPS.
- Store client_id, client_secret in Secure Properties.
- Cache access tokens and reuse until expiry.
- Validate token on every request (or use short-lived token).
- Use least-privilege scopes.
- Handle token expiry and refresh gracefully.
- Log and monitor authorization failures.

9. COMMON ERROR SCENARIOS

- 401 Unauthorized → Missing/invalid/expired token.
- 403 Forbidden → Valid token but insufficient scope.
- invalid_client → Wrong client_id or client_secret.
- invalid_grant → Wrong grant type or expired refresh token.
- invalid_token → Token is invalid or revoked.

10. USEFUL COMPONENTS IN MULESOFT

Component	Use
HTTP Request	Call token endpoint or protected APIs
HTTP Listener	Expose protected API
JWT Validate	Validate JWT tokens
OAuth 2.0 Module	Advanced OAuth flows
Secure Properties	Store client secrets securely

Apply OAuth 2.0 the right way: Secure tokens, validate properly, and protect your APIs!

11.2 JWT (JSON WEB TOKEN)

MuleSoft from salesforce

JWT (JSON Web Token) is an open standard (RFC 7519) used to securely transmit information between parties as a JSON object. It is compact, self-contained and digitally signed.

1. OVERVIEW

- JWT is used for secure data exchange between client and server.
- It is commonly used for Authentication and Authorization.
- JWTs are stateless and can be verified without calling the issuer.
- It consists of three parts: Header, Payload, Signature.

2. JWT STRUCTURE

Header	.	Payload	.	Signature
Defines the type of token and signing algorithm.		Contains the claims (data).		Created using header, payload and secret key.
Base64Url Encode		Base64Url Encode		Base64Url Encode

The three parts are concatenated with dots (.) and form the JWT.

3. EXAMPLE JWT

JWT:

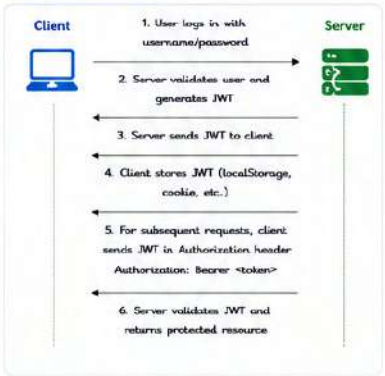
```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjMiLCJuYW1lIjoiSm9obiIsImlhdCI6MTcxNTg2MjAwMH0.
dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

Header (Base64) | Payload (Base64) | Signature

Decoded:

```
Header    → {"alg":"HS256","typ":"JWT"}
Payload   → {"sub":"123", "name":"John", "iat":1715862000}
Signature → HMACSHA256(base64Url(header) + "." +
            base64Url(payload), secret_key)
```

4. HOW JWT WORKS



5. JWT CLAIMS (PAYLOAD)

Claims are the statements about an entity (user) and additional data.

		Description
Registered Claims	iss (Issuer)	Issuer of the token
	sub (Subject)	Subject (user id)
	aud (Audience)	Audience for the token
	exp (Expiration Time)	Token expiry time (timestamp)
	nbf (Not Before)	Token not valid before (timestamp)
	iat (Issued At)	Time when token was issued (timestamp)
	jti (JWT ID)	Unique identifier for the token
Public Claims		Any custom claim defined by the application.
Private Claims		Custom claims used between parties that agree on them.

6. SIGNING ALGORITHMS

Common Algorithms	Description
HS256	HMAC + SHA256 (Symmetric)
HS384	HMAC + SHA384 (Symmetric)
HS512	HMAC + SHA512 (Symmetric)
RS256	RSA + SHA256 (Asymmetric)
RS384	RSA + SHA384 (Asymmetric)
RS512	RSA + SHA512 (Asymmetric)
ES256	ECDSA + SHA256 (Asymmetric)

Symmetric: Same secret key to sign and verify.
Asymmetric: Public key to verify, private key to sign.

7. JWT VALIDATION PROCESS

- Verify the signature using the secret/public key.
- Check the algorithm used.
- Validate expiration time (exp).
- Validate not before time (nbf) if present.
- Validate issuer (iss) and audience (aud) if required.
- If all validations pass, token is trusted.

8. JWT IN AUTHORIZATION HEADER

The client sends JWT in the Authorization header for protected API calls.

```
Authorization: Bearer <JWT_TOKEN>
```

Example:

```
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjMiLCJuYW1lIjoiSm9obiIsImlhdCI6MTcxNTg2MjAwMH0.
dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

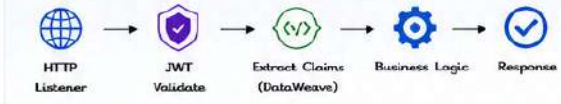
9. USING JWT IN MULESOFT

1. Receive JWT in HTTP request header (Authorization).
2. Use "JWT Validate" scope to validate the token.
3. Configure public key / secret key and algorithms.
4. Extract claims using DataWeave.
5. Use claims for authorization and flow logic.

Key Components:

- HTTP Listener
- JWT Validate
- DataWeave (extract claims)
- Error Handling (for invalid token)

10. EXAMPLE FLOW IN MULESOFT



On invalid token:

- Return 401 Unauthorized
- Send error message
- Do not process the request

11. SAMPLE CLAIMS (PAYLOAD)

```
{
  "iss": "https://auth.example.com",   → Issuer
  "sub": "12345",                      → User ID
  "name": "John Doe",                  → User Name
  "role": "admin",                     → Custom Claim (Role)
  "aud": "api://orders",               → Audience
  "iat": 1715862000,                   → Issued At (timestamp)
  "exp": 1715865600                    → Expires At (timestamp)
}
```

12. BEST PRACTICES

- Always use HTTPS.
- Use strong signing algorithms (HS256/RS256 or above).
- Keep secret keys and private keys secure.
- Set short expiration time for tokens.
- Validate all claims (exp, iss, aud, etc.).
- Use scopes/roles from claims for authorization.
- Do not store sensitive data in JWT payload.

13. COMMON ERRORS

- ✖ Expired token (401 Unauthorized).
- ✖ Invalid signature (Token tampered or wrong key).
- ✖ Invalid issuer or audience.
- ✖ Token missing in Authorization header.
- ✖ Using symmetric algorithm with public key.

14. WHEN TO USE JWT

- When you need stateless authentication.
- When tokens need to be shared across multiple services.
- When high performance and scalability are required.
- When you need to include user info in the token (claims).

JWT = Secure, Stateless, Signed Tokens for Modern APIs!

11.3 TLS (TRANSPORT LAYER SECURITY)

MuleSoft from salesforce

TLS (Transport Layer Security) is a cryptographic protocol that provides secure communication over an insecure network (like the Internet). It ensures data confidentiality, integrity and authenticity between client and server.

1. WHAT IS TLS?

- TLS is the security layer between the application layer (e.g., HTTP) and the transport layer (TCP).
- It encrypts data in transit and protects it from eavesdropping and tampering.
- TLS is the successor of SSL (Secure Sockets Layer).
- Used by HTTPS, SMTPS, IMAPS, FTPS and other secure protocols.

2. TLS IN THE OSI MODEL

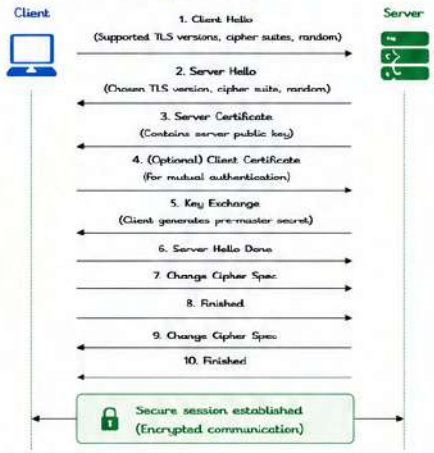
7	Application
6	Presentation
5	Session
4	Transport Layer Security (TLS)
3	Transport (TCP)
2	Network (IP)
1	Data Link
0	Physical

TLS sits between Application Layer and Transport Layer (TCP).

3. TLS GOALS

- **Confidentiality** — Ensures data is encrypted and cannot be read by unauthorized parties.
- **Integrity** — Ensures data is not altered in transit without detection.
- **Authentication** — Ensures the communicating parties are who they claim to be.
- **Non-repudiation (Optional)** — Provides proof of origin and delivery.

4. TLS HANDSHAKE (OVERVIEW)



Step	Description
1. Client Hello	Client proposes TLS version, cipher suites and sends random value.
2. Server Hello	Server selects TLS version & cipher suite and sends random value.
3. Server Certificate	Server sends its certificate (contains public key) signed by a CA.
4. (Optional) Client Certificate	Client sends its certificate if mutual TLS is required.
5. Key Exchange	Client and server use key exchange algorithm to generate pre-master secret.
6. Server Hello Done	Server indicates it has finished its part of handshake.
7-10. Change Cipher Spec & Finished	Both sides switch to encrypted mode and verify the handshake.

5. TLS VERSIONS

Version	Status	Description
SSL 2.0	Deprecated	Insecure. Not used.
SSL 3.0	Deprecated	Insecure due to vulnerabilities.
TLS 1.0	Deprecated	Old and insecure.
TLS 1.1	Deprecated	Better than 1.0 but deprecated.
TLS 1.2	Recommended	Secure and widely used.
TLS 1.3	Recommended	Latest and most secure.

6. TLS CIPHER SUITE

A cipher suite is a combination of algorithms used in TLS:

- Key Exchange Algorithm (e.g., ECDHE, DHE, RSA)
- Authentication Algorithm (e.g., RSA, ECDSA)
- Encryption Algorithm (e.g., AES, 3DES)
- Hash Algorithm (e.g., SHA256, SHA384)

7. TLS CONNECTION FLOW (HIGH-LEVEL)

8. TLS RECORD PROTOCOL

After the handshake, TLS uses Record Protocol to transmit data securely.

At the receiver, the reverse process is applied.

9. DIFFERENCE: TLS vs SSL

Feature	SSL	TLS
Security	Insecure	Secure
Performance	Slow	Fast
Algorithm	Weak	Strong
Status	Deprecated	Current Standard

10. WHERE IS TLS USED?

- HTTPS (Web)
- API gateways and Microservices
- SMTP over TLS (Email)
- IMAP/POP over TLS
- Database connections
- File transfers (FTPS)

11. TLS BEST PRACTICES

- Always use TLS 1.2 or TLS 1.3.
- Disable outdated protocols (SSL, TLS 1.0, 1.1).
- Use strong cipher suites (ECDHE + AESGCM + SHA256).
- Enable HSTS for HTTPS.
- Validate certificates and hostnames.
- Use Perfect Forward Secrecy (PFS).

12. COMMON TLS ERRORS

- Certificate expired or not trusted
- Hostname mismatch
- Unsupported TLS version
- Weak cipher suite
- Insecure protocol (SSL/TLS 1.0/1.1)

13. TLS IN MULESOFT

- Mule runtime uses TLS for secure connections by default in HTTPS.
- Configure TLS in Global Elements → TLS Configuration.
- Use trusted keystore for client certificates (mTLS).
- Validate server certificates and hostnames.
- Supports TLS 1.2 and TLS 1.3 (depending on Mule runtime version).

14. USEFUL COMMANDS (OPENSSL)

- View certificate details:
  `openssl s_client -connect example.com:443 -showcerts`
- Check TLS version supported by server:
  `openssl s_client -connect example.com:443 -tls1_2`
- Test cipher suite:
  `openssl ciphers -v 'ECDHE'`

15. KEY TAKEAWAYS

- TLS secures data in transit.
- It provides confidentiality, integrity and authenticity.
- TLS handshake establishes a secure channel before data exchange.
- Always use the latest TLS version and strong cipher suites.

TLS = Secure Channel. Protect Data. Build Trust.

11.4 SSL CERTIFICATES

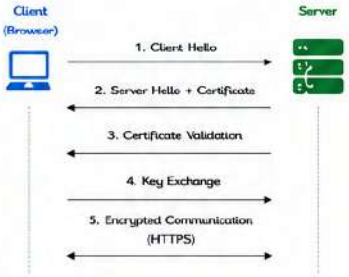
MuleSoft from salesforce

SSL (Secure Sockets Layer) Certificates are digital certificates that authenticate a website/server identity and enable encrypted communication over SSL/TLS.

1. WHAT IS AN SSL CERTIFICATE?

- An SSL certificate binds a domain name to an organization and enables HTTPS.
- It is issued by a trusted authority called Certificate Authority (CA).
- It contains the public key and identity information of the server/organization.
- It is used to establish encryption and authenticate the server.

2. HOW SSL WORKS (OVERVIEW)



3. TYPES OF SSL CERTIFICATES

Type	Description
Domain Validated (DV)	Validates domain ownership only. Basic level of validation.
Organization Validated (OV)	Validates domain and organization information.
Extended Validation (EV)	Highest level of validation. Shows organization name in browser.
Wildcard Certificate	Secures main domain and all its subdomains. Example: *.example.com
Multi-Domain (SAN/UCC)	Secures multiple domains and subdomains in a single certificate.

4. SSL CERTIFICATE HIERARCHY

- Root Certificate Authority (Self-Signed) — Pre-installed in browsers. Trusts Intermediate CA.
- ↓
- Intermediate Certificate Authority (Issued by Root CA) — Issued by Root CA. Signs and issues server certificates.
- ↓
- Server Certificate (Issued by Intermediate CA) — Issued to domain/server. Contains public key
- ↓
- Your Server / Website (example.com) — Presents certificate to client for secure communication.

5. SSL CERTIFICATE COMPONENTS

Component	Description
Subject (CN)	Domain name (example.com) or organization.
Issuer	Certificate Authority (who issued the certificate).
Validity Period	From (issued date) and To (expiry date).
Public Key	Used by clients to encrypt data to server.
Signature Algorithm	Algorithm used by CA to sign the certificate.
Serial Number	Unique number of the certificate.
SAN (Subject Alternative Name)	Additional domains/subdomains covered by the certificate.

6. SSL CERTIFICATE FILES

File / Format	Purpose
.crt / .cer	Certificate file (public certificate).
.key	Private key of the server.
.csr	Certificate Signing Request – used to request a certificate from CA.
.pfx / .p12	PKCS#12 bundle (Certificate + Private Key).
.pem	Base64 encoded certificate file.
.chain / .ca-bundle	Intermediate and root certificates (bundle).

7. SSL vs TLS

Feature	SSL (Old)	TLS (Current)
Secure Protocol	SSL 2.0, 3.0	TLS 1.0, 1.1, 1.2, 1.3
Security	Insecure / Deprecated	Secure and Recommended
Performance	Slower	Faster
Status	Deprecated	Current Standard
Use	Not recommended	Use TLS (disable SSL)

8. SELF-SIGNED vs CA-SIGNED CERTIFICATE

Self-Signed Certificate	CA-Signed Certificate
• Created by yourself.	• Issued by trusted CA.
• Not trusted by browsers.	• Trusted by browsers.
• Browser shows warning.	• No warnings.
• Used for development, testing, internal systems.	• Used for production environments.

9. SSL CERTIFICATE LIFECYCLE

1. **Generate CSR** (Certificate Signing Request) — Create CSR with private key.
2. **Submit CSR to CA** — Provide domain and organization details.
3. **Validation by CA** — CA validates domain/organization.
4. **Certificate Issued** — CA issues SSL certificate.
5. **Install Certificate** — Install certificate on server.
6. **Renew / Revoke** — Renew before expiry or revoke if needed.

10. SSL CERTIFICATE INSTALLATION (GENERAL STEPS)

1. Generate CSR and Private Key on server.
2. Submit CSR to Certificate Authority (CA).
3. Download the issued SSL certificate and CA bundle.
4. Install certificate and intermediate bundle on server.
5. Configure server to use certificate (Enable HTTPS).
6. Test SSL using browser or SSL tools.

11. SSL CERTIFICATE IN MULESOFT

- Mule runtime can act as both SSL client and SSL server.
- Import SSL certificate and private key into Keystore.
- Configure HTTPS Listener or HTTP Request Connector with the keystore.

Steps in MuleSoft:

1. Import certificate (.pfx / .jks) into Keystore.
2. Configure HTTPS Listener (port, keystore, password).
3. (Client) Configure SSL Context for outbound requests.
4. Deploy and test secure connectivity.

12. COMMON SSL ERRORS

- ✖ Certificate expired
- ✖ Hostname mismatch
- ✖ Self-signed certificate not trusted
- ✖ Missing intermediate certificate
- ✖ Using weak protocols (SSLv3, TLS 1.0)
- ✖ Invalid certificate chain

13. COMMON SSL TOOLS

Tool	Use
OpenSSL	Create keys, CSR, view certificates
SSL Labs (ssllabs.com)	Test SSL server configuration
KeyStore Explorer	View/manage keystores
Postman	Test HTTPS APIs
Browser Developer Tools	View certificate details

14. BEST PRACTICES

- ✓ Use TLS 1.2 or TLS 1.3 only.
- ✓ Disable SSL and TLS 1.0 / 1.1.
- ✓ Use strong ciphers (AES, ECDHE).
- ✓ Keep certificates and private keys secure.
- ✓ Renew certificates before expiry.
- ✓ Use HSTS (HTTP Strict Transport Security).
- ✓ Install full certificate chain (including intermediates).

15. KEY TAKEAWAYS

- ★ SSL certificates authenticate server identity.
- ★ They enable encrypted and secure communication.
- ★ Always use trusted CA-signed certificates in production.
- ★ Keep certificates up to date and secure.
- ★ TLS is the modern replacement for SSL.

SSL Certificates = Authenticate Identity + Encrypt Data + Build Trust

11.5.1 HOW TO SECURE PROPERTY VALUE IN MULESOFT

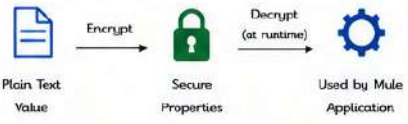
MuleSoft from salesforce

In MuleSoft, sensitive property values (passwords, tokens, API keys, etc.) can be stored in encrypted format using Secure Properties so that they are not exposed in plain text.

1. OVERVIEW

- Secure Properties module encrypts the property values.
- Encrypted values are stored in a properties file.
- At runtime, Mule automatically decrypts and uses the values.
- Encryption uses AES algorithm (default).
- The key (master key) must be kept safe.

2. HOW IT WORKS



Values are encrypted using a key.
At runtime, Mule uses the key to decrypt the values.

3. PREREQUISITES

- Anypoint Studio
- Secure Properties module (comes with Studio)
- Access to Mule application project

4. STEPS TO SECURE PROPERTY VALUE

1. Generate a secure key.
2. Create a properties file and add plain text values.
3. Encrypt the properties file.
4. Use the encrypted file in your application.
5. At runtime, Mule decrypts and uses the values.
6. Protect the key file (.key) securely.

5. STEP-BY-STEP EXAMPLE

Step 1: Generate a Secure Key

```
mule secure-properties generate-key
```

This creates a file like:
mykey.key

Step 2: Create Properties File (Plain Text)

File: config-dev.yaml

```
db:
  host: localhost
  user: admin
  password: MySecret123
api:
  token: eyJhbGciOiJIUzI1NiIs...
```

Step 3: Encrypt the Properties File

```
mule secure-properties encrypt \
  --key mykey.key \
  --input config-dev.yaml \
  --output config-dev-secure.yaml
```

Output File (Encrypted): config-dev-secure.yaml

```
db:
  host: localhost
  user: admin
  password: '::(cipher)AQAAANCMnd8BFdERjHoAwE/...'
api:
  token: '::(cipher)AQAAANCMnd8BFdERjHoAwE/...'
```

6. HOW TO READ PROPERTY VALUES

A. READ NORMAL (PLAIN TEXT) VALUES

Use as usual with ${property}.

Example:

```
${db.host}
```

In Mule:

```
<set-variable variableName="host"
  value="${db.host}" />
```

B. READ SECURE VALUES

Use secure::() to read encrypted values.

Example:

```
${secure::db.password}
```

In Mule:

```
<set-variable variableName="password"
  value="${secure::db.password}" />
```

7. USE SECURE PROPERTIES IN MULE APPLICATION

1. Place the encrypted file in the application (src/main/resources)

```
src/main/resources
  ├── config-dev-secure.yaml  (encrypted)
  └── mykey.key  (secure key)
```

2. Reference in mule-artifact.json

```
"secureProperties": [
  {
    "key": "mykey.key",
    "file": "config-dev-secure.yaml"
  }
]
```

3. Use secure::() in your flows

```
<set-variable variableName="dbPassword"
  value="${secure::db.password}" />
```

8. COMMAND REFERENCE

Command	Purpose
mule secure-properties generate-key	Generates a new secure key file (.key)
mule secure-properties encrypt	Encrypts a plain properties file
mule secure-properties decrypt	Decrypts a secure properties file
mule secure-properties change-key	Re-encrypts a file with a new key

Important Notes

- Keep the key file secret.
- If the key is lost, encrypted values cannot be decrypted.
- Never commit key files or encrypted property files to version control.

9. EXAMPLE USAGE IN FLOW

```
<flow name="exampleFlow">
  <set-variable variableName="dbHost"
    value="${db.host}" />
  <set-variable variableName="dbUser"
    value="${db.user}" />
  <set-variable variableName="dbPassword"
    value="${secure::db.password}" />
  <http:request method="GET" url="http://myhost/api"
    >
    <http:headers>
      <http:header name="Authorication"
        value="Berarer ${secure::api.token}" />
    </http:headers>
  </http:request>
</flow>
```

10. COMMON USE CASES

- Database passwords
- API keys and tokens
- Client secrets
- Private keys / Certificates
- Any sensitive configuration

11. ERROR SCENARIOS

Error	Cause	Solution
Unable to decrypt property value	Wrong key file used	Ensure correct .key file is used
Key not found	Key file path is incorrect	Check path in mule-artifact.json
Invalid cipher text	Encrypted file is corrupted	Re-encrypt the properties file
Decryption failed	Key file is corrupted or mismatched	Regenerate key and re-encrypt

Secure your property values. Protect your applications.
Encrypt → Store Securely → Decrypt at Runtime → Use Safely

11.6 SECRETS MANAGEMENT

MuleSoft from salesforce

Secrets Management in MuleSoft helps you store, manage, and protect sensitive information (passwords, tokens, API keys, certificates, etc.) outside of your application and access them securely.

1. WHAT IS SECRETS MANAGEMENT?

- Secure storage and management of sensitive information.
- Secrets are not stored in code, properties files, or configuration.
- Secrets are encrypted at rest and in transit.
- Access is controlled using policies and permissions.

2. HOW IT WORKS

- Application requests a secret using a key or reference.
- Secrets Manager authenticates and retrieves the secret.
- Secret is returned to the application at runtime only.
- Secret is never stored in the application or configuration.

3. KEY BENEFITS

- Centralized and secure secret storage.
- No hardcoding of sensitive data.
- Secrets are encrypted and access controlled.
- Easier rotation and lifecycle management.
- Audit and compliance support.

4. TYPES OF SECRETS

- Database credentials
- API keys and access tokens
- Client secrets
- Private keys / Certificates
- Encryption keys
- Any other sensitive information

5. SECRETS MANAGEMENT ARCHITECTURE

6. SUPPORTED SECRETS STORES

- Anypoint Secrets Manager (Salesforce Managed)
- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- Custom Secrets Store (Using Secrets SPI)

7. HOW TO USE SECRETS IN MULESOFT

7.1 Store Secret in Secrets Manager

Create a secret in your chosen secrets store with a key.

Example:

```
Key: db.password
Value: My$3cr3tP@ssw0rd
```

7.2 Read Secret in Mule Application

Use secure::() to read the secret at runtime.

Example (in DataWeave):

```
${secure::db.password}
```

7.3 Usage in XML Configuration

```
<set-variable variableName="dbPassword"
    value="${secure::db.password}" />
```

8. CONFIGURE SECRETS IN MULE APPLICATION

1. Add dependency (if using custom secrets store)

```
<dependency>
   <groupId>org.mule.modules</groupId>
   <artifactId>mule-secrets-manager-module</artifactId>
   <version>1.4.0</version>
</dependency>
```

2. Configure Secrets Manager in mule-artifact.json

```
"secretsManager": {
   "name": "anypoint-secrets-manager",
   "config": {
      "serviceUrl": "https://anypoint.mulesoft.com",
      "clientId": "<CLIENT_ID>",
      "clientSecret": "<CLIENT_SECRET>",
      "environment": "Sandbox"
   }
}
```

Configuration varies based on the secrets store used.

9. EXAMPLE: READ SECRET VALUE

DataWeave Example

```
%dw 2.0
output application/json
---
{
   "dbUser": "admin",
   "dbPassword": "${secure::db.password}",
   "apiToken": "${secure::api.token}"
}
```

Flow Example (XML)

```
<flow name="get-data">
  <set-variable variableName="token"
        value="${secure::api.token}" />
  <http:request config-ref="HTTP_Request_config"
    path="/data" method="GET" >
     <http:headers>
       <http:header name="Authorication"
            value="${vars.token}" />
     </http:headers>
  </http:request>
</flow>
```

10. SECRET ROTATION

- Rotate secrets regularly.
- Update the secret in the store.
- Applications automatically use the latest value at runtime.
- No redeployment required.

11. ACCESS CONTROL

- Use roles and policies to control access.
- Grant least privilege access.
- Audit who accessed what and when.

12. BEST PRACTICES

- Never store secrets in code, properties, or repo.
- Use Secrets Manager for all sensitive data.
- Use secure::() to access secrets in application.
- Rotate secrets regularly.
- Monitor and audit secret access.
- Use environment-specific secret values.

13. COMMON USE CASES

- Secure database passwords
- API authentication tokens
- OAuth client secrets
- Private keys and certificates
- Third-party service credentials
- Encryption / signing keys

14. ERROR SCENARIOS & TROUBLESHOOTING

Error	Cause	Solution
Secret not found	Wrong key or secret doesn't exist	Verify key and secret in store
Access denied	Insufficient permissions	Check roles and policies
Unable to connect	Connectivity or auth issue	Check network and credentials
Decryption failed	Wrong key / corrupted secret	Verify secret value and format
Runtime error	secure::() used in unsupported place	Use in supported components only

15. KEY TAKEAWAYS

- Secrets Management keeps sensitive data secure and centralized.
- Use secure::() to access secrets in your Mule application.
- Supports multiple secret stores.
- Follows best practices for security and compliance.

Store Securely. Access Safely. Protect Always.

Secrets Management = Security + Control + Compliance

MODULE 12

12.1 GIT FUNDAMENTALS

MuleSoft from salesforce

Git is a distributed version control system used to track changes in source code during software development. It helps multiple developers collaborate efficiently, maintain history, and manage versions of the code.

WHAT IS GIT?

- Git is a distributed Version Control System (DVCS).
- It tracks changes in files over time.
- It allows multiple developers to work on the same codebase.
- It stores the entire project history in a local repository.
- It is fast, lightweight and branch-based.

HOW GIT WORKS?

1. Working Directory	2. Staging Area	3. Local Repository	4. Remote Repository	5. Others
Actual files on your local machine that you are working on.	Changes are added here before committing.	Commits are stored in the local repository.	Commits are pushed to remote repository (e.g., GitHub, Bitbucket).	Changes can be pulled from remote or discarded.

COMMON USE CASES

- Caching reference data (e.g., country codes, product list).
- Tracking processed message IDs to avoid duplicates.
- Storing user sessions or authentication tokens.
- Maintaining counters and sequence numbers.
- State management in long-running processes.
- Storing intermediate data between flows.

GIT SYSTEM COMPONENTS

Component	Description	Role
Working Directory	Files in your local file system.	Where you edit and work on files.
Staging Area	Temporary area for changes.	Prepares changes for the next commit.
Local Repository	Stores all commits and history locally.	Your project history stored on your machine.
Remote Repository	Repository hosted on a remote server.	Shared repository (GitHub, Bitbucket, GitLab, etc.).

KEY OPERATIONS

Add	Add changes from working directory to staging area.
Commit	Save staged changes to local repository with a message.
Push	Upload local commits to remote repository.
Pull	Download changes from remote repository and merge.
Branch	Create a new branch to work on features independently.
Merge	Combine changes from one branch into another.
Delete	Delete branches or files.

Example Flow

1. Add : git add file.txt
2. Commit : git commit -m "Initial commit"
3. Push : git push origin main
4. Pull : git pull origin main
5. Branch : git branch feature-x
6. Checkout : git checkout feature-x
7. Merge : git merge feature-x
8. Delete : git branch -d feature-x

GIT ARCHITECTURE

Local Machine: Working Directory ↔ Staging Area ↔ Local Repository ↔ Remote Server (Remote Repository)

Changes flow from Working Directory → Staging Area → Local Repository → Remote Repository and vice-versa.

GIT BENEFITS

- Tracks every change and maintains history.
- Enables collaboration among multiple developers.
- Allows working in parallel using branches.
- Provides backup and disaster recovery.
- Supports offline work.
- Improves code quality and traceability.
- Widely adopted and actively supported.
- Integrates well with CI/CD tools.

COMMON GIT TERMS

Term	Description	Term	Description
Repository (Repo)	A storage location for your project and its history.	Merge	Combine changes from one branch into another.
Commit	A permanent snapshot of changes.	Rebase	Reapply commits on top of another base.
Push	Send local commits to remote repository.	Fetch	Download changes from remote without merging.
Pull	Fetch and merge changes from remote repository.	Tag	Mark a specific commit with a name.
Clone	Create a copy of a remote repository.	Stash	Temporarily save changes without committing.
Branch	A parallel version of the code.	Remote	A version of repository hosted on a server.

GIT BEST PRACTICES

- Write meaningful commit messages.
- Commit small and logical changes.
- Pull latest changes before starting work.
- Use branches for new features or fixes.
- Review code before merging.
- Keep your repository clean and organized.

INTERVIEW TIPS

- What is Git and why is it used?
- What is the difference between Git and other VCS?
- Explain the Git workflow.
- What is the difference between Git pull and fetch?
- How do you resolve merge conflicts?
- What is branching and why is it useful?
- How do you undo last commit?

REAL-WORLD EXAMPLE

In a MuleSoft project, multiple developers work on different features. Git helps them collaborate without overwriting each other's code. Changes are tracked, reviewed and merged, ensuring code quality and stable releases.

It integrates with CI/CD pipelines for automated builds, tests and deployments.

KEY TAKEAWAY

Git helps developers track changes, collaborate effectively, and maintain the history of their codebase. Mastering Git is essential for modern software development and CI/CD workflows.

Use Git smartly to collaborate, version, and deliver better Mule applications!

MODULE 12

12.2 GIT COMMANDS

MuleSoft from salesforce

Git commands are used to manage your code repository. They help you create, save, track, update, and collaborate on code efficiently.

GIT COMMANDS OVERVIEW

KEY POINTS

- Git is distributed and fast.
- Most commands work locally.
- You can always undo or revert changes.
- Use meaningful commit messages.
- Commit often, push regularly.

COMMON GIT COMMANDS

1 git clone

Clones a remote repository to your local machine.

Syntax:
```
git clone <repository-url>
```
Example:
```
git clone https://github.com/user/repo.git
```

2 git init

Initializes a new Git repository in your local directory.

Syntax:
```
git init
```
Example:
```
git init
```

3 git add

Adds changes from the working directory to the staging area.

Syntax:
```
git add <file>   or   git add .
```
Example:
```
git add app.py   or   git add .
```

4 git commit

Commits the staged changes to the local repository.

Syntax:
```
git commit -m "commit message"
```
Example:
```
git commit -m "Add login feature"
```

5 git push

Pushes local commits to the remote repository.

Syntax:
```
git push <remote> <branch>
```
Example:
```
git push origin main
```

6 git pull

Fetches and merges changes from the remote repository.

Syntax:
```
git pull <remote> <branch>
```
Example:
```
git pull origin main
```

7 git fetch

Downloads changes from remote without merging.

Syntax:
```
git fetch <remote>
```
Example:
```
git fetch origin
```

8 git merge

Merges changes from one branch into current branch.

Syntax:
```
git merge <branch-name>
```
Example:
```
git merge feature/login
```

9 git rebase

Reapplies commits on top of another base.

Syntax:
```
git rebase <branch-name>
```
Example:
```
git rebase main
```

10 git stash

Saves your uncommitted changes temporarily.

Syntax:
```
git stash
```
Example:
```
git stash
```

11 git tag

Creates a tag for a specific commit.

Syntax:
```
git tag <tag-name>
```
Example:
```
git tag v1.0.0
```

12 git status

Shows the status of changes in working directory and staging area.

Syntax:
```
git status
```
Example:
```
git status
```

13 git log

Displays commit history.

Syntax:
```
git log
```
Example:
```
git log --oneline --graph
```

14 git diff

Shows differences between changes, commits or branches.

Syntax:
```
git diff
```
Example:
```
git diff HEAD
```

15 git branch

Lists, creates or deletes branches.

Syntax:
```
git branch                (list branches)
git branch <name>         (create branch)
git branch -d <name>      (delete branch)
```
Example:
```
git branch feature/login
```

COMMON REMOTES

Remote Name	Description
origin	Default name for remote repository.
upstream	Usually refers to the original repo you forked.

UNDO & REVERT

git restore <file>	Discard changes in working directory.
git restore --staged <file>	Unstage a file (remove from staging area).
git reset --soft <commit>	Move HEAD, keep changes staged.
git reset --mixed <commit>	Move HEAD, unstage changes.
git reset --hard <commit>	Move HEAD, discard all changes.
git revert <commit>	Create a new commit that undoes changes.

BEST PRACTICES

- Commit small and meaningful changes.
- Write clear and descriptive commit messages.
- Pull latest changes before pushing.
- Use branches for new features or fixes.
- Review and test before merging.

KEY TAKEAWAY Mastering Git commands helps you manage code efficiently, collaborate with teams, and maintain a clean and reliable codebase.

Practice regularly, use branches wisely, and keep your repository clean and organized!

MODULE 12

12.3 GITHUB

MuleSoft from salesforce

GitHub is a web-based platform for version control and collaboration using Git. It helps developers host code, review changes, manage projects, and build better software together.

WHAT IS GITHUB?

- GitHub is a cloud-based Git repository hosting service.
- It provides a web interface for collaboration.
- It supports pull requests, code review, project management, and CI/CD.
- It integrates with many tools and services.

GitHub

HOW GITHUB WORKS

Developer	Push to GitHub	Collaborate	Merge	Deploy
Writes code locally	Push code to remote repository	Create Pull Request, Review & Discuss	Code is merged to main branch	Build, Test & Deploy

KEY FEATURES

- Repository hosting (public & private)
- Branching and merging
- Pull requests and code reviews
- Issues and project management
- Actions for CI/CD automation
- Wiki and Documentation
- Security and access control
- Integrations with tools and services

GITHUB REPOSITORY

A repository (repo) is where your project lives on GitHub.

- Can be Public (visible to everyone) or Private (restricted access)
- Contains all project files and revision history
- Can have multiple branches

REPOSITORY VISIBILITY

Type	Description
Public	Anyone on the internet can view the repository.
Private	Only you and people you authorize can view.

COMMON GITHUB WORKFLOW

1. Create / Clone	2. Create Branch	3. Make Changes	4. Push Changes	5. Open Pull Request	6. Review & Discuss	7. Merge
Create a new repo or clone existing repository.	Create a new branch for your feature or fix.	Make changes in your code and commit locally.	Push your branch to GitHub.	Create a Pull Request to merge changes into main branch.	Team reviews the code, discusses and suggests changes.	Once approved, merge the Pull Request.

IMPOTTANT GITHUB CONCEPTS

Concept	Description
Repository	A project folder that contains all files and history.
Branch	A separate line of development.
Commit	A snapshot of your changes.
Pull Request (PR)	A request to merge changes from one branch to another.
Issue	Used to track tasks, bugs or feature requests.
Fork	A copy of a repository under your GitHub account.
Clone	A copy of a repository on your local machine.

COMMON GITHUB ACTIONS

New Repository	Create a new repository.
Fork	Create a personal copy of a repository.
Watch	Receive notifications about updates.
Star	Bookmark a repository.
Branch	Create a new branch.
Pull Request	Propose and discuss changes.
Merge	Merge changes into target branch.

GITHUB ACCESS LEVELS

Role	Permissions
Read	Can view and clone the repository.
Triage	Can manage issues and pull requests.
Write	Can push to repository and create branches.
Maintain	Can manage repository settings and collaborators.
Admin	Full access including danger zone operations.

GITHUB INTEGRATIONS

- GitHub Actions – CI/CD automation
- Slack – Notifications
- Jira – Issue tracking
- SonarQube – Code quality
- Docker – Container registry
- AWS / Azure – Cloud deploy

GITHUB ACTIONS (CI/CD)

GitHub Actions allows you to automate your software workflow right in your repository.

- Build, test and deploy automatically on every push or PR.
- Supports Linux, Windows and macOS runners.
- Uses YAML workflow files.

GITHUB BEST PRACTICES

- Write meaningful commit messages.
- Use branches for new features or fixes.
- Keep branches small and focused.
- Review code through Pull Requests.
- Keep your repository clean and organized.
- Enable security features and protect main branch.
- Use GitHub Actions for automation.

COMMON GITHUB CLI COMMANDS (USING GIT)

Command	Description
`git clone <url>`	Clone a repository from GitHub.
`git remote add origin <url>`	Add a remote repository.
`git push -u origin <branch>`	Push branch to GitHub.
`git pull origin <branch>`	Pull latest changes from GitHub.
`git branch`	List all branches.
`git checkout -b <branch>`	Create and switch to a new branch.
`git add .`	Stage all changes.
`git commit -m "message"`	Commit staged changes.
`git push`	Push committed changes to GitHub.

INTERVIEW TIPS

- What is GitHub and how is it different from Git?
- What is a repository?
- What is a branch and why is it used?
- What is a Pull Request?
- How does GitHub Actions work?
- What is the difference between git fetch and git pull?
- How do you protect a branch in GitHub?

REAL-WORLD EXAMPLE

In a MuleSoft project, developers use GitHub to store Mule applications, share code, review changes and automate builds.

- Developer creates a feature branch.
- Pushes code to GitHub.
- Creates a Pull Request for review.
- After approval, code is merged and deployed using GitHub Actions.

KEY TAKEAWAY GitHub simplifies collaboration, code review, and automation. It is an essential tool for version control, teamwork and modern DevOps practices.

Use GitHub to collaborate, automate and deliver high quality Mule applications!

MODULE 12

12.4 BITBUCKET

MuleSoft from salesforce

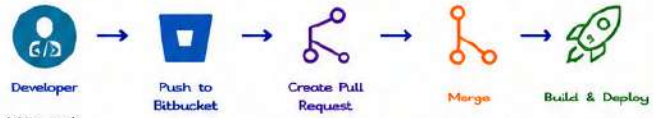
Bitbucket is a Git-based code management and collaboration platform by Atlassian. It helps teams host code, collaborate, automate builds, and deploy applications.

WHAT IS BITBUCKET?

- Git repository hosting service.
- Supports Git and Mercurial.
- Built-in CI/CD with Bitbucket Pipelines.
- Integrates with Jira, Confluence and other Atlassian tools.
- Secure and scalable for teams.

Bitbucket

HOW BITBUCKET WORKS



Developer	Push to Bitbucket	Create Pull Request	Merge	Build & Deploy
Writes code locally	Push code to remote repo	Review & discuss changes	Code merged to main branch	Automate build, test & deploy

KEY FEATURES

- Unlimited private repositories
- Branching and merging
- Pull requests and code reviews
- Built-in CI/CD (Bitbucket Pipelines)
- Jira and Confluence integration
- User and group management
- Security and access control
- Deployment environments
- Audit logs and repository insights

BITBUCKET REPOSITORY

A repository (repo) is where your project lives in Bitbucket.

- Can be Private (restricted access) or Public (visible to anyone).
- Stores all project files, commits and history.
- Supports multiple branches.
- Includes Pull Requests, Issues and Wiki.

REPOSITORY VISIBILITY

Type	Description
Private	Only users with access can view and clone the repository.
Public	Anyone on the internet can view the repository.

COMMON BITBUCKET WORKFLOW

1. Create Repository	2. Create Branch	3. Make Changes	4. Push Changes	5. Open Pull Request	6. Review & Approve	7. Merge	8. Build & Deploy
Create a new repo on Bitbucket.	Create a branch for new work.	Make changes and commit locally.	Push your branch to Bitbucket.	Create PR to merge changes to target branch.	Team reviews, comments and approves the Pull Request.	Merge PR into target branch.	Pipelines build, test and deploy code.

IMPORTANT BITBUCKET CONCEPTS

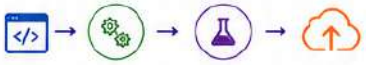
Concept	Description
Repository	A project folder that contains all files and version history.
Branch	A separate line of development.
Commit	A snapshot of your changes.
Pull Request (PR)	A request to merge changes from one branch to another.
Issue	Used to track tasks, bugs or feature requests.
Fork	A copy of a repository under your account.
Clone	A copy of a repository on your local machine.

BITBUCKET PERMISSIONS

Permission	Description
Read	Can clone and view the repository.
Write	Can push code and create branches.
Admin	Can manage repository settings, branches and permissions.
Repo Admin	Full control over repository including danger zone actions.

BITBUCKET PIPELINES (CI/CD)

- Built-in CI/CD tool for automation.
- Uses bitbucket-pipelines.yml file.
- Runs on Linux Docker containers.
- Supports parallel steps, caching and deployment.
- Integrates with AWS, Azure, GCP and other platforms.



Code → Build → Test → Deploy

BITBUCKET INTEGRATIONS

- Jira – Issue tracking
- Confluence – Documentation
- Slack – Notifications
- Bamboo – CI/CD (legacy)
- Docker – Container registry
- AWS / Azure / GCP – Cloud deployment

COMMON BITBUCKET CLI COMMANDS (USING GIT)

Command	Description
git clone <url>	Clone a repository from Bitbucket.
git remote add origin <url>	Add a remote repository.
git push -u origin <branch>	Push branch to Bitbucket.
git pull origin <branch>	Pull latest changes from Bitbucket.
git branch	List all branches.
git checkout -b <branch>	Create and switch to a new branch.
git add .	Stage all changes.
git commit -m "message"	Commit staged changes.
git push	Push committed changes to Bitbucket.

BITBUCKET BEST PRACTICES

- Write meaningful commit messages.
- Create feature branches and open PRs.
- Review code before merging.
- Keep branches short-lived and clean.
- Enable branch permissions and restrictions.
- Use Pipelines for automated builds and tests.
- Keep integrations and secrets secure.

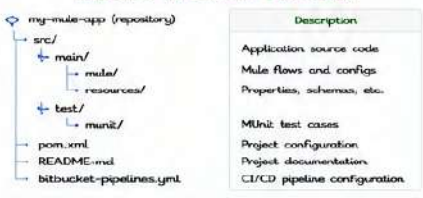
INTERVIEW TIPS

- What is Bitbucket and its key features?
- Difference between GitHub and Bitbucket?
- What is a repository and how does it work?
- What is a Pull Request and why is it used?
- How do Bitbucket Pipelines work?
- How do you restrict access to a repository?
- How do you integrate Bitbucket with Jira?

REAL-WORLD EXAMPLE

In a MuleSoft project, developers use Bitbucket to store Mule applications, create feature branches, raise Pull Requests for review and use Pipelines to run MUnit tests, build artifacts and deploy to CloudHub or RTF environments.

BITBUCKET REPOSITORY STRUCTURE



Description
Application source code
Mule flows and configs
Properties, schemas, etc.
MUnit test cases
Project configuration
Project documentation
CI/CD pipeline configuration

KEY TAKEAWAY

Bitbucket provides a secure and powerful platform for Git-based collaboration, code review, automation and deployment, making it ideal for enterprise integration projects.

Use Bitbucket to collaborate better, automate faster, and deliver high quality Mule applications!

MODULE 12

12.5 MAVEN

MuleSoft from salesforce

Maven is a build automation and project management tool used primarily for Java projects.
It simplifies the build process, manages dependencies, and provides a standard project structure.

WHAT IS MAVEN?

- Build automation tool.
- Manages project build, reporting and documentation.
- Manages dependencies.
- Standard project structure.
- Extensible using plugins.
- Used widely in Java and MuleSoft projects.

Apache Maven™

HOW MAVEN WORKS

1. pom.xml — Project object model file that contains project configuration.
2. Build Lifecycle — Maven executes phases in order to build the project.
3. Plugins — Plugins perform tasks during different phases.
4. Dependencies — Maven downloads and manages project dependencies.
5. Build Output — Generates output such as JAR, WAR, reports, etc.

PROJECT STRUCTURE

```
my-mule-app/
├── src/
│   ├── main/
│   │   ├── mule/
│   │   └── resources/
│   └── test/
│       └── munit/
├── target/
├── pom.xml
└── README.md
```

Directory / File	Description
src/main/mule	Mule flows and configs
src/main/resources	Properties, schemas, etc.
src/test/munit	MUnit test cases
target	Compiled output and packaged artifacts
pom.xml	Project Object Model (configuration file)
README.md	Project documentation

POM.XML

The pom.xml (Project Object Model) is the heart of any Maven project.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-mule-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>
</project>
```

Element	Description
groupId	Unique identifier for the organization.
artifactId	Unique name of the project.
version	Version of the project.
packaging	Type of packaging (jar, war, etc.).
dependencies	List of required dependencies.
build	Build configuration (plugins, resources, etc.).

DEPENDENCIES

Dependencies are libraries that your project needs to compile and run. Maven downloads them automatically.

```
<dependencies>
  <dependency>
    <groupId>org.mule.modules</groupId>
    <artifactId>mule-http</artifactId>
    <version>1.6.10</version>
  </dependency>
</dependencies>
```

Scope	Description
compile	Default scope. Available everywhere.
provided	Provided by JDK or container at runtime.
runtime	Needed at runtime but not for compilation.
test	Only for test compilation and run.

BUILD LIFECYCLE

Maven has three built-in lifecycles.
The default lifecycle is used most commonly.

Lifecycle	Purpose
clean	Cleans up the project.
default	Builds and deploys the project.
site	Generates project documentation.

DEFAULT LIFECYCLE PHASES

validate	Validates the project structure.
compile	Compiles the source code.
test	Runs the unit tests.
package	Packages the compiled code.
verify	Runs checks on results.
install	Installs the package to local repo.
deploy	Deploys the package to remote repo.

COMMON MAVEN COMMANDS

Command	Description
mvn clean	Cleans the target directory.
mvn compile	Compiles the source code.
mvn test	Runs the unit tests.
mvn package	Packages the application.
mvn install	Installs the package in local repository.
mvn deploy	Deploys the package to remote repository.
mvn clean install	Cleans and installs the package.

POPULAR MAVEN PLUGINS

Plugin	Purpose
maven-compiler-plugin	Compiles the source code.
maven-surefire-plugin	Runs unit tests.
maven-jar-plugin	Creates JAR files.
maven-deploy-plugin	Deploys artifacts.
maven-site-plugin	Generates documentation.

MAVEN REPOSITORIES

Repository	Description
Local Repository	Stored on your machine. (~/.m2/repository)
Central Repository	Central Maven repository (https://repo.maven.apache.org/maven2)
Remote Repository	Third-party or private repositories.

MAVEN BEST PRACTICES

- Keep pom.xml clean and organized.
- Use latest stable plugin and dependency versions.
- Avoid duplicate dependencies.
- Use meaningful artifact and package names.
- Run mvn clean install before committing.
- Use dependency scope properly.
- Keep repositories secure and private.

INTERVIEW TIPS

- What is Maven and why is it used?
- What is pom.xml?
- Explain Maven lifecycles and phases.
- What are Maven scopes?
- How does Maven resolve dependencies?
- How to create a Maven project?
- How to deploy a Maven project?
- What is the difference between clean and install?

REAL-WORLD EXAMPLE

In a MuleSoft project, Maven is used to:

- Manage Mule and third-party dependencies.
- Build and package Mule applications (JAR/WAR).
- Run MUnit tests.
- Deploy artifacts to Nexus or Anypoint Exchange.
- Automate builds using CI/CD tools.

KEY TAKEAWAY

Maven standardizes the build process, manages dependencies and plugins, and makes project management easy and consistent across teams.

Use Maven to build smarter, manage dependencies effortlessly, and deliver better Mule applications!

MODULE 12

12.6 JENKINS

MuleSoft from salesforce

Jenkins is an open-source automation server used to build, test and deploy applications continuously. It supports CI/CD by automating the integration and delivery of code changes.

WHAT IS JENKINS?

- Open-source automation server.
- Used for Continuous Integration and Continuous Delivery.
- Helps automate build, test and deployment processes.
- Supports plugins to extend functionality.
- Platform independent and easy to configure.

Jenkins

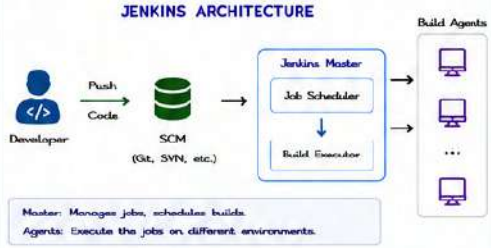
HOW JENKINS WORKS

1. Code Commit — Developer commits code to SCM.
2. Build Trigger — Jenkins is triggered automatically.
3. Build & Test — Jenkins builds the code and runs tests.
4. Deploy — Artifacts are deployed.
5. Report — Results are generated.

KEY FEATURES

- Easy installation and configuration
- Large plugin ecosystem
- Pipeline support for CI/CD
- Distributed builds
- Integrates with many tools
- Role-based access control
- Extensive reporting and notifications

JENKINS ARCHITECTURE



Master: Manages jobs, schedules builds.
Agents: Execute the jobs on different environments.

POPULAR USE CASES

- Automate build and test processes
- Continuous Integration and Continuous Delivery
- Automate deployments
- Nightly builds
- Code quality checks
- Automated testing and reports

JOB TYPES

Job Type	Description
Freestyle Project	The simplest type. A sequence of build steps.
Pipeline	Defines build steps as code using Jenkins Pipeline.
Multi-branch Pipeline	Automatically creates pipelines for branches.
Folder	Organizes jobs into folders.
Multiconfiguration Project	Builds with different configurations.

PIPELINE TYPES

Type	Description
Declarative Pipeline	Easy and structured syntax (Jenkinsfile with pipeline {}).
Scripted Pipeline	Flexible syntax using Groovy (Jenkinsfile with nodes).

JENKINSFILE (PIPELINE AS CODE)

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the code'
                sh 'mvn clean package'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests'
                sh 'mvn test'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application'
                sh './deploy.sh'
            }
        }
    }
}
```

BUILD & DEPLOY PROCESS

1. Code Commit — Code is pushed to repository
2. Webhook Trigger — Jenkins is triggered automatically.
3. Build — Compile code and create artifacts.
4. Test — Run automated tests.
5. Deploy — Deploy to target environment.
6. Report — Generate build reports.

PLUGIN ECOSYSTEM

Jenkins has 1800+ plugins to extend its capabilities.

Popular Plugins:

- Git Plugin
- Pipeline Plugin
- Maven Integration Plugin
- Blue Ocean
- Email Extension Plugin
- Slack Notification Plugin

NOTIFICATIONS

Notify teams about build status.

Supported channels:

- Email
- Slack
- Microsoft Teams
- Telegram
- Webhook

USER MANAGEMENT & SECURITY

Secure your Jenkins instance with:

- User authentication (Jenkins / LDAP)
- Role-based access control
- API tokens for integration
- Matrix Authorization Strategy

DISTRIBUTED BUILDS

Jenkins can distribute builds across multiple agents.

- Faster builds
- Load balancing
- Different environments
- Isolated builds

COMMON JENKINS CLI COMMANDS (jenkins-cli.jar)

Command	Description
java -jar jenkins-cli.jar -s <url> list-jobs	List all jobs
java -jar jenkins-cli.jar -s <url> build <job>	Trigger a build
java -jar jenkins-cli.jar -s <url> console <job> <build#>	View console output
java -jar jenkins-cli.jar -s <url> stop-build <job> <build#>	Stop a running build
java -jar jenkins-cli.jar -s <url> who-am-i	Current user info
java -jar jenkins-cli.jar -s <url> enable-job <job>	Enable a job
java -jar jenkins-cli.jar -s <url> disable-job <job>	Disable a job

INTERVIEW TIPS

- What is Jenkins and why is it used?
- Explain Jenkins architecture.
- What are different types of jobs in Jenkins?
- What is Jenkins Pipeline?
- What is the difference between freestyle project and pipeline?
- How do you integrate Jenkins with Git?
- How do you secure Jenkins?
- How do you configure email notifications?

REAL-WORLD EXAMPLE

In a MuleSoft project, Jenkins is used to:

- Pull code from GitHub.
- Build Mule application using Maven.
- Run MUnit tests.
- Package the application (JAR/WAR).
- Deploy to CloudHub or Runtime Fabric.
- Notify team about build status.

KEY TAKEAWAY

Jenkins automates the entire software delivery lifecycle. With Pipelines, plugins and integrations, it helps teams deliver high-quality software faster and more reliably.

Automate more. Deliver faster. Jenkins makes CI/CD simple and powerful!

MODULE 12

12.7 GITHUB ACTIONS

MuleSoft from salesforce

GitHub Actions is a CI/CD platform built into GitHub. It allows you to automate your software workflows directly in your repository, from testing and building to deploying your applications.

WHAT IS GITHUB ACTIONS?

- Built-in CI/CD and automation tool in GitHub.
- Automate build, test, and deployment workflows.
- Use workflows defined in YAML files.
- Works with Linux, Windows, and macOS.
- Large marketplace of pre-built actions.
- Secure, scalable, and easy to use.

HOW GITHUB ACTIONS WORKS

1. Event — An event occurs (push, PR, etc.).
2. Workflow Triggered — Workflow defined in YAML is triggered.
3. Runner Starts — GitHub provides or uses self-hosted runner.
4. Jobs Run — Jobs and steps execute on the runner.
5. Result — Workflow completes with success or failure.

KEY FEATURES

- Event-driven automation.
- YAML-based workflow configuration.
- Matrix builds and parallel jobs.
- Reusable workflows.
- Secrets management.
- Integration with GitHub and external tools.
- Marketplace actions to save time.

WORKFLOW COMPONENTS

Component	Description
Workflow	Automated process defined in a YAML file.
Event	Activity that triggers the workflow (e.g., push, pull_request).
Job	Set of steps that run on the same runner.
Step	Individual task in a job.
Action	Reusable unit of code to perform a task.
Runner	Server that runs the workflow (GitHub-hosted or self-hosted).

EVENTS THAT TRIGGER WORKFLOWS

- push – Code pushed to a branch.
- pull_request – PR created, opened, or synchronized.
- workflow_dispatch – Manual trigger.
- schedule – Time-based schedule.
- release – Release published.
- workflow_call – Reuse another workflow.
- Other events – issues, create, delete, etc.

SAMPLE WORKFLOW (YAML)

```yaml
name: Build and Test

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build with Maven
        run: mvn -B clean package

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: app-jar
          path: target/*.jar
```

RUNNERS

GitHub-Hosted Runners

- Managed by GitHub.
- Available on Ubuntu, Windows, macOS.
- Great for most use cases.

Ubuntu · Windows · macOS

Self-Hosted Runners

- You manage and maintain.
- Run on your own infrastructure.
- Useful for private networks and specialized requirements.

SECRETS & VARIABLES

Secrets

- Store sensitive data like API keys, passwords, tokens.
- Encrypted and secure.
- Access using `${{ secrets.SECRET_NAME }}`

Variables

- Store non-sensitive configuration data.
- Access using `${{ vars.VARIABLE_NAME }}`

EXAMPLE: USE SECRET

```yaml
- name: Deploy
  run: |
    echo "Deploying..."
    ./deploy.sh ${{ secrets.DEPLOY_TOKEN }}
```

GITHUB ACTIONS WORKFLOW FLOW

1. Code Push / PR — Developer pushes code or creates PR.
2. Workflow Triggered — Workflow YAML is triggered by event.
3. Runner Provisioned — GitHub provides or uses self-hosted runner.
4. Jobs & Steps Execute — Jobs run steps using actions on the runner.
5. Artifacts / Results — Artifacts uploaded and results recorded.
6. Workflow Completed — Success or failure status reported.

POPULAR ACTIONS (EXAMPLES)

Action	Purpose
actions/checkout	Checkout your repository
actions/setup-java	Set up Java environment
actions/setup-node	Set up Node.js environment
actions/cache	Cache dependencies
actions/upload-artifact	Upload build artifacts
actions/download-artifact	Download artifacts
actions/deploy-pages	Deploy to GitHub Pages

BENEFITS

- Native integration with GitHub.
- Easy to set up and maintain.
- Automate tests, builds, and deployments.
- Support for matrix and parallel jobs.
- Large marketplace of ready-to-use actions.
- Secure secrets management.

BEST PRACTICES

- Keep workflows small and focused.
- Use specific version of actions (e.g., @v4).
- Cache dependencies to speed up builds.
- Use secrets for sensitive information.
- Use environments for deployments.
- Review and monitor workflow runs.

INTERVIEW TIPS

- What is GitHub Actions?
- How does a workflow get triggered?
- Explain jobs, steps, actions and runners.
- Difference between GitHub-hosted and self-hosted runners?
- How do you store secrets in GitHub Actions?
- How do you run matrix builds?
- How do you reuse workflows?

REAL-WORLD EXAMPLE

In a MuleSoft project, GitHub Actions can be used to:

- Run MUnit tests on every push or PR.
- Build the application using Maven.
- Package the application (JAR/WAR).
- Deploy to CloudHub or Runtime Fabric.
- Notify team via Slack/Teams on build status.

USE CASES

- CI: Run tests and builds automatically.
- CD: Deploy to environments automatically.
- Automate code quality checks.
- Schedule recurring workflows.

KEY TAKEAWAY

GitHub Actions helps you automate your software workflows right from your repository. It's flexible, powerful, and perfect for CI/CD in modern development.

Automate intelligently. Build continuously. Deliver confidently with GitHub Actions!

MODULE 12

12.8 AZURE DEVOPS

MuleSoft from salesforce

Azure DevOps is a cloud-based suite of development tools and services by Microsoft. It helps teams plan, develop, test, and deploy software efficiently, using CI/CD and agile practices.

WHAT IS AZURE DEVOPS?

- A complete DevOps platform by Microsoft.
- Supports the entire application lifecycle.
- Integrates with Git, CI/CD, Agile, and more.
- Hosted in the cloud or on-premises.
- Scalable, secure, and extensible.
- Works with any language and platform.

Azure DevOps

HOW AZURE DEVOPS WORKS

1. Plan: Plan work, track progress, and manage backlogs.
2. Code: Write code and store in Git repositories.
3. Build & Test: Build and test code using pipelines.
4. Release: Release and deploy to different environments.
5. Monitor: Monitor applications and get feedback to improve.

AZURE DEVOPS SERVICES

Azure Boards	Plan, track and discuss work.
Azure Repos	Git repositories and version control.
Azure Pipelines	CI/CD to build, test and deploy.
Azure Test Plans	Plan and track testing activities.
Azure Artifacts	Create, host and share packages.

AZURE DEVOPS ARCHITECTURE

KEY FEATURES

- End-to-end DevOps toolchain.
- Agile planning with Boards.
- Unlimited Git repositories.
- CI/CD pipelines with YAML.
- Extensive marketplace extensions.
- Security, compliance and governance.
- Integration with Azure and third-party tools.

AZURE BOARDS (AGILE PLANNING)

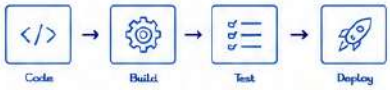
- Manage work using Kanban boards and backlogs.
- Track features, user stories, tasks, bugs.
- Supports Scrum, Kanban and custom processes.

AZURE REPOS (SOURCE CONTROL)

- Git repos with unlimited private repositories.
- Branching, pull requests and code reviews.
- Advanced file management and policies.

AZURE PIPELINES (CI/CD)

- Automate build, test and release processes.
- Define pipelines using YAML or Classic editor.
- Supports multi-stage and multi-environment deployments.



AZURE TEST PLANS

- Create test plans and test suites.
- Manual and exploratory testing.
- Track test results and bugs.
- Integrates with Azure Pipelines.

AZURE ARTIFACTS

- Create and host NuGet, Maven, npm, Python and Universal packages.
- Share packages across projects and teams.
- Use in builds and releases.

PIPELINE TYPES

Type	Description
CI Pipeline	Automates build and test on every code change.
CD Pipeline	Automates deployment to environments.
Multi-stage Pipeline	Defines stages (build, test, deploy) with approvals.
Release Pipeline	Classic release management (agents and tasks).

EXAMPLE: SIMPLE CI PIPELINE (YAML)

```yaml
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

steps:
- checkout: self
- script: |
    echo "Building the application..."
    mvn clean build
  displayName: 'Build'
- script: |
    echo "Running tests..."
    mvn test
  displayName: 'Test'
```

SECURITY & ACCESS CONTROL

- Role-based access control (RBAC).
- Permissions at organization, project and resource level.
- Integrates with Microsoft Entra ID.
- Audit logs and compliance reports.
- Secure secrets with variable groups and key vault.

INTEGRATIONS

- Integrates with Azure, GitHub, GitLab, Jenkins, Slack, ServiceNow and more.
- Extensible with extensions from Azure DevOps Marketplace.

COMMON AZURE DEVOPS CLI COMMANDS (az devops)

Command	Description
az devops login	Login to Azure DevOps.
az repos list	List repositories.
az repos create	Create a new repository.
az pipelines list	List pipelines.
az pipelines run	Run a pipeline.
az boards work-item list	List work items.
az artifacts universal publish	Publish a universal package.

INTERVIEW TIPS

- What is Azure DevOps and its components?
- How does Azure DevOps support CI/CD?
- What is the difference between CI and CD?
- What are pipelines in Azure DevOps?
- How do you manage code in Azure Repos?
- What is the role of variable groups and secrets?
- How do you secure access in Azure DevOps?

REAL-WORLD EXAMPLE

In a MuleSoft project, Azure DevOps can be used to:

- Store Mule application code in Git repositories.
- Build and test using Azure Pipelines.
- Package and publish artifacts.
- Deploy to CloudHub or Runtime Fabric.
- Track work items and sprints using Boards.

KEY TAKEAWAY

Azure DevOps provides a comprehensive platform for planning, developing, testing and deploying software. It improves collaboration, automates workflows and helps deliver high quality applications faster.

Use Azure DevOps to plan better, build faster, test continuously and deliver with confidence!

MODULE 9

MuleSoft Deployment Pipeline

MuleSoft from salesforce

A MuleSoft Deployment Pipeline is an automated process that builds, tests, and deploys Mule applications across environments in a consistent and repeatable way using CI/CD practices.

WHAT IS MULESOFT DEPLOYMENT PIPELINE?

- Automates build, test and deployment of Mule applications.
- Ensures consistent and repeatable deployments.
- Supports multiple environments (DEV, QA, STAGE, PROD).
- Reduces manual effort and deployment errors.
- Integrates with CI/CD tools and Anypoint Platform.
- Enables faster release cycles and better quality.

HOW MULESOFT DEPLOYMENT PIPELINE WORKS

1. **Code Commit** – Developer commits code to source repository.
2. **Build** – Dependencies are resolved and the application is built.
3. **Test** – Automated unit, functional and security tests run.
4. **Deploy** – Application is deployed to the target environment.
5. **Monitor** – Application is monitored for health and usage.

PIPELINE OVERVIEW

Environments

- Development (DEV)
- Quality Assurance (QA)
- Staging (STAGE)
- Production (PROD)

CI/CD Pipeline

Code Commit → Build → Test → Deploy → Monitor

Supported Deployment Targets

CloudHub 2.0 · Runtime Fabric · On-Premises · Hybrid

KEY BENEFITS

- Faster time to market.
- Improved software quality.
- Consistent deployments across environments.
- Better collaboration between teams.
- Rollback and versioning support.
- Visibility and monitoring of releases.

PIPELINE COMPONENTS

Component	Description
Source Control	Stores Mule application code and configuration.
Build Automation	Builds the application and resolves dependencies.
Automated Testing	Runs unit, functional, and security tests.
Artifact Repository	Stores build artifacts (JAR/WAR).
Deployment	Deploys application to the target environment.
Monitoring	Monitors application health and performance.

DEPLOYMENT TARGETS

Target	Description
CloudHub 2.0	Fully managed cloud platform to deploy Mule applications.
Runtime Fabric	Kubernetes-based deployment on any cloud or on-prem.
On-Premises	Deploy on customer-managed Mule runtime.
Hybrid	Combination of cloud and on-premises deployments.

MULESOFT DEPLOYMENT STRATEGIES

Strategy	Description
Blue/Green	Deploy to a parallel environment and switch traffic.
Rolling	Deploy in batches with zero downtime.
Canary	Deploy to a small subset of users first.
Recreate	Stop the old version and deploy the new version.

SAMPLE PIPELINE FLOW

1. **Code Commit** – Code is committed to Git repository.
2. **Build** – Mule application is built and packaged.
3. **Automated Tests** – Run unit, functional and security tests.
4. **Artifact Publish** – Application artifact is published to repository.
5. **Deploy to Environment** – Deploy to DEV/QA/STAGE/PROD environment.
6. **Verify & Smoke Test** – Run smoke tests to verify deployment.
7. **Monitor** – Monitor application health and logs.

MULESOFT DEPLOYMENT PIPELINE (EXAMPLE YAML)

```yaml
pipeline:
  agent:
    any: true
  stages:
  - stage: Build
    steps:
    - run: mule-artifact:build
  - stage: Test
    steps:
     - run: mule-artifact:test
  - stage: Deploy
    steps:
      - run: mule-cloudhub:deploy
        variables:
          environment: "DEV"
          applicationName: "my-mule-app"
  - stage: Verify
    steps:
      - run: mule-smoke-test:run
```

TOOLS & INTEGRATIONS

- Git (GitHub, GitLab, Azure Repos, Bitbucket)
- CI/CD Tools (Jenkins, GitHub Actions, Azure DevOps)
- Anypoint Platform (CloudHub 2.0, Runtime Fabric)
- Artifact Repositories (Nexus, Artifactory)

BEST PRACTICES

- Keep environments consistent and isolated.
- Automate tests at every stage.
- Use version control and proper branching strategy.
- Store configurations in externalized properties.
- Use secure properties and secrets management.
- Implement rollback strategy.
- Monitor, log and alert for issues.

INTERVIEW TIPS

- What is a MuleSoft Deployment Pipeline?
- What are the stages in a deployment pipeline?
- How do you deploy a Mule application to CloudHub?
- What is the difference between CloudHub and Runtime Fabric?
- How do you handle rollback in MuleSoft?
- What tools can be used for CI/CD with MuleSoft?

REAL-WORLD EXAMPLE

In a MuleSoft project, a Deployment Pipeline is used to:

- Automatically build and test Mule applications.
- Deploy to DEV for development and testing.
- Deploy to QA for validation.
- Release to STAGE for user acceptance testing.
- Deploy to PROD with zero downtime.
- Monitor application and collect feedback.

USE CASES

- Automated API and integration deployments.
- Microservices and API-led connectivity projects.
- Multi-environment enterprise integrations.
- Continuous delivery for business-critical APIs.
- Large teams with frequent releases.

KEY TAKEAWAY

A MuleSoft Deployment Pipeline automates the path from code to production, ensuring quality, consistency, and speed across all environments.

Automate. Test. Deploy. Monitor. Deliver better integrations with MuleSoft Deployment Pipeline!

MODULE 12

12.10 MUNIT AUTOMATION

MuleSoft from salesforce

MUnit is the unit testing framework for MuleSoft. It enables developers to write and run automated tests for Mule applications to ensure code quality, reliability and maintainability.

WHAT IS MUNIT?

- MuleSoft's native unit testing framework.
- Develop and run automated tests for Mule applications.
- Validates flow logic, data transformation and integration behavior.
- Supports mocking, assertions, data driven tests and coverage reporting.
- Integrates with CI/CD tools for continuous testing.

MUnit

HOW MUNIT WORKS

1. **Write Tests** — Create MUnit tests to validate flows, payloads and behaviors.
2. **Mock Dependencies** — Mock external systems, connectors and components.
3. **Run Tests** — Execute tests to validate expected results.
4. **Assert Results** — Use assertions to verify data, events and behavior.
5. **Generate Reports** — View test results and coverage reports.

MUNIT KEY FEATURES

- Easy to write and maintain tests.
- Mocking of flows, processors and connectors.
- Rich set of assertions for payload, variables, attributes and more.
- Data-driven test support.
- Code coverage analysis.
- CI/CD integration for automated builds.

MUNIT TEST ARCHITECTURE

Test Runner → MUnit Test (Setup, Execution, Validation, Tear Down) → Application → Mocks (• Flow Mock • Processor Mock • Connector Mock • Service Mock)

MUnit isolates the unit under test by mocking external dependencies and validates the actual behavior.

TYPES OF MUNIT TESTS

- **Flow Test** — Tests a specific flow.
- **API Test** — Tests a RAML API implementation.
- **DataWeave Test** — Tests DataWeave scripts and transformations.
- **Parameterized Test** — Runs the same test with multiple data sets.

SAMPLE MUNIT TEST (XML)

```
<munit:test name="getCustomerFlow-test">
  <munit:behavior>
    <munit:execution>
      <flow-ref name="getCustomerFlow"/>
    </munit:execution>
    <munit:validation>
      <munit-tools:assert-that
          expression="#[payload.id]"
          is="#[MunitTools::equalTo(1001)]" />
      <munit-tools:assert-that
          expression="#[attributes.statusCode]"
          is="#[MunitTools::equalTo(200)]"/>
    </munit:validation>
  </munit:behavior>
</munit:test>
```

COMMON ASSERTIONS

Assertion	Description
equalTo()	Validates equality.
notEqualTo()	Validates inequality.
containsString()	Checks if string contains a substring.
isNull()	Checks if value is null.
isEmpty()	Checks if collection/string is empty.
greaterThan()	Checks if value is greater.
matches()	Validates regex match.
isTrue() / isFalse()	Validates boolean values.

MOCKING IN MUNIT

Mock Type	Purpose
Flow Mock	Mocks a flow and returns a custom payload.
Processor Mock	Mocks a specific processor.
Connector Mock	Mocks external connectors (DB, HTTP, etc.)
Service Mock	Mocks external services/APIs.

DATA DRIVEN TEST

```
<munit:parameterized-test >
  <munit:parameters>
    <munit:parameter value="1001"/>
    <munit:parameter value="1002"/>
    <munit:parameter value="1003"/>
  </munit:parameters>
  <munit:behavior>
    <munit:execution>
      <flow-ref name="getCustomerFlow"/>
    </munit:execution>
    <munit:validation>
      <munit-tools:assert-that
        expression="#[payload.id]"
        is="#[vars.expectedId]"/>
    </munit:validation>
  </munit:behavior>
</munit:parameterized-test>
```

MUNIT CONFIGURATION

```
<munit:config name="MUnit_Config">
  <munit:enable-flow-sources>true</munit:enable-flow-sources>
  <munit:mock-when-no-behavior>true</munit:mock-when-no-behavior>
</munit:config>
```

- enable-flow-sources: Enables listening to flows during tests.
- mock-when-no-behavior: Auto-mocks dependencies without explicit mocks.

CODE COVERAGE

- Measures how much of the application code is covered by tests.
- Run tests with coverage to identify gaps.
- Helps improve test quality and reliability.

```
mvn test -Dmunit.coverage=true
```

MUNIT REPORTS

- HTML reports with detailed test results.
- Includes pass/fail status, execution time and coverage.
- Helps track quality over time.

MUNIT IN CI/CD PIPELINE

1. **Code Commit** — Developer commits code to repository.
2. **Build** — Build application and run MUnit tests.
3. **Test** — MUnit executes tests and validates results.
4. **Report** — Generate test and coverage reports.
5. **Deploy** — Deploy to target environment.
6. **Monitor** — Monitor application and test results.

MUIT TOOLS MODULE (munit-tools)

- Provides utilities for assertions, data generation, and test helpers.
- Example: #[MunitTools::randomInt()]
- Example: #[MunitTools::mbToBytes('10MB')]

BEST PRACTICES

- Write tests for happy path and negative scenarios.
- Mock external dependencies.
- Keep tests independent and isolated.
- Use meaningful test names and descriptions.
- Maintain high code coverage.
- Integrate tests in CI/CD for early feedback.

INTERVIEW TIPS

- What is MUnit?
- How do you create a MUnit test?
- How do you mock a connector in MUnit?
- What are different types of MUnit tests?
- How do you achieve data-driven testing?
- How do you generate coverage reports?

REAL-WORLD EXAMPLE

In a MuleSoft project, MUnit is used to:

- Validate business logic in flows.
- Ensure correct data transformation.
- Test API implementations.
- Prevent defects in production by early testing.

KEY TAKEAWAY — MUnit Automation ensures reliable, high-quality Mule applications through automated, repeatable and maintainable testing across the development lifecycle.

Automate tests. Improve quality. Deliver with confidence using MUnit!

MODULE 12

12.11 CI/CD BEST PRACTICES

MuleSoft from salesforce

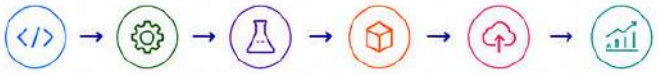
Following CI/CD best practices helps teams deliver high-quality software faster, more reliably and with less risk. These practices improve automation, security, collaboration and feedback.

WHAT IS CI/CD?

CI/CD is a set of practices that automate the integration, testing and delivery of code changes.

- CI (Continuous Integration): Frequently merge code changes and run automated tests.
- CD (Continuous Delivery/Deployment): Automatically deliver and deploy the changes to users.

CICD PIPELINE STAGES



1. Code — Commit changes to repository
2. Build — Compile code and resolve dependencies
3. Test — Run automated tests (unit, integration, etc.)
4. Package — Package and store artifacts
5. Deploy — Deploy to environments automatically
6. Monitor — Monitor performance and get feedback

CI/CD BEST PRACTICES

1. AUTOMATE EVERYTHING

- Automate build, test, security checks, deployments and infrastructure provisioning.
- Minimize manual steps and human intervention.

2. BUILD ONCE, DEPLOY MANY

- Build artifacts once and promote the same artifact through environments.
- Ensures consistency and traceability.

3. KEEP IT SMALL

- Make small, frequent and incremental changes.
- Smaller changes are easier to test, review and roll back.

4. TEST EARLY AND OFTEN

- Shift testing left. Run unit tests, integration tests and API tests early in the pipeline.
- Catches defects early and reduces cost.

5. MAINTAIN A FAST PIPELINE

- Optimize build and test times.
- Use parallel execution and efficient caching.
- Fast feedback drives productivity.

6. ENSURE QUALITY & SECURITY

- Integrate static code analysis, dependency scanning and vulnerability checks.
- Make security a continuous activity (Shift Left Security).

7. USE INFRASTRUCTURE AS CODE

- Define and manage infrastructure using code (IaC).
- Ensure repeatability, consistency and version control.

8. MANAGE SECRETS SECURELY

- Store secrets in secure vaults.
- Never hardcode credentials in code or pipelines.
- Rotate secrets regularly.

9. USE VERSION CONTROL

- Keep all code, configurations, pipeline definitions in version control.
- Enable traceability and collaboration.

10. DEPLOY AUTOMATICALLY & SAFELY

- Automate deployments with approval gates.
- Use strategies like Blue/Green, Canary or Rolling deployments.

11. MONITOR & GET FEEDBACK

- Monitor applications, pipelines and infrastructure.
- Collect logs, metrics and alerts.
- Use feedback to improve continuously.

12. HANDLE FAILURES EFFECTIVELY

- Fail fast and provide clear error messages.
- Enable automatic rollback and easy recovery.
- Analyze failures and fix root causes.

13. STANDARDIZE & REUSE

- Standardize templates, pipeline steps and scripts.
- Reuse shared libraries and components.
- Ensures consistency and saves time.

14. COLLABORATE & COMMUNICATE

- Promote collaboration across Dev, QA, Ops and Security.
- Share knowledge and improve processes together.
- CI/CD is a team sport.

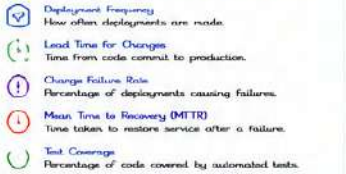
15. CONTINUOUS IMPROVEMENT

- Review and improve pipelines regularly.
- Measure metrics (e.g., lead time, failure rate, deployment frequency).
- Strive for better efficiency and quality.

ENABLERS FOR SUCCESSFUL CI/CD

- **Strong Culture** — Embrace automation, ownership and learning.
- **Right Tools** — Choose tools that fit your needs and integrate well.
- **Reliable Environments** — Use consistent, isolated and reproducible environments.
- **Clear Processes** — Define and document repeatable CI/CD processes.
- **Governance & Compliance** — Ensure policies, audits and compliance requirements.
- **Training & Upskilling** — Invest in learning and skill development.

COMMON METRICS TO TRACK



- **Deployment Frequency** — How often deployments are made.
- **Lead Time for Changes** — Time from code commit to production.
- **Change Failure Rate** — Percentage of deployments causing failures.
- **Mean Time to Recovery (MTTR)** — Time taken to restore service after a failure.
- **Test Coverage** — Percentage of code covered by automated tests.

TIPS FOR MULESOFT PROJECTS

- Automate MUnit tests and API tests in pipeline.
- Use consistent Anypoint Platform environments.
- Store configurations in externalized properties.
- Use DataWeave and API mocking for reliable tests.
- Monitor APIs with Anypoint Monitoring.
- Use Exchange assets and reusable components.

SAMPLE TECH STACK

Source Control	GitHub / Azure Repos / Bitbucket
CI/CD	GitHub Actions / Azure DevOps / Jenkins
Build & Test	Maven / MUnit / Anypoint CLI
Artifact Repository	Nexus / Artifactory / Azure Artifacts
Cloud / Runtime	CloudHub 2.0 / Runtime Fabric / On-Premises
Monitoring	Anypoint Monitoring / Datadog / New Relic

KEY TAKEAWAY

Apply these CI/CD best practices to build reliable, secure and high-quality Mule applications and deliver value to customers faster.

Automate. Test. Deploy. Monitor. Improve. Deliver better, every time!

MODULE 12

12.12 STEPS TO DO THE DEPLOYMENT THROUGH CI/CD

MuleSoft from salesforce

CI/CD automates the build, test and deployment of Mule applications to deliver changes faster and more reliably. Follow these steps to deploy a Mule application through CI/CD.

OVERVIEW OF CI/CD DEPLOYMENT FLOW

1. Code Commit — Developer commits code to repository
2. Build — Compile code and resolve dependencies
3. Test — Run automated tests (unit, integration, API)
4. Package — Create deployable artifact (jar/zip)
5. Deploy — Deploy to target environment
6. Monitor — Monitor health and get feedback

DETAILED STEPS

1 CODE COMMIT

- Developer writes or updates the Mule application.
- Commits the code to a version control system (GitHub, Bitbucket, Azure Repos, etc.).
- Triggers the CI pipeline automatically.

Example

```
git push origin main
```

Key points

- Use feature branches and Pull Requests.
- Follow code review process.

2 BUILD

- CI server checks out the code.
- Builds the project using Maven.
- Downloads dependencies and compiles the code.

Example (Maven)

```
mvn clean package
```

Key points

- Fail fast on build errors.
- Use specific Java and Mule versions.

3 TEST

- Run automated tests.
- Includes unit tests (MUnit), integration tests, API tests, and security scans.
- Generate test reports.

Example (MUnit)

```
mvn test
```

Key points

- Maintain high test coverage.
- Mock external systems for reliable tests.

4 PACKAGE

- Package the application into a deployable artifact (jar/zip).
- Store the artifact in an artifact repository (Nexus, Artifactory, Azure Artifacts, etc.).

Example

```
mvn package
```

Key points

- Use versioning for artifacts.
- Keep artifacts immutable.

5 DEPLOY

- Deploy the artifact to the target environment (DEV/QA/STAGE/PROD).
- Use deployment tools (Anypoint Platform, CloudHub 2.0, Runtime Fabric, or On-Prem).

Examples

- CloudHub 2.0: Deploy via Anypoint CLI / API / Maven plugin
- Runtime Fabric: Deploy to Kubernetes
- On-Prem: Deploy to Mule Runtime

Key points

- Promote the same artifact across environments.
- Use deployment strategies (Blue/Green, Rolling, Canary).

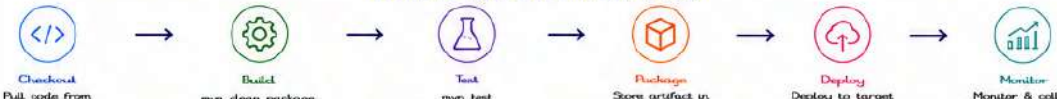
6 MONITOR

- Monitor application health and performance.
- Use Anypoint Monitoring, logs and alerts.
- Collect feedback and metrics.

Key points

- Set up alerts and dashboards.
- Track SLA, errors and business metrics.
- Continuously improve based on feedback.

EXAMPLE CI/CD PIPELINE (JENKINS)



- Checkout — Pull code from repository
- Build — mvn clean package
- Test — mvn test
- Package — Store artifact in repository
- Deploy — Deploy to target environment
- Monitor — Monitor & collect feedback

BEST PRACTICES

- Automate everything.
- Keep builds fast and reliable.
- Run tests early and often.
- Deploy the same artifact across environments.
- Use version control and branching strategy.
- Secure credentials and configurations.
- Monitor and alert proactively.
- Continuously improve the pipeline.

TOOLS COMMONLY USED

Source Control	GitHub, Bitbucket, Azure Repos
CI Server	Jenkins, GitHub Actions, Azure DevOps
Build Tool	Maven
Testing	MUnit, Postman, Newman
Artifact Repository	Nexus, Artifactory, Azure Artifacts
Deployment	Anypoint Platform, Runtime Fabric, CLI
Monitoring	Anypoint Monitoring, Datadog, New Relic

KEY BENEFITS

- Faster time to market
- Improved quality and consistency
- Reduced manual errors
- Better collaboration and visibility
- Faster rollback and recovery

KEY TAKEAWAY — A CI/CD pipeline automates code delivery from commit to production, ensuring speed, quality, security and reliability.

Automate. Test. Deploy. Monitor. Repeat. Deliver value continuously with CI/CD!

MODULE 13

13.1 MUNIT FUNDAMENTALS

MuleSoft from salesforce

MUnit is MuleSoft's official testing framework used to write automated unit and integration tests for Mule applications. It helps developers validate flows, mock external systems, verify processors, and improve application quality before deployment.

1. WHAT IS MUNIT?

- Official testing framework for Mule 4
- Used for Unit Testing and Integration Testing
- Automates application testing
- Supports mocking, spying and assertions
- Generates detailed test reports
- Integrates with Maven and CI/CD pipelines

MUnit

2. WHY USE MUNIT?

- Verify application logic
- Catch bugs early
- Improve application reliability
- Automate regression testing
- Increase deployment confidence
- Generate code coverage reports

3. MUNIT TEST FLOW

Create Test
↓
Mock External Systems
↓
Execute Mule Flow
↓
Verify Processors
↓
Assertions
↓
Coverage Report

4. MUNIT TEST STRUCTURE

munit:test

- Behavior
  - Mock When
  - Spy
- Execution
  - Flow Reference
  - Run Flow
- Validation
  - Assert
  - Verify Call

5. MUNIT TEST LIFECYCLE

1. Setup Test Data
2. Mock Dependencies
3. Execute Flow
4. Validate Output
5. Verify Calls
6. Generate Report

6. BASIC MUNIT TEST EXAMPLE

```
<munit:test name="getUserTest">

   <munit:execution>
      <flow-ref name="get-user-flow"/>
   </munit:execution>

   <munit:validation>
      <munit-tools:assert-that
        expression="#[payload]"
        is="#[MunitTools::equalTo('Success')]"/>
   </munit:validation>
</munit:test>
```

7. MUNIT COMPONENTS

Component	Purpose
Test	Defines test case
Behavior	Mock external calls (Mock When, Spy)
Execution	Executes Mule Flow
Validation	Assertions on payload, variables, etc.
Verify Call	Checks that a processor was called
Spy	Inspect payload, attributes, variables
Before Test	Runs before the test (setup)
After Test	Runs after the test (cleanup)

8. WHAT CAN MUNIT TEST?

- HTTP Requests
- Database Operations
- SalesForce APIs
- JMS Queue
- File Connector
- FTP / SFTP
- Object Store
- Batch Jobs
- DataWeave Transformations
- Error Handling

9. MUNIT PROJECT STRUCTURE

```
src
├── main
│   └── mule
└── test
    ├── munit
    ├── resources
    └── mock-data
```

10. MAVEN COMMANDS

Run all tests	`mvn test`
Generate Coverage Report	`mvn clean test`
Package Application	`mvn clean package`

11. BEST PRACTICES

- Test one flow at a time
- Mock external systems
- Write independent test cases
- Use meaningful test names
- Verify important processors
- Assert payload and variables
- Maintain high code coverage
- Store test data separately

12. INTERVIEW QUESTIONS

- What is MUnit?
- Why do we use MUnit?
- Difference between Unit Testing and Integration Testing?
- What are Behavior, Execution and Validation?
- What is Mock When?
- What is Spy?
- What is Verify Call?
- How do you generate MUnit reports?
- How do you run MUnit using Maven?
- Can MUnit test DataWeave transformations?

13. REAL-WORLD SCENARIO

HTTP Request → Mule Application → Database, Salesforce, JMS Queue

A Mule application receives an HTTP request and invokes Database, Salesforce, and JMS Queue. Instead of calling real systems, MUnit mocks each connector, executes the flow, verifies processor execution, validates the response payload, and generates a coverage report.

14. SAMPLE MUNIT REPORT (CONCEPT)

Run	Failures	Errors	Coverage
25	0	0	92%

Coverage by Flow

get-user-flow	100%
create-user-flow	85%
update-user-flow	90%
delete-user-flow	95%

All tests passed successfully!

15. KEY TAKEAWAY

MUnit is the foundation of automated testing in MuleSoft. It enables developers to validate business logic, mock external dependencies, verify flow execution, and deliver reliable, production-ready integrations with confidence.

Test early. Automate always. Deliver with confidence using MUnit!

MODULE 13

13.2 MOCKING

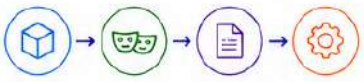
MuleSoft from salesforce

Mocking in MUnit allows you to simulate the behavior of external systems and isolate the unit under test. It helps in testing different scenarios without calling real systems.

1. WHAT IS MOCKING?

- Mocking means simulating the behavior of external dependencies.
- It helps to isolate the unit under test.
- No real calls are made to external systems.
- Improves test speed, reliability and consistency.
- Achieved in MUnit using "Mock When".

2. HOW MOCKING WORKS



3. MOCK WHEN SYNTAX

```
<munit:behavior>
  <munit-tools:mock-when processor="...">
    <munit-tools:with-attributes>
      <!-- Matching attributes -->
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <!-- Mocked response -->
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4. EXAMPLES OF MOCKING DIFFERENT CONNECTORS

4.1 MOCK HTTP REQUEST

```
<munit:behavior>
  <munit-tools:mock-when processor="http:request">
    <munit-tools:with-attributes>
      <munit-tools:where value="#[attributes.method == 'GET'
          and attributes.path == '/api/users']"/>
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <munit-tools:payload value="{{"id":1,"name":"John"}}"/>
      <munit-tools:attributes value="#[{
        statusCode: 200,
        headers: {"content-type":"application/json"}
        }]"/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4.2 MOCK DATABASE SELECT

```
<munit:behavior>
  <munit-tools:mock-when processor="db:select">
    <munit-tools:with-attributes>
      <munit-tools:where value="#[attributes.query
          contains 'SELECT * FROM USERS']"/>
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <munit-tools:payload value='[
        {"ID":1,"NAME":"John","AGE":30}
      ]"/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4.3 MOCK SALESFORCE

```
<munit:behavior>
  <munit-tools:mock-when processor="sfdc:query">
    <munit-tools:with-attributes>
      <munit-tools:where value="#[attributes.query
          contains 'SELECT Id, Name FROM Account']"/>
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <munit-tools:payload value="#[
        { records: [ { Id:"001xx000003DHP", Name:"Acme" } ] }
      ]"/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4.4 MOCK FILE READ

```
<munit:behavior>
  <munit-tools:mock-when processor="file:read">
    <munit-tools:with-attributes>
      <munit-tools:where value="#[attributes.path
          == '/input/data.json']"/>
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <munit-tools:payload value='{"key":"value"}'/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4.5 MOCK JMS LISTENER

```
<munit:behavior>
  <munit-tools:mock-when processor="jms:listener">
    <munit-tools:then-return>
      <munit-tools:payload value='{"event":"order","id":100}'/>
      <munit-tools:attributes value="#[{
        JMSMessageID: "ID:12345",
        JMSCorrelationID: "CORR-100",
        JMSDeliveryMode: "PERSISTENT"
      }]"/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

4.6 MOCK OBJECT STORE

```
<munit:behavior>
  <munit-tools:mock-when processor="os:retrieve">
    <munit-tools:with-attributes>
      <munit-tools:where value="#[attributes.key == 'cust-123']"/>
    </munit-tools:with-attributes>
    <munit-tools:then-return>
      <munit-tools:payload value='{"id":"cust-123","status":"ACTIVE"}'/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

5. MATCHING ATTRIBUTES

Use with-attributes to match requests.

Type	Example
Method	#[attributes.method == 'POST']
Path	#[attributes.path == '/api/users']
Query Params	#[attributes.queryParams.id == '10']
Headers	#[attributes.headers['x-api-key'] == 'abc']
All Attributes	#[true()] (matches all)

6. THEN-RETURN OPTIONS

Define what the mock should return.

Option	Description
Payload	Return payload
Attributes	Return attributes (status, headers, etc.)
Error	Simulate an error response
Variables	Set variables on return

7. MOCK ERROR RESPONSE

Simulate error from external system.

```
<munit-tools:then-return>
  <munit-tools:error type="HTTP:NOT_FOUND"
    description="Resource not found">
    <munit-tools:attributes value="#[ statusCode: 404,
      headers: {"content-type":"application/json"} ]"/>
    <munit-tools:payload value='{"message":"Not Found"}'/>
  </munit-tools:error>
</munit-tools:then-return>
```

8. TIPS & BEST PRACTICES

- Mock only external dependencies.
- Use specific matchers to avoid false matches.
- Keep mocks simple and focused.
- Reuse common mock data using variables or external files.
- Cover all scenarios: success, failure and edge cases.

9. IMPORTANT NOTES

- Mocking is only for external systems.
- Do not mock components within the same flow.
- Mocks are defined inside <munit:behavior>.
- Order of mock-when does not matter.
- MUnit ensures the mock intercepts the call before it reaches the real connector.

10. INTERVIEW QUESTIONS

- What is mocking in MUnit?
- How do you mock an HTTP request?
- How do you mock a database call?
- What is Mock When?
- How do you simulate an error using mocks?
- Can we mock internal processors?
- Where do we define mocks in MUnit?

KEY TAKEAWAY

Mocking isolates the unit under test by simulating external systems. It makes tests faster, reliable and independent of actual systems.

Mock smart. Test fast. Deliver with confidence using MUnit!

MODULE 13

13.3 SPY

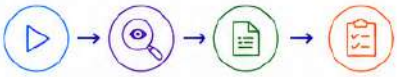
MuleSoft from salesforce

Spy in MUnit allows you to listen to and capture the events, payload, attributes and variables of a processor without mocking it. It helps in validating and asserting the actual behavior of a specific processor.

1. WHAT IS SPY?

- Spy is a test component in MUnit.
- It "spies" on a real processor and captures its behavior.
- It does not change the flow execution.
- Helps to validate payloads, attributes or variables at a specific point.
- Useful for debugging and verification.

2. HOW SPY WORKS



Flow executes normally → Spy listens to the processor → Spy captures the event data → Test validates the captured data

3. SPY SYNTAX

```
<munit:behavior>
  <munit-tools:spy processor="processor-name"
      doc:name="Spy on Processor"/>
</munit:behavior>
```

Where:
processor-name = name or id of the processor you want to spy on.

4. EXAMPLES OF SPY ON DIFFERENT PROCESSORS

4.1 SPY ON TRANSFORM MESSAGE

```
<munit:behavior>
  <munit-tools:spy
    processor="transform-message-1"
    doc:name="Spy on Transform"/>
</munit:behavior>
```

4.2 SPY ON HTTP REQUEST

```
<munit:behavior>
  <munit-tools:spy
    processor="http-request"
    doc:name="Spy on HTTP Request"/>
</munit:behavior>
```

4.3 SPY ON DB SELECT

```
<munit:behavior>
  <munit-tools:spy
    processor="db:select"
    doc:name="Spy on DB Select"/>
</munit:behavior>
```

4.4 SPY ON FLOW REFERENCE

```
<munit:behavior>
  <munit-tools:spy
    processor="flow-ref-to-other-flow"
    doc:name="Spy on Flow Ref"/>
</munit:behavior>
```

4.5 SPY ON SET VARIABLE

```
<munit:behavior>
  <munit-tools:spy
    processor="set-variable"
    doc:name="Spy on Set Variable"/>
</munit:behavior>
```

5. CAPTURING DATA WITH SPY

Spy captures the following event data:

- Message Payload
- Attributes
- Variables
- Metadata
- Event Source

6. ACCESSING SPY DATA IN VALIDATION

Spy data is stored in variables automatically.

Variable name format:

```
munitTools::spyEvent(processorName)
```

Example:

```
#[munitTools::spyEvent('transform-message-1')]
payload
```

7. VALIDATION EXAMPLE USING SPY

```
<munit:validation>
  <munit-tools:assert-that
    expression="#[munitTools::spyEvent('transform-message-1')
                .payload]"
    is="#[MunitTools::equalTo('Success')]" />
  <munit-tools:assert-that
    expression="#[munitTools::spyEvent('transform-message-1')
                .attributes.statusCode]"
    is="#[MunitTools::equalTo(200)]" />
</munit:validation>
```

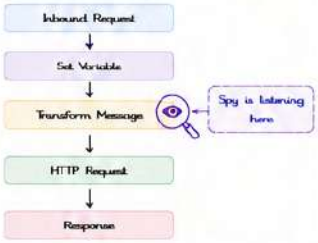
8. BEFORE & AFTER CALL (SPY EVENT)

Event	Description
Before Call	Captures event data before the processor executes.
After Call	Captures event data after the processor executes.

By default, Spy captures After Call event.
Use 'when' attribute to capture Before Call.

9. SPY FLOW

Example Flow Execution with Spy



10. SPY PROPERTIES

Property	Description
processor	Name/id of the processor to spy.
doc:name	Name of the spy for identification.
event	Type of event to capture: BEFORE_CALL or AFTER_CALL. (Default: AFTER_CALL)
attributes	Optional. Define specific attributes to capture.

11. BEST PRACTICES

- Use Spy when you want to listen without changing behavior.
- Spy only on necessary processors.
- Use meaningful spy names.
- Validate important fields (payload, attributes, variables).
- Avoid spying on every processor (affects performance).

12. COMMON USE CASES

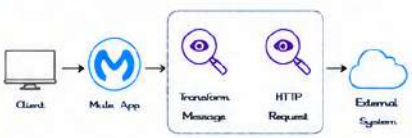
- Validate payload after transformation.
- Check headers set in HTTP Request.
- Verify database query output.
- Ensure variables are set correctly.
- Debug and analyse flow behavior.

13. INTERVIEW QUESTIONS

- What is Spy in MUnit?
- How is Spy different from Mock?
- How do you access Spy data in validation?
- What events can Spy capture?
- Can Spy change the flow behavior?
- When should we use Spy?

14. REAL-WORLD SCENARIO

A Mule application transforms an incoming request and calls an external API. Using Spy on the Transform Message and HTTP Request processors, we can validate the transformed payload and the request headers without mocking the actual calls.



15. KEY TAKEAWAY

Spy helps you observe and validate the real behavior of processors in your flow. It captures actual runtime data without affecting the flow execution, making your tests more accurate and reliable.

Listen. Validate. Ensure Quality with Spy in MUnit!

MODULE 13

13.4 VERIFY CALL

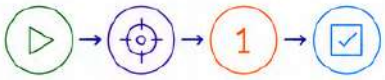
MuleSoft from salesforce

Verify Call in MUnit is used to verify that a specific processor or flow-ref was called a certain number of times with expected attributes.

1. WHAT IS VERIFY CALL?

- Verify Call is a validation tool in MUnit.
- It verifies that a specific processor or flow-ref was called.
- You can verify how many times it was called and with what attributes.
- Helps in testing the flow behavior and integration points.

2. HOW VERIFY CALL WORKS



Test executes the flow → Verify Call monitors the processor → Checks number of calls → Validates attributes (optional)

3. VERIFY CALL SYNTAX

```
<munit:validation>
  <munit-tools:verify-call
          processor="processor-name"
          times="1"
          doc:name="Verify Call" />
</munit:validation>
```

Attributes:

processor	: Name or id of the processor or flow-ref
times	: Expected number of times the processor should be called
where	: (Optional) Filter conditions

4. EXAMPLES OF VERIFY CALL

4.1 VERIFY PROCESSOR CALLED ONCE

```
<munit:validation>
  <munit-tools:verify-call
      processor="set-payload"
      times="1"
      doc:name="Verify Set Payload"/>
</munit:validation>
```

4.2 VERIFY PROCESSOR CALLED MULTIPLE TIMES

```
<munit:validation>
  <munit-tools:verify-call
      processor="logger"
      times="3"
      doc:name="Verify Logger 3 Times"/>
</munit:validation>
```

4.3 VERIFY FLOW REFERENCE CALL

```
<munit:validation>
  <munit-tools:verify-call
      processor="flow-ref-to-sub-flow"
      times="1"
      doc:name="Verify Flow Ref" />
</munit:validation>
```

4.4 VERIFY WITH WHERE CONDITION

```
<munit:validation>
  <munit-tools:verify-call
      processor="http-request"
      times="1"
      where="#[attributes.method == 'GET']"
      doc:name="Verify HTTP GET Call"/>
</munit:validation>
```

5. VERIFY CALL ATTRIBUTES

Attribute	Required	Description	Example
processor	✓	Id or name of the processor/flow-ref to verify	processor="db-select"
times	✓	Expected number of times the processor is called	times="2"
where	✗	Optional condition to filter calls	where="#[payload.id == 101]"
doc:name	✗	Optional label for the component	doc:name="Verify DB Call"

6. COMMON WHERE CONDITIONS

Condition Type	Example
Match Method	where="#[attributes.method == 'POST']"
Match Path	where="#[attributes.path == '/api/users']"
Match Query Param	where="#[attributes.queryParams.id == '10']"
Match Headers	where="#[attributes.headers['x-api-key'] == 'abc']"
Match Payload	where="#[payload.id == 100]"

7. VERIFY CALL OUTPUT EXAMPLE

```
Run: munit test
-------------------------------------------
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
-------------------------------------------
✓ getUserTest
  ✓ Verify HTTP Request (1 time)
  ✓ Verify DB Select (1 time)
  ✓ Verify Logger (2 times)

BUILD SUCCESS
```

8. WHEN TO USE VERIFY CALL?

- When you want to ensure a specific processor is executed.
- When testing error flows and choice routers.
- To verify external system calls (HTTP, DB, Salesforce, JMS, etc.).
- To check that a flow-ref is invoked.
- Along with assertions to fully validate the flow.

9. REAL-WORLD SCENARIO

A flow receives an HTTP request, validates the data, stores it in DB, and sends a success response.
Use Verify Call to ensure:

- ✓ DB Select is called once
- ✓ DB Insert is called once
- ✓ HTTP Request to external API is called once
- ✓ Success Logger is called once

10. VERIFY CALL WITH SPY (COMBO)

Use Spy to capture data and Verify Call to ensure the processor was called.

```
<munit:behavior>
  <munit-tools:spy processor="http-request" />
</munit:behavior>
<munit:validation>
  <munit-tools:verify-call processor="http-request"
      times="1" />
  <munit-tools:assert-that
      expression="#[munitTools::spyEvent('http-request')
                  .attributes.statusCode]"
      is="#[MunitTools::equalTo(200)]" />
</munit:validation>
```

11. BEST PRACTICES

- ✓ Use Verify Call only for important processors.
- ✓ Keep tests independent and focused.
- ✓ Use meaningful processor names.
- ✓ Combine with assertions for complete validation.
- ✓ Use where condition to filter specific calls.
- ✓ Avoid hardcoding values; use variables.

12. COMMON MISTAKES

- ✗ Verifying processors that may not be called in certain scenarios.
- ✗ Not using where condition when multiple calls happen.
- ✗ Relying only on payload assertions.
- ✗ Using incorrect processor name/id.

13. INTERVIEW QUESTIONS

- What is Verify Call in MUnit?
- How do you verify a processor is called once?
- How do you verify a processor multiple times?
- What is the use of 'where' attribute in Verify Call?
- Can we verify a flow-reference call?
- How is Verify Call different from Spy?

14. QUICK REFERENCE

Goal	Example
Verify processor called once	processor="set-payload" times="1"
Verify processor called N times	processor="logger" times="3"
Verify flow-ref call	processor="flow-ref-to-sub-flow" times="1"
Verify with condition	processor="http-request" times="1" where="#[attributes.method == 'GET']"

15. KEY TAKEAWAY

Verify Call ensures that your flow behaves as expected by verifying the execution of critical processors and integrations. It builds confidence, prevents regressions, and improves the reliability of your Mule applications.

Verify execution. Validate behavior. Deliver with confidence using MUnit!

MODULE 13

13.5 ASSERTIONS

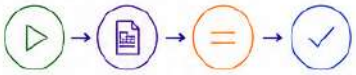
MuleSoft from salesforce

Assertions in MUnit are used to validate the actual output (payload, attributes, variables, etc.) against the expected result. If the validation fails, the test case fails.

1. WHAT ARE ASSERTIONS?

- Assertions are validation statements.
- They compare actual values with expected values.
- If the condition is true, the test passes.
- If the condition is false, the test fails.
- Used in the <munit:validation> block.

2. HOW ASSERTIONS WORK



Flow is executed → Actual value is obtained → Assertion compares values → Test Pass / Fail

3. ASSERTION SYNTAX

```
<munit:validation>
   <munit-tools:assert-that
        expression="#[actual]"
        is="#[MunitTools::matcher(expected)]"
        message="Assertion message (optional)" />
</munit:validation>
```

expression	: Actual value
is	: Matcher with expected value
message	: (Optional) Message to display on failure

4. COMMON ASSERTIONS (MATCHERS)

Matcher	Description	Example	Validates
equalTo( )	Validates equality between actual and expected value.	`<munit-tools:assert-that expression="#[payload]" is="#[MunitTools::equalTo('Success')]"/>`	Actual value is equal to expected value.
notEqualTo( )	Validates that actual value is not equal to expected value.	`<munit-tools:assert-that expression="#[payload]" is="#[MunitTools::notEqualTo('Error')]"/>`	Actual value is not equal to expected value.
isNull( )	Validates that actual value is null.	`<munit-tools:assert-that expression="#[attributes.headers.authorization]" is="#[MunitTools::isNull()]"/>`	Actual value is null.
isNotNull( )	Validates that actual value is not null.	`<munit-tools:assert-that expression="#[vars.userId]" is="#[MunitTools::isNotNull()]"/>`	Actual value is not null.
isTrue( )	Validates that actual value is true.	`<munit-tools:assert-that expression="#[vars.isActive]" is="#[MunitTools::isTrue()]"/>`	Actual value is true.
isFalse( )	Validates that actual value is false.	`<munit-tools:assert-that expression="#[vars.isDeleted]" is="#[MunitTools::isFalse()]"/>`	Actual value is false.
containsString( )	Validates that actual string contains the expected string.	`<munit-tools:assert-that expression="#[payload.message]" is="#[MunitTools::containsString('created')]"/>`	Actual string contains expected string.
matches( )	Validates that actual string matches the regex.	`<munit-tools:assert-that expression="#[payload.email]" is="#[MunitTools::matches('^[\\w]+@\\w+\\.com$')]"/>`	Actual string matches the regular expression.
greaterThan( )	Validates that actual number is greater than expected.	`<munit-tools:assert-that expression="#[vars.count]" is="#[MunitTools::greaterThan(10)]"/>`	Actual number is greater than expected number.
lessThan( )	Validates that actual number is less than expected.	`<munit-tools:assert-that expression="#[vars.count]" is="#[MunitTools::lessThan(100)]"/>`	Actual number is less than expected number.

5. DATAWEAVE ASSERTION EXAMPLE

```
<munit:validation>
    <munit-tools:assert-that
        expression="#[payload.users size()]"
        is="#[MunitTools::equalTo(3)]"
        message="Users count should be 3" />
    <munit-tools:assert-that
        expression="#[payload.users[0].name]"
        is="#[MunitTools::equalTo('John')]" />
</munit:validation>
```

6. ASSERTING COMPLEX OBJECTS

Validate complete JSON object

```
<munit-tools:assert-that
    expression="#[payload]"
    is="#[MunitTools::equalTo({
            id: '101',
            name: 'John',
            status: 'Active'
        })]" />
```

7. CUSTOM ERROR MESSAGE

```
<munit-tools:assert-that
   expression="#[payload.status]"
   is="#[MunitTools::equalTo('Success')]"
   message="Expected status as 'Success' but
            was #[payload.status]" />
```

8. ASSERTIONS IN VALIDATION BLOCK

```
<munit:test name="getUserTest">
   <munit:execution>
      <flow-ref name="get-user-flow"/>
   </munit:execution>
   <munit:validation>
      <munit-tools:assert-that
         expression="#[payload]"
         is="#[MunitTools::isNotNull()]"/>
   </munit:validation>
</munit:test>
```

9. BEST PRACTICES

- Assert important business values.
- Use meaningful assertion messages.
- Validate both positive and negative scenarios.
- Avoid too many assertions in one test.
- Use variables for expected values.

10. COMMON USE CASES

- Validate response payload
- Validate variables and attributes
- Validate status codes
- Validate headers
- Validate counts and sizes
- Validate data types and values

11. INTERVIEW QUESTIONS

- What are Assertions in MUnit?
- What is assert-that?
- How do you validate null and not null values?
- What are common matchers used in MUnit?
- Can we write custom assertion messages?

12. QUICK REFERENCE

Matcher	Purpose
equalTo( )	Equal to expected value
notEqualTo( )	Not equal to expected value
isNull( )	Value is null
isNotNull( )	Value is not null
isTrue( )	Value is true
isFalse( )	Value is false
containsString( )	String contains expected text
matches( )	String matches regex
greaterThan( )	Number greater than expected
lessThan( )	Number less than expected

13. REAL-WORLD SCENARIO

Use Assertions to validate:

- ✓ Response status should be 200
- ✓ Payload should not be null
- ✓ Response contains expected fields
- ✓ Data values are correct

14. KEY TAKEAWAY

Assertions are the heart of MUnit testing. They validate the correctness of your application by comparing actual results with expected values, ensuring reliable and error-free integrations.

Good assertions lead to reliable tests and high quality Mule applications!

MODULE 13

13.6 COVERAGE REPORT

MuleSoft from salesforce

Coverage Report in MUnit shows how much of your Mule application (flows, processors, DataWeave, error handling, etc.) is executed during tests. It helps measure test effectiveness and identify untested areas.

1. WHAT IS COVERAGE?

- Measures how much code is executed by your tests.
- Indicates test completeness and quality.
- Helps identify untested flows, processors, branches and DataWeave code.
- Generates detailed HTML report.
- Supports line, branch and element level coverage.

Coverage Report

2. HOW COVERAGE WORKS

Tests execute application components → MUnit tracks executed vs not executed → Coverage data is collected → Report is generated

3. ENABLE COVERAGE

Add the following plugin in pom.xml

```
<plugin>
   <groupId>org.mule.tools.maven</groupId>
   <artifactId>munit-maven-plugin</artifactId>
   <version>${munit.version}</version>
   <extensions>true</extensions>
   <configuration>
      <coverage>true</coverage>
   </configuration>
</plugin>
```

4. RUN TESTS WITH COVERAGE

Run the following Maven command:

```
mvn clean test
```

This will execute tests and generate the coverage report.

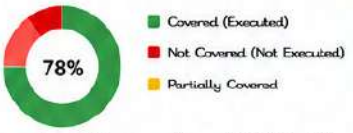
5. COVERAGE REPORT LOCATION

After test execution, the report is generated at:

```
target/munit-reports/coverage/
```

Open the file: index.html in a browser to view the report.

6. UNDERSTANDING THE REPORT



Overall Coverage = (Covered / Total) x 100

7. COVERAGE METRICS EXPLAINED

Metric	Description	What it Measures
Lines	Percentage of lines executed	Code lines executed by tests
Branches	Percentage of branches executed	If/Else, Choice, When, For Each branch coverage
Elements	Percentage of Mule elements executed	Processors, Components, Error Handlers, Flow References
DataWeave	Percentage of DataWeave code executed	DW scripts and functions executed
Overall	Combined coverage of all metrics	Overall test effectiveness

8. SAMPLE REPORT VIEW

Overall Coverage

78%

Lines: 82%
Branches: 70%
Elements: 76%
DataWeave: 85%

Covered: 340 Not Covered: 95 Total: 435

9. REPORT DETAILS (DRILL DOWN)

10. IMPROVING COVERAGE

- Write tests for all flows and sub-flows.
- Cover all processors and components.
- Test all branches (if/else, choice, when).
- Test error handling scenarios.
- Validate DataWeave transformations.
- Add negative test cases.
- Avoid hardcoding and use test data.

11. TIPS & BEST PRACTICES

- Aim for 80%+ overall coverage.
- Focus on critical business flows first.
- Cover both positive and negative paths.
- Include exception and boundary cases.
- Keep tests independent and reusable.
- Review coverage regularly.

12. COMMON EXCLUSIONS

- Generated code
- Third party libraries
- Configurations
- Mule runtime internal code
- Static resources (properties, csv, xml, json files)

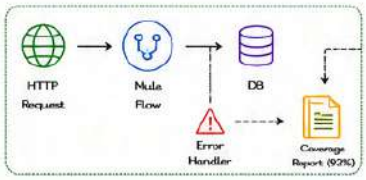
13. INTERVIEW QUESTIONS

- What is Coverage Report in MUnit?
- How do you enable coverage in MUnit?
- How do you generate the coverage report?
- What metrics are shown in coverage report?
- What is a good coverage percentage?
- How do you improve coverage?
- Where is the coverage report generated?

14. REAL-WORLD SCENARIO

A Mule application processes orders, validates data, invokes external APIs and stores data in DB.

- ✓ MUnit tests are created for all flows and scenarios.
- ✓ Coverage report shows 78% overall coverage.
- ✓ Uncovered parts include one error path and a DataWeave function.
- ✓ Additional tests are written to cover missing parts.
- ✓ Coverage improves to 92%, ensuring high quality.



15. KEY TAKEAWAY

Coverage reports help ensure that your tests are effective and your application is reliable. Higher coverage means better confidence, fewer defects and production-ready integrations.

Good coverage means confidence in your code. Test more. Cover more. Deliver better!

MODULE 13 | 13.7

TEST SUITES & TEST CASES

MuleSoft from salesforce

In MUnit, Test Suite is a collection of related test cases. Each Test Case is a single test that validates a specific behavior of the application.

1. WHAT IS TEST SUITE?

- A Test Suite groups multiple test cases.
- Helps organize tests logically.
- Executed together as a single unit.
- Can share common setup and configuration.

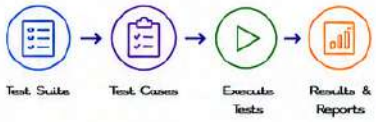
Test Suite

2. WHAT IS TEST CASE?

- A Test Case validates a specific functionality.
- Independent and repeatable.
- Contains Behavior, Execution and Validation.
- Returns Pass / Fail.

Test Case

3. TEST EXECUTION FLOW



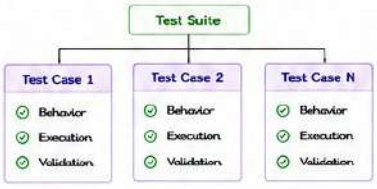
4. MUNIT TEST SUITE EXAMPLE

```
<munit:test-suite name="Order API Test Suite" description="Tests for Order APIs">
   <munit:description>
      This suite contains test cases for Order flows.
   </munit:description>
   <munit:test name="Create Order Test">
      <!-- Test Case 1 -->
   </munit:test>
   <munit:test name="Get Order Test">
      <!-- Test Case 2 -->
   </munit:test>
</munit:test-suite>
```

5. MUNIT TEST CASE EXAMPLE

```
<munit:test name="Create Order Test" description="Validate create order flow">
  <munit:behavior>
    <munit-tools:mock-when processor="http:request" doc:name="Mock Create Order API">
      <munit-tools:with-attributes>
        <munit-tools:with-attribute attributeName="method" whereValue="POST"/>
      </munit-tools:with-attributes>
      <munit-tools:then-return payload="#[{ 'status':'created' })]" />
    </munit-tools:mock-when>
  </munit:behavior>
  <munit:execution>
    <flow-ref name="create-order-flow"/>
  </munit:execution>
  <munit:validation>
    <munit-tools:assert-that expression="#[payload.status]" is="#[MunitTools::equalTo('created')]"/>
  </munit:validation>
</munit:test>
```

6. TEST SUITE STRUCTURE



7. TEST SUITE & TEST CASE PROPERTIES

Property	Level	Description	Example
name	Suite / Case	Unique name of suite or test case	name="Order Suite"
description	Suite / Case	Description of suite or test case	description="Test for orders"
runMode	Suite / Case	Controls when to run the test (default: ALWAYS)	runMode="ALWAYS"
enabled	Suite / Case	Enable/Disable suite or test case	enabled="false"
tags	Suite / Case	Group tests using tags	tags="smoke,regression"
parallel	Suite	Run test cases in parallel	parallel="true"

8. ENABLE / DISABLE TESTS

Disable a Test Case

```
<munit:test
   name="Disabled Test"
   enabled="false">
   ...
</munit:test>
```

Disable a Test Suite

```
<munit:test-suite
   name="My Suite"
   enabled="false">
   ...
</munit:test-suite>
```

9. TAGS EXAMPLE

```
<munit:test name="Create Order Test" tags="smoke,orders">
   ...
</munit:test>
```

Run only tests with tag smoke:

```
mvn test -Dmunit.tags=smoke
```

10. RUN MODE

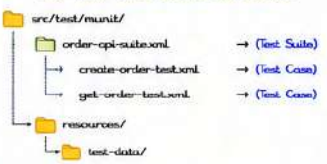
- ALWAYS : Run every time
- ON_ERROR : Run only when previous test fails
- NEVER : Never run

11. TEST DATA SHARING (AT SUITE LEVEL)

You can define variables at suite level and use them in all test cases.

```
<munit:test-suite name="User API Suite">
  <munit:config name="Suite Config">
    <munit:variables>
      <munit:variable key="baseUrl"
        value="https://api.example.com"/>
    </munit:variables>
  </munit:config>
  <!-- Test cases will use #[vars.baseUrl] -->
</munit:test-suite>
```

12. TEST SUITE FILE STRUCTURE



13. BENEFITS

- Better organization of tests.
- Reuse common configuration and test data.
- Easy maintenance.
- Faster execution with parallel mode.
- Improves readability and reporting.

14. BEST PRACTICES

- Create suites based on business domain.
- Keep test cases independent.
- Use meaningful names and descriptions.
- Use tags to group and filter tests.
- Share common setup at suite level.

15. COMMON MISTAKES

- Creating huge suites with unrelated tests.
- Hardcoding test data inside test cases.
- Ignoring tags and run modes.
- Not disabling unused tests.
- Not reviewing test reports.

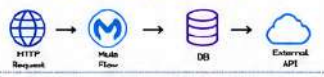
16. INTERVIEW QUESTIONS

- What is Test Suite in MUnit?
- What is Test Case in MUnit?
- How do you enable or disable a test case?
- What are run modes in MUnit?
- How do you run tests by tags?
- Can test cases run in parallel?

17. QUICK REFERENCE

Task	Command / Attribute
Run all tests	mvn test
Run by tag	mvn test -Dmunit.tags=smoke
Run specific suite	mvn test -Dmunit.suite=order-api-suite.xml
Enable / Disable test	enabled="true/false"
Set run mode	runMode="ALWAYS|ON_ERROR|NEVER"

18. REAL-WORLD SCENARIO



A team creates a Test Suite "Order API Suite" with test cases for Create Order, Get Order, Update Order and Cancel Order flows.

19. KEY TAKEAWAY

Well-structured Test Suites and Test Cases make your tests organized, reusable, maintainable and give better visibility into the quality of your Mule application.

Organized tests today, reliable integrations tomorrow!

MODULE 13

13.8 TESTING ERROR HANDLING

MuleSoft from salesforce

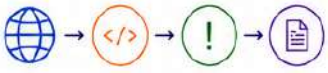
Testing Error Handling in MUnit ensures that your application behaves as expected when errors occur. It helps validate error responses, error types and custom error handling logic.

1. WHY TEST ERROR HANDLING?

- Errors are inevitable in real integrations.
- Ensures proper error response to the client.
- Validates custom error handling logic.
- Prevents unhandled errors in production.
- Improves reliability and user experience.
- Ensures correct error type and attributes.

Test Errors Early!

2. ERROR HANDLING FLOW



3. BASIC ERROR HANDLING EXAMPLE (MULE FLOW)

```
<flow name="order-flow">
  <http:listener config-ref="HTTP_Listener" path="/order"/>
  <try>
    <!-- Flow processing -->
    <db:select config-ref="DB_Config" doc:name="Get Data"/>
  </try>
  <error-handler>
    <on-error-propagate type="DB:CONNECTIVITY">
      <set-payload value="Database connection error"/>
      <set-variable variableName="errorCode" value="DB_CONN_ERROR"/>
    </on-error-propagate>
  </error-handler>
</flow>
```

4. MUNIT: TEST ERROR SCENARIO

- Simulate error condition
- Execute flow
- Capture error
- Validate error type
- Validate error attributes
- Validate payload / response

Test Smart!

5. MOCK ERROR CONDITION EXAMPLE

```
<munit:behavior>
  <munit-tools:mock-when processor="db:select">
    <munit-tools:then-return>
      <munit-tools:throw-error
        type="DB:CONNECTIVITY"
        description="Database not available"/>
    </munit-tools:then-return>
  </munit-tools:mock-when>
</munit:behavior>
```

6. ASSERT ERROR TYPE

```
<munit:validation>
  <munit-tools:assert-error
    type="DB:CONNECTIVITY"
    description="Database not available"/>
</munit:validation>
```

7. ASSERT ERROR ATTRIBUTES

Attribute	Description	Example Assertion
type	Error type	type="DB:CONNECTIVITY"
description	Error description	description="Database not available"
errorCode	Custom error code	expression="#[error.errorCode]" is="#['DB_CONN_ERROR']"
httpStatus	HTTP status code	expression="#[attributes.statusCode]" is="#[500]"
payload	Error payload / message	expression="#[payload]" is="#['Database connection error']"

8. COMPLETE MUNIT TEST EXAMPLE

```
<munit:test name="orderFlow-DB-Error-Test">
  <munit:behavior>
    <munit-tools:mock-when processor="db:select">
      <munit-tools:then-return>
        <munit-tools:throw-error
          type="DB:CONNECTIVITY"
          description="Database not available"/>
      </munit-tools:then-return>
    </munit-tools:mock-when>
  </munit:behavior>
  <munit:execution>
    <flow-ref name="order-flow"/>
  </munit:execution>
  <munit:validation>
    <munit-tools:assert-error
      type="DB:CONNECTIVITY"
      description="Database not available"/>
    <munit-tools:assert-that
      expression="#[error.errorCode]"
      is="#['DB_CONN_ERROR']"/>
    <munit-tools:assert-that
      expression="#[payload]"
      is="#['Database connection error']"/>
  </munit:validation>
</munit:test>
```

9. TEST COMMON ERROR SCENARIOS

- Database connectivity error
- Timeout error
- Validation error
- Transformation error
- HTTP 4xx / 5xx errors
- File not found error
- Custom business errors
- External system down
- Authentication failure
- Authorization failure

10. ASSERT HTTP ERROR RESPONSE

```
<munit:validation>
  <munit-tools:assert-that
    expression="#[attributes.statusCode]"
    is="#[404]"/>
  <munit-tools:assert-that
    expression="#[payload.error.message]"
    is="#['Resource not found']"/>
</munit:validation>
```

HTTP Errors

11. TEST ON-ERROR-CONTINUE

```
<munit:test name="continue-error-test">
  <munit:behavior>
    <munit-tools:mock-when processor="http:request">
      <munit-tools:then-return>
        <munit-tools:throw-error type="HTTP:TIMEOUT"
          description="Request timeout"/>
      </munit-tools:then-return>
    </munit-tools:mock-when>
  </munit:behavior>
  <munit:execution>
    <flow-ref name="flow-with-onerror-continue"/>
  </munit:execution>
  <munit:validation>
    <munit-tools:assert-that
      expression="#[payload]"
      is="#['fallback response']"/>
  </munit:validation>
</munit:test>
```

Flow continues after error

12. VERIFY CUSTOM ERROR RESPONSE

```
<munit:validation>
  <munit-tools:assert-that
    expression="#[payload.errorCode]"
    is="#['ORDER_PROCESSING_ERROR']"/>
  <munit-tools:assert-that
    expression="#[payload.message]"
    is="#['Unable to process order']"/>
</munit:validation>
```

13. ENABLE ERROR TESTS IN SUITE

- Add all error test cases in a test suite.
- Run before and after critical changes.

```
<munit:test-suite name="error-tests">
  <munit:test name="db-error-test"/>
  <munit:test name="http-timeout-test"/>
  <munit:test name="validation-error-test"/>
</munit:test-suite>
```

14. BEST PRACTICES

- Test all error paths, not just success paths.
- Assert error type and attributes.
- Validate custom error codes.
- Use meaningful error messages.
- Keep tests independent and reusable.
- Maintain high error test coverage.

15. COMMON MISTAKES

- Not testing error scenarios.
- Asserting only error type, not attributes.
- Hardcoding error messages.
- Ignoring custom error codes.
- Not validating HTTP status codes.

16. INTERVIEW QUESTIONS

- Why is it important to test error handling?
- How do you mock an error in MUnit?
- What is assert-error in MUnit?
- How do you validate error attributes?
- How do you test on-error-continue?
- How do you test HTTP error responses?
- What are common error scenarios?

17. REAL-WORLD SCENARIO

An Order API flow calls a Database and an external payment service. If the DB is down or payment service times out, the flow returns a custom error response. MUnit tests simulate each failure and validates that the correct error type, code and message are returned to the client.

18. KEY TAKEAWAY

Testing error handling ensures your application fails gracefully, returns meaningful messages and maintains reliability even when things go wrong.

Well tested error handling = Reliable application + Happy users!

MODULE 13

13.9 INTEGRATION TESTING

MuleSoft from salesforce

Integration Testing in MUnit ensures that multiple flows, components, and systems work together as expected. It validates end-to-end interactions between APIs, databases, external services, and messaging systems.

1. WHAT IS INTEGRATION TESTING?

- Tests interactions between multiple components or systems.
- Validates data flow, transformations, and integrations.
- Ensures contracts between systems are honored.
- Detects issues that unit tests cannot catch.
- Improves confidence in real-world scenarios.

Integrated Confidence!

2. INTEGRATION TEST FLOW

Request Received → Flow A (Processing) → Flow B (Processing) → External System / DB → Response Returned

What we validate:

- ✓ End-to-end data flow
- ✓ Data integrity
- ✓ Correct system interactions
- ✓ Error propagation

3. EXAMPLE: INTEGRATION SCENARIO

Flow A calls Flow B and then saves data to a Database.

```
<flow name="processOrderFlow">
   <http:listener config-ref="HTTP_Listener" path="/orders"/>
    <flow-ref name="validateOrderFlow"/>
    <flow-ref name="saveOrderFlow"/>
    <set-payload value="#[payload]" />
    <http:response statusCode="200"/>
</flow>
```

We will test:

- HTTP Request → Flow A → Flow B → Database
- Response returned to the client

4. INTEGRATION TEST SETUP

- Use real or test instances of systems.
- Mock only external systems you want to isolate.
- Configure test data for all systems.
- Ensure clean state before and after tests.

5. SAMPLE INTEGRATION TEST (MUNIT)

```
<munit:test name="order-integration-test">
  <munit:behavior>
    <!-- Mock external payment service -->
    <munit-tools:mock-when processor="http:request" doc:name="Payment Service">
      <munit-tools:then-return>
        <munit-tools:payload value="{"status":"PAID"}" mediaType="application/json"/>
        <munit-tools:attributes value="#[{'statusCode':200}]" />
      </munit-tools:then-return>
    </munit-tools:mock-when>
  </munit:behavior>
  <munit:execution>
    <http:request config-ref="HTTP_Listener" path="/orders" method="POST">
      <http:body><![CDATA[{"orderId":"123","amount":100}]]></http:body>
    </http:request>
  </munit:execution>
  <munit:validation>
    <munit-tools:assert-that
      expression="#[attributes.statusCode]" is="#[MunitTools::equalTo(200)]" />
    <munit-tools:assert-that
      expression="#[payload.orderId]" is="#[MunitTools::equalTo('123')]" />
  </munit:validation>
</munit:test>
```

6. INTEGRATION TESTING APPROACHES

Approach	Description
Big Bang Integration	Integrate all components at once and test.
Top Down Integration	Start from top level module and integrate downwards.
Bottom Up Integration	Start from low level modules and integrate upwards.
Sandwich Integration	Combination of top down and bottom up.
Incremental Integration	Integrate modules one by one and test continuously.

7. INTEGRATION TEST VALIDATION POINTS

Validation Area	What to Validate	Example
Data Flow	Data is passed correctly between systems	Request → Flow A → Flow B → DB → Response
Data Transformation	Data is transformed as expected	Field mapping, formatting, enrichment
Business Logic	Business rules are applied correctly	Discount calculation, status update
Error Propagation	Errors are handled and propagated properly	If DB fails, proper error response returned
System Interaction	External systems are called with correct data	HTTP call with correct endpoint and payload
Response Validation	Final response to client is correct	Status code, headers, payload
Performance	Response time is within acceptable limits	Throughput, latency

8. MANAGING TEST DATA

- Use dedicated test data.
- Seed required data before tests.
- Clean up or rollback data after tests.
- Use unique data to avoid conflicts.
- Maintain data consistency.

9. TEST ENVIRONMENTS

Environment	Purpose
Local	Local development and initial testing
DEV	Integration with development systems
QA / Test	Full integration testing before release
Staging	Pre-production validation
Production	Live environment (monitor not test)

10. BEST PRACTICES

- Keep integration tests reliable and repeatable.
- Do not rely on manual setup.
- Use mocks for external dependencies.
- Validate happy path and error scenarios.
- Maintain tests along with changes.
- Keep tests independent.
- Use meaningful test names.

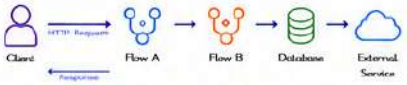
11. COMMON CHALLENGES & SOLUTIONS

Challenge	Solution
External system unavailable	Use mocks or stubs
Test data inconsistency	Use dedicated test data and cleanup
Long running tests	Optimize data and use parallel execution
Flaky tests	Avoid dependencies on external factors
Environment differences	Standardize configurations

12. INTERVIEW QUESTIONS

- What is Integration Testing in MUnit?
- Why is integration testing important?
- What are different integration testing approaches?
- How do you handle external system dependencies?
- How do you manage test data?
- How do you ensure test isolation?
- What are common challenges in integration testing?

13. SAMPLE INTEGRATION TEST SCENARIO



Client → HTTP Request → Flow A → Flow B → Database → External Service; Response back to Client

Test Steps:

1. Client sends request to Flow A.
2. Flow A validates and calls Flow B.
3. Flow B saves data to Database.
4. Flow B calls External Service.
5. Response returned to Client.

Validation Points:

- ✓ Correct data flow
- ✓ Data saved in Database
- ✓ External Service called successfully
- ✓ Correct response returned
- ✓ Error scenarios handled

14. CODE SNIPPET: VERIFY DB DATA

```
<munit:validation>
  <db:select config-ref="DB_Config" doc:name="Verify Order Saved">
    <db:sql>SELECT orderId, status FROM orders WHERE orderId = :orderId</db:sql>
    <db:input-parameters>
      <db:input-parameter key="orderId" value="#[vars.orderId]" />
    </db:input-parameters>
  </db:select>
  <munit-tools:assert-that
    expression="#[payload[0].status]"
    is="#[MunitTools::equalTo('PAID')]" />
</munit:validation>
```

15. KEY TAKEAWAY

Integration testing ensures all parts of your application work together seamlessly.

Good integration tests give confidence that your application behaves correctly in real-world scenarios.

Test the integrations, not just individual units!

Strong integrations build reliable applications. Test together. Deliver better!

MODULE 13

13.10 MUNIT BEST PRACTICES

MuleSoft from salesforce

Following MUnit best practices helps you build reliable, maintainable, and high-quality tests that give confidence in your integrations.

1. FOLLOW AAA PATTERN

Structure tests using Arrange, Act, Assert for clarity.

- **Arrange** — Set up test data and mocks.
- **Act** — Execute the flow.
- **Assert** — Verify the results.

Keeps tests simple, readable and maintainable.

2. TEST BEHAVIOR, NOT IMPLEMENTATION

Focus on what the flow should do, not how it is implemented.

Makes tests resilient to internal code changes.

3. KEEP TESTS SMALL AND FOCUSED

Test one behavior or scenario per test case.

Small tests are easier to write, debug and maintain.

4. USE MEANINGFUL TEST AND SUITE NAMES

Use clear, descriptive names for suites, tests and test cases.

Improves readability and helps everyone understand the intent.

5. MOCK EXTERNAL SYSTEMS

Always mock external systems (APIs, DB, MQ, etc.).

Tests become faster, reliable and not dependent on externals.

6. USE APPROPRIATE ASSERTIONS

- Use assert-that for single conditions.
- Use assert-equals for exact matches.
- Use assert-true / assert-false for boolean checks.
- Give custom failure messages for better debugging.

Clear assertions give clear failures.

7. VALIDATE BOTH POSITIVE AND NEGATIVE SCENARIOS

Positive Scenarios

- Happy path
- Valid data

Negative Scenarios

- Invalid data
- Error conditions
- Edge cases

Ensures your integration is robust and handles failures gracefully.

8. CREATE REUSABLE TEST DATA

- Store test data in DataWeave files or external files.
- Reuse data across tests.
- Keep data realistic and meaningful.

Reduces duplication and improves maintainability.

9. KEEP TESTS INDEPENDENT

- Tests should not depend on each other.
- Do not share state or data between tests.
- Each test should set up its own data and mocks.

Independent tests can run in any order and in parallel.

10. USE SETUP AND TEARDOWN EFFECTIVELY

- Use before-suite / before-test to set up common data or mocks.
- Use after-test / after-suite to clean up if required.

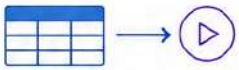
Keeps tests clean and reduces code duplication.

11. AVOID OVER-ASSERTING

- Do not assert every payload field unless necessary.
- Assert only what is important for the behavior.
- Over-asserting makes tests brittle.

Tests remain stable even if non-critical parts of payload change.

12. USE DATA-DRIVEN TESTING WHERE IT MAKES SENSE



- Run the same test with multiple input datasets.
- Use parameterized tests.

Improves coverage with minimal additional effort.

13. HANDLE ERRORS AND EXCEPTIONS

- Test error flows and exceptions.
- Assert correct error type, message and status code.
- Verify error handling logic.

Ensures your flow fails in a controlled and expected way.

14. USE MUNIT TOOLS WISELY

- Use mock-when / then-return correctly.
- Use spy to verify processor execution.
- Use verify-call to check interactions.
- Avoid unnecessary spies and verifications.

Use the right tool for the right purpose.

15. MAINTAIN GOOD COVERAGE WITH QUALITY

- Lines
- Branches
- DataWeave
- Flows
- Errors

Aim for high coverage, but never compromise on test quality.

16. KEEP TESTS FAST

- Avoid complex logic in tests.
- Mock slow systems.
- Keep payloads small.

Fast tests encourage developers to run them frequently.

17. ORGANIZE TESTS WELL

Logical structure makes tests easy to find and maintain.

18. FOLLOW NAMING CONVENTIONS

Element	Naming Convention
Test Suite	<feature>-suite
Test Case	<scenario>-test
Test Data File	<scenario>-data.json
Mock File	<system>-mock.xml

Consistent naming improves readability and team collaboration.

19. REVIEW AND REFINE REGULARLY

- Review tests as flows evolve.
- Remove outdated or redundant tests.
- Refactor tests for better readability and performance.

Keep tests up to date with the application.

20. DOCUMENT TEST INTENT

- Add clear descriptions for suites and tests.
- Explain what is being tested and why.

Helps future developers understand the purpose of tests.

KEY TAKEAWAY

Good MUnit tests are: Reliable, Independent, Readable, Maintainable and Fast.
Follow these best practices to build confidence in your integrations and deliver quality faster!

MODULE 14

14.1 LOGGER

MuleSoft from salesforce

Logger is a core component in MuleSoft used to log messages, variables, payloads, and errors. It helps in monitoring, debugging, and tracking the flow execution.

1. WHAT IS LOGGER?

Logger writes messages to the configured logging system (console, file, external systems like Splunk, etc.).

It provides visibility into the flow execution and helps in troubleshooting.

2. LOGGER COMPONENT

Display Name	Name to identify the logger component.
Message	The log message or expression to be logged.
Level	The severity level of the log.
Category	The log category (optional).
Log Object	Object to log (payload, attributes, variables, etc.).

Logger

3. LOGGER CONFIGURATION

- Logger uses Log4j2 under the hood.
- You can configure logging in `src/main/resources/log4j2.xml`
- You can define appenders (Console, File, Rolling File, Splunk, etc.) and log levels.

Example: Console Logger (log4j2.xml)

```
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%level]
                    %logger - %msg%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="INFO">
      <AppenderRef ref="Console"/>
    </Root>
  </Loggers>
</Configuration>
```

4. LOGGER MESSAGE EXPRESSIONS

You can use DataWeave expressions to log dynamic data.

Examples

- Log a simple message
  Message: "Flow started successfully"
- Log payload
  Message: `#[payload]`
- Log specific field from payload
  Message: `#[payload.name]`
- Log attributes
  Message: `#[attributes]`
- Log variable
  Message: `#[vars.customerId]`
- Log multiple values
  Message: "Customer ID: `#[vars.customerId]`, Status: `#[attributes.statusCode]`"

Note: Always use expressions (`#[]`) to log dynamic values.

5. LOGGING PAYLOAD, ATTRIBUTES & VARIABLES

Log Payload — Message: `#[payload]`

</> Log Attributes — Message: `#[attributes]`

{x} Log Variable — Message: `#[vars.customerId]`

Tip: Be careful when logging large payloads in production. It may impact performance and log storage.

6. LOGGER LEVELS

Logger supports different log levels to control the importance of messages.

Level	Description	Use Case
TRACE	Detailed trace information	Very detailed debugging
DEBUG	Detailed debug information	Debugging during development
INFO	General operational messages	Normal flow execution info
WARN	Warning messages	Potential issues
ERROR	Error messages	Errors that should be investigated
FATAL	Severe errors causing failure	Critical failures

7. BEST PRACTICES

Use Appropriate Log Levels
Use INFO for normal logs and ERROR for exceptions.

Do Not Log Sensitive Data
Avoid logging passwords, tokens, credit card details, or PII.

Avoid Logging Large Payloads
Log only required fields to avoid performance degradation.

Use Structured Logging
Use JSON or key-value format for better searchability.

Integrate with Monitoring Tools
Send logs to Splunk, ELK, or cloud monitoring tools for analysis.

KEY TAKEAWAY: Logger is essential for observability, debugging, and monitoring in Mule applications. Use it wisely with proper log levels and without exposing sensitive data.

MODULE 14

14.2 CORRELATION ID

MuleSoft from salesforce

Correlation ID is a unique ID that helps to identify and track a request as it flows across multiple systems and APIs. It is essential for logging, monitoring, debugging and end-to-end tracing.

1. WHAT IS CORRELATION ID?

- Correlation ID is a unique identifier associated with a request.
- It helps to trace the same request across multiple systems, APIs, microservices and logs.
- It improves troubleshooting and monitoring capabilities.

Correlation ID

550e8400-e29b-41d4-a716-446655440000

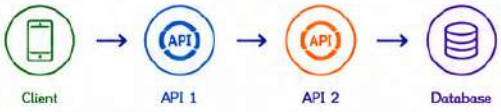
2. AUTO-GENERATED VS CUSTOM CORRELATION ID

Type	Description	Example	Use Case
Auto-generated	System generates a unique ID if not provided in the incoming request.	550e8400-e29b-41d4-a716-446655440000	Default behavior when client does not send any ID.
Custom	Client or external system provides the ID in the request.	ABC12345-PAYMENT-2024-05-30-001	When client wants to track the request using its own ID.

Best Practice: Always accept incoming Correlation ID if provided, otherwise generate a new one.

3. PASSING CORRELATION ID ACROSS APIS

The Correlation ID must be passed from one system/API to another in every request.



Same Correlation ID is passed in every request/response to maintain traceability.

4. HTTP HEADERS

Correlation ID is usually passed in HTTP headers.

Header Name	Description
X-Correlation-ID	Custom header used to carry the correlation ID. (Recommended)
X-Request-ID	Another commonly used header name.

Example:

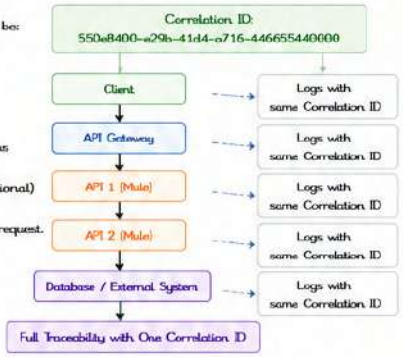
```
GET /api/orders HTTP/1.1
Host: api.mycompany.com
X-Correlation-ID: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json
```

5. END-TO-END REQUEST TRACKING

The same Correlation ID should be:

- Received from the client
- Logged in every system
- Passed to downstream systems
- Returned in the response (optional)

This enables full traceability of a request.



6. BEST PRACTICES

- **Always Accept if Present**
  If a Correlation ID is present in the request, accept and use it.
- **Generate if Missing**
  If not present, generate a new unique Correlation ID.
- **Pass in Every Request**
  Always pass the Correlation ID in headers to downstream services.
- **Log Everywhere**
  Log the Correlation ID in every log entry for easy tracking.
- **Return in Response**
  Return the Correlation ID in response headers for client reference.
- **Use Standard Header**
  Use a standard header name like X-Correlation-ID.
- **Do Not Change**
  Do not modify the Correlation ID during the flow.

IMPLEMENTATION EXAMPLE IN MULE (FLOW LOGIC)

1. Check if Correlation ID exists in request header
2. If not, generate a new UUID
3. Set the Correlation ID in a variable
4. Add it to the outbound request headers
5. Log it in every step

```
<set-variable variableName="correlationId" value="#[attributes.headers['X-Correlation-ID'] default uuid()]" />

<logger level="INFO" message="Received request - Correlation ID: #[vars.correlationId]" />

<http:request config-ref="HTTP_Request_Configuration" path="/orders" method="POST">
   <http:headers>
     <http:header headerName="X-Correlation-ID" value="#[vars.correlationId]" />
   </http:headers>
</http:request>
```

KEY TAKEAWAY: Correlation ID is the key to end-to-end visibility. Use it consistently across all systems, log it everywhere, and follow best practices to make debugging and monitoring seamless.

MODULE 14 | 14.3

SPLUNK INTEGRATION

MuleSoft from salesforce

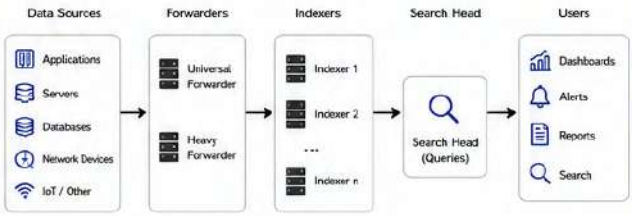
Splunk is a powerful platform for collecting, indexing, searching and visualizing machine data in real time.
Integrating Mule applications with Splunk helps in centralized logging, monitoring, alerting and operational visibility.

1. WHY SPLUNK?

- Centralized log collection from multiple systems and environments.
- Real-time search, monitoring and alerting.
- Powerful dashboards and visualizations.
- Scalability to handle large volumes of data.
- Helps in troubleshooting and compliance audits.

splunk>

2. SPLUNK ARCHITECTURE



Data is collected from various sources, forwarded to indexers for storage and indexing.
Search Head is used for querying and visualization by users.

3. HTTP EVENT COLLECTOR (HEC)

HTTP Event Collector (HEC) is a simple and reliable way to send data to Splunk using HTTPS.

HEC Key Features

- Uses REST API over HTTPS
- Accepts JSON formatted events.
- Supports acknowledgment for reliable delivery.
- Built-in token based authentication.

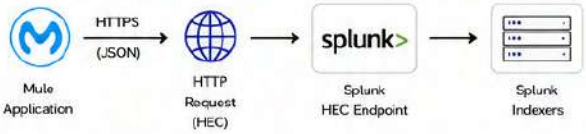
HEC Endpoint Format

```
POST https://<splunk-host>:8088/services/collector/event
Authorization: Splunk <HEC_TOKEN>
Content-Type: application/json
```

Generate HEC Token in Splunk:
Settings > Data Inputs > HTTP Event Collector > New Token

4. LOG FORWARDING

Mule application sends logs to Splunk using HEC. This can be implemented using HTTP Request connector or custom logger.



Implementation Steps

1. Create HEC token in Splunk.
2. Use HTTP Request connector to POST logs in JSON format to HEC endpoint.
3. Include Authorization header with token.
4. Splunk indexers receive the data and make it searchable.

5. JSON LOG FORMAT

Splunk works best with JSON formatted logs.

Example JSON Log

```
{
  "time": "2024-05-30T10:15:30.123Z",
  "level": "INFO",
  "message": "Order created successfully",
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "application": "customer-api",
  "environment": "DEV",
  "host": "mule-worker-1",
  "payload": {
    "orderId": "ORD12345",
    "customerId": "CUST1001",
    "amount": 250.75
  },
  "statusCode": 201,
  "responseTimeMs": 145
}
```

Recommended Fields

- time : Event timestamp (ISO 8601)
- level : Log level (INFO, ERROR, etc.)
- message : Log message
- correlationId : Unique ID for tracing
- application : Application name
- environment : DEV / QA / PROD
- host : Hostname / Instance
- statusCode : HTTP status code
- responseTimeMs: Response time in ms

6. DASHBOARD INTEGRATION

Once logs are indexed, create dashboards and alerts in Splunk.

Common Visualization Examples

- Request Count Over Time
- Error Rate / Status Code Distribution
- Response Time (95th percentile)
- Top APIs by Volume
- Latency by Environment
- Failed Transactions

Sample Search Queries

- All logs
  index=main sourcetype=mule:logs
- Error logs
  index=main level=ERROR
- API response time
  index=main | stats avg(responseTimeMs) by apiName
- Top failed APIs
  index=main statusCode>=400 | stats count by apiName

Alert Example

Trigger: count where ERROR count > 100 in 5 minutes.
Action: Send email / Slack notification.

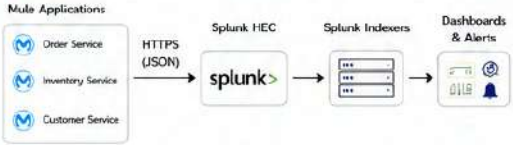
7. ENTERPRISE EXAMPLE

A retail company uses Mule applications for order processing, inventory and customer services. All logs are sent to Splunk for monitoring.

Solution Overview

1. Mule apps generate structured JSON logs.
2. Logs are sent to Splunk using HEC.
3. Splunk indexes logs and provides real-time dashboards.
4. Alerts are configured for critical errors.
5. Operations and DevOps teams monitor health and performance.

Architecture



Business Benefits

- Faster issue detection and resolution
- End-to-end visibility of transactions
- Better operational efficiency
- Audit and compliance support
- Data-driven insights for optimization

BEST PRACTICES

- Use structured JSON logs for better searchability.
- Always include correlationId for tracing.
- Secure HEC token and use HTTPS.
- Do not log sensitive data (passwords, tokens).
- Use appropriate log levels.
- Create meaningful dashboards and alerts.

MODULE 14 **14.4** # MONITORING DASHBOARD

MuleSoft from salesforce

Monitoring helps you track the health, performance and availability of your Mule applications and APIs in real time.
MuleSoft provides built-in monitoring through Anypoint Monitoring, Runtime Manager and API Manager.

1. ANYPOINT MONITORING

Anypoint Monitoring provides end-to-end visibility of your integrations, APIs and applications across environments.

Anypoint Monitoring

- Real-time operational visibility
- Performance metrics and trends
- Error tracking and alerts
- Custom dashboards and reports
- Drill-down to transaction level
- Historical data and analytics

2. RUNTIME MANAGER METRICS

Runtime Manager monitors the health and performance of Mule applications deployed in CloudHub or on-premises runtimes.

CloudHub
On-Premises
Workers
Alerts

Category	Description
Applications	Status, uptime, deployments, restarts
Workers	Worker count, status, CPU, memory
Servers	Server health and availability
Logs	View and search application logs
Alerts	Create and manage alerts
Insights	Automatic anomaly detection and insights

3. API MANAGER MONITORING

API Manager provides analytics and monitoring for your APIs in real time.

Analytics
Security
SLAs
Consumers

Metric	Description
Traffic	Requests, bandwidth, top APIs
Performance	Response time, latency, cached responses
Errors	Error rate, fault details, top error responses
Clients	Top consuming applications / clients
SLA	SLA compliance and violations
Security	Policy violations and threat protection

4. CPU / MEMORY / THREADS

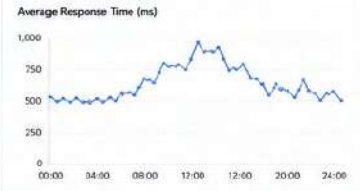
Track infrastructure and runtime resource usage.

CPU Usage	Memory Usage	Active Threads
58%	62%	128
Average CPU	Heap Memory	Live Threads

High CPU or Memory usage can impact performance.
Monitor thread count to detect potential bottlenecks.

5. RESPONSE TIME

Monitor the response time of your applications and APIs.



Avg Response Time: 420 ms
95th Percentile: 980 ms
99th Percentile: 1,320 ms

Track average and percentile response times to ensure consistent performance and meet SLA targets.

6. ERROR MONITORING

Identify errors quickly and take action.

Error Rate: 2.35% (Last 24h)
Total Errors: 1,248

- Monitor error rate trends and error types.
- Set alerts for error rate thresholds.

7. DASHBOARD WALKTHROUGH

Step-by-step guide to use Monitoring Dashboard in Anypoint Platform.

1. **Login** — Log in to Anypoint Platform.
2. **Select Application / API** — Choose the application or API you want to monitor.
3. **Open Monitoring** — Go to Monitoring > Overview Dashboard.
4. **View Overview** — See key metrics like traffic, response time, errors, and resources.
5. **Drill Down** — Click on any metric to drill down to details and transactions.
6. **Create Alerts** — Set alerts for critical metrics and receive notifications.
7. **Custom Dashboards** — Create and save custom dashboards for your team.

Typical Monitoring Dashboard View

BEST PRACTICES

- Monitor key metrics continuously.
- Set appropriate alert thresholds.
- Use percentile response time for analysis.
- Regularly review logs and error trends.
- Create role-based dashboards for teams.
- Act on alerts and optimize proactively.

MODULE 14 **14.5** # ALERTS

MuleSoft from salesforce

Alerts help you proactively identify and respond to issues in your Mule applications and APIs. They notify you when certain conditions or thresholds are met so you can take action before users are impacted.

1. WHAT ARE ALERTS?

Alerts are notifications triggered when a specific condition is met in your application, API or infrastructure.

- Detect issues in real time
- Reduce MTTR (Mean Time To Resolve)
- Improve system reliability
- Proactive monitoring and response
- Can be sent via Email, Slack, Webhook etc.

2. RUNTIME ALERTS

Runtime Alerts monitor the health and performance of your Mule application runtime (CloudHub or on-premises).

Alert Type	Description	Common Use Cases
Application Down	Application is not running or unavailable	Notify when app is stopped unexpectedly
High CPU Usage	CPU usage exceeds threshold	Detect high resource utilization
High Memory Usage	Memory usage exceeds threshold	Prevent OutOfMemory issues
Worker Not Responding	Worker is unresponsive or restarted	Detect worker crashes or hangs
Deployment Failed	Application deployment failed	Notify DevOps/Support immediately

3. API ALERTS

API Alerts monitor API performance, availability and usage in API Manager.

Alert Type	Description
High Error Rate	Error rate exceeds configured threshold (e.g., > 5%)
High Response Time	Average response time exceeds configured threshold
API Down	API is not reachable or returning errors
SLA Breach	API SLA (availability, latency) is violated
Spike in Traffic	Traffic exceeds expected threshold
Quota Limit Reached	API usage quota limit is reached by a consumer

4. EMAIL NOTIFICATIONS

Send alert notifications via Email to individuals or distribution lists.

How to Configure

1. Create an alert in Anypoint Platform.
2. Select "Email Notification" as the notification channel.
3. Add recipient email addresses.
4. Customize subject and message.
5. Save and activate the alert.

5. SLACK / WEBHOOK INTEGRATION

Integrate alerts with Slack or any external system using Webhooks.

How to Configure

1. Create an alert in Anypoint Platform.
2. Select "Webhook" as the notification channel.
3. Provide the Slack Incoming Webhook URL.
4. Customize the JSON payload (message format)
5. Save and activate the alert.

Example Slack Message

```
{
  "text": "🚨 Alert: High CPU usage detected in Order-API (CPU > 80%)",
  "attachments": [
    {
      "color": "danger",
      "fields": [
        {"title": "Application", "value": "Order-API", "short": true},
        {"title": "Metric", "value": "CPU Usage", "short": true},
        {"title": "Value", "value": "85%", "short": true}
      ]
    }
  ]
}
```

6. AUTO SCALING ALERTS

Auto Scaling Alerts notify you when scaling actions occur or are required.

Alert Type	Description
Scale Up Triggered	Application scaled up due to high load
Scale Down Triggered	Application scaled down due to low load
Scale Action Failed	Auto scaling action failed
High Concurrent Requests	Concurrent requests exceed threshold
System Resource Threshold	CPU/Memory thresholds reached that may trigger scaling

Note

Auto Scaling Alerts are available for CloudHub 2.0 applications. Configure thresholds based on your application behavior.

7. BEST PRACTICES

Define Meaningful Thresholds

- Set realistic and achievable thresholds.
- Avoid too many alerts.

Reduce Noise

- Alert only on critical conditions.
- Use warning vs critical levels.

Notify the Right People

- Send alerts to the right teams or on-call engineers.

Act Quickly and Review

- Respond to alerts promptly.
- Regularly review and tune alerts.

Document Alert Policies

- Maintain documentation of alert rules and escalation process.

Monitor the Monitors

- Ensure alerting system itself is healthy and operational.

KEY TAKEAWAY Alerts enable proactive issue detection and faster resolution. Configure the right alerts, use effective notification channels and follow best practices to keep your systems healthy.

MODULE 14 **14.6** # LOG LEVELS

MuleSoft from salesforce

Log levels define the severity or importance of a log message. They help control the amount of logging information and focus on what is most relevant in each environment.

1. LOG LEVELS OVERVIEW

Level	Priority	Description	Typical Use
TRACE	1	Finest level of logging. Very detailed information, usually for debugging code.	Deep diagnostics. Used only during development.
DEBUG	2	Detailed information for debugging and troubleshooting.	Development and testing.
INFO	3	General operational messages about the normal execution of the application.	Production and business as usual operations.
WARN	4	Indicates something unexpected or unusual, but the application can continue.	Production – needs attention but not critical.
ERROR	5	Indicates a serious problem that prevents a specific operation from completing.	Production – action required.
FATAL	6	Very severe error that causes application or system failure.	Critical failure – immediate action required.

2. WHEN TO USE EACH LOG LEVEL

TRACE
- Step-by-step trace of program flow.
- Variable values, method entry/exit.
- Very high volume, enable only when needed.

DEBUG
- Detailed debugging information.
- SQL queries, request/response payloads (sanitized).
- Useful for developers and testers.

INFO
- Successful operations and key events.
- Startup/shutdown messages.
- Business transaction summaries.

WARN
- Recoverable issues.
- Configuration issues.
- External service slow responses.

ERROR
- Operation failed due to an error.
- Exception stack traces.
- Upstream/Downstream service failures.

FATAL
- Unrecoverable errors.
- System is going to shutdown.
- Data corruption, out of memory, etc.

3. COMPARISON TABLE

Level	Priority (Low → High)	Purpose	Example Log Message	Enabled In
TRACE	1	Deep diagnostics and tracing	TRACE [OrderService] Entered getOrderDetails() with orderId=12345	Dev (Temporary)
DEBUG	2	Debugging and troubleshooting	DEBUG [OrderService] Request payload: {"orderId":"12345"}	Dev / Test
INFO	3	General operational information	INFO [OrderService] Order 12345 processed successfully	All Environments
WARN	4	Potential issues	WARN [InventoryService] Low stock for productId=987	Test / Prod
ERROR	5	Operation failures	ERROR [PaymentService] Failed to process payment. Reason: Timeout	All Environments
FATAL	6	Critical failures	FATAL [OrderService] Application is shutting down due to OutOfMemoryError	All Environments

4. EXAMPLE IN MULE LOGGER

```
<logger level="TRACE"  message="Trace message: #[vars.data]" />
<logger level="DEBUG"  message="Debug message: #[payload]" />
<logger level="INFO"   message="Order #[vars.orderId] processed successfully" />
<logger level="WARN"   message="Low stock for product #[vars.productId]" />
<logger level="ERROR"  message="Failed to call payment API" />
<logger level="FATAL"  message="Application shutting down due to critical error" />
```

Note: Set the appropriate level in log4j2.xml to control what gets logged.

5. PRODUCTION LOGGING GUIDELINES

- Use INFO for normal operational logs.
- Use WARN for situations that need attention but do not stop the flow.
- Use ERROR for failures that require investigation.
- Use FATAL only for unrecoverable issues.
- Avoid using DEBUG and TRACE in production (high volume and sensitive data risk).
- Do not log sensitive information (passwords, tokens, card details, PII).
- Log correlationId with every request for end-to-end tracing.
- Use structured (JSON) logs for better search and analytics in Splunk.
- Keep log messages clear, consistent and meaningful.
- Configure log retention and log rotation to manage disk usage.

6. RECOMMENDED LOG LEVEL BY ENVIRONMENT

Environment	Recommended Level	Description
Development	DEBUG / TRACE	Detailed logs for debugging and development.
Testing / QA	INFO / DEBUG	Useful for testing and issue identification.
Staging / UAT	INFO	Verify real operational behavior with moderate logs.
Production	INFO / WARN / ERROR	Log important events and issues only.

7. LOG LEVEL HIERARCHY

TRACE (1)
↓
DEBUG (2)
↓
INFO (3)
↓
WARN (4)
↓
ERROR (5)
↓
FATAL (6)

- Lower the level number → More logs
- Higher the level number → Fewer logs (only important)

Example:
If level is set to INFO, only INFO, WARN, ERROR and FATAL logs will be recorded.

KEY TAKEAWAY Choose the right log level to capture the right amount of information. Good logging improves troubleshooting, monitoring and operational efficiency without impacting performance.

MODULE 14 14.7

API ANALYTICS

MuleSoft from salesforce

API Analytics in MuleSoft helps monitor API usage, performance, consumers, and business metrics. It enables organizations to understand API health, optimize performance, and make data-driven decisions.

1. WHAT IS API ANALYTICS?

- Collects API usage metrics.
- Tracks API performance.
- Monitors consumer behavior.
- Measures API health.
- Supports business insights.

Data-driven API Management

2. API ANALYTICS DASHBOARD

Requests	Response Time	Errors	Consumers	Policies	Geography
1.25M Total	215 ms Avg	2.35% Error %	842 Active	56 Executed	32 Countries

3. REQUEST COUNT

- Total API requests
- Requests by hour/day/month
- Peak traffic analysis
- Top APIs by usage
- Trending requests

4. RESPONSE TIME

- Average Response Time
- Minimum Response Time
- Maximum Response Time
- P95 Latency
- P99 Latency

Avg Response Time: 215 ms

5. ERROR PERCENTAGE

$$\text{Error \%} = \left(\frac{\text{Failed Requests}}{\text{Total Requests}}\right) \times 100$$

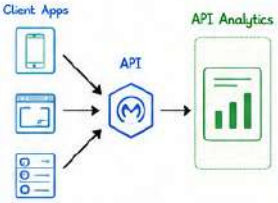
Track:

- 4xx Errors
- 5xx Errors
- Timeout Errors
- Policy Violations

Error % (Last 24h): 2.35%

6. CONSUMER ANALYTICS

- Client Applications
- Consumer IDs
- API Keys
- OAuth Clients
- Top Consumers
- Usage by Application



7. SLA MONITORING

- SLA Tiers
- Request Limits
- Quota Usage
- Rate Limits
- Throttling

SLA Tier	Bronze	Silver	Gold
Request Limit (per day)	10,000	50,000	200,000
Rate Limit (per minute)	100	500	2,000
Quota Usage	65%	40%	25%
Throttling	Enabled	Enabled	Enabled
Support	Standard	Priority	Premium
Availability Target	99.0%	99.5%	99.9%

8. CUSTOM REPORTS

- Daily Reports
- Weekly Reports
- Monthly Reports
- Export CSV
- Export PDF
- Scheduled Reports
- Executive Dashboard

Build custom reports with filters, time range, API, environment, consumer and more.

9. COMMON METRICS

Metric	Description
Request Count	Total API Calls
Response Time	API Latency
Availability	API Uptime
Error Rate	Failed Requests
Consumer Count	Active Clients
Traffic Trend	Usage Growth
Policy Hits	Policy Executions

10. ENTERPRISE EXAMPLE

Analytics captures:

- Total Requests
- Average Response Time
- Consumer Usage
- Error Rate
- SLA Compliance

BEST PRACTICES

- Monitor APIs continuously.
- Review analytics regularly.
- Configure SLA alerts.
- Track response time trends.
- Analyze consumer behavior.
- Export reports for audits.
- Optimize high-traffic APIs.
- Monitor policy violations.

INTERVIEW QUESTIONS

- What is API Analytics?
- Difference between Monitoring and Analytics?
- Which metrics are most important?
- How do you monitor API performance?
- What is SLA Monitoring?
- How do you identify slow APIs?

KEY TAKEAWAYS

- API Analytics improves API visibility.
- Helps optimize performance.
- Tracks consumers and SLA usage.
- Supports proactive monitoring.
- Enables data-driven API management.

You can't improve what you don't measure. API Analytics provides the insights needed to build reliable, scalable, and high-performing APIs.

MODULE 14 14.8

LOG4J2 CONFIGURATION (RECOMMENDED)

MuleSoft from salesforce

Log4j2 is the recommended logging framework for Mule applications. It is fast, flexible and supports asynchronous logging, rolling files, multiple appenders and environment-specific configurations.

1. log4j2.xml

The main configuration file for Log4j2. Place it in:

`src/main/resources/log4j2.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" monitorInterval="60">
  <Properties>
    <Property name="LOG_PATH">${sys:LOG_PATH:-logs}</Property>
    <Property name="APP_NAME">mule-application</Property>
    <Property name="PATTERN">%d{yyyy-MM-dd HH:mm:ss.SSS}
        %-5level [%t] %c{1.} - %msg%n</Property>
  </Properties>
  <Appenders>
    <!-- Appenders defined here -->
  </Appenders>
  <Loggers>
    <!-- Loggers defined here -->
  </Loggers>
</Configuration>
```

- status="WARN" : Internal Log4j2 status logging level
- monitorInterval : Reloads configuration every 60 seconds (in seconds)

2. Rolling File Appender

Writes logs to files and rolls them based on size and date.

```
<RollingFile name="RollingFile" fileName="${LOG_PATH}/${APP_NAME}.log"
             filePattern="${LOG_PATH}/archive/${APP_NAME}-%d{yyyy-MM-dd}-%i.log.gz">
  <PatternLayout pattern="${PATTERN}" />
  <Policies>
    <TimeBasedTriggeringPolicy interval="1" modulate="true" />  <!-- daily roll -->
    <SizeBasedTriggeringPolicy size="100 MB" />                 <!-- size roll -->
  </Policies>
  <DefaultRolloverStrategy max="30" fileIndex="min" />          <!-- keep 30 files -->
</RollingFile>
```

- Rolls every day or when file size exceeds 100 MB.
- Old log files are compressed (.gz) and limited to 30 archives.
- Prevents disk space from consuming unlimited storage.

3. Console Appender

Prints logs to the console (STDOUT).

```
<Console name="Console" target="SYSTEM_OUT">
  <PatternLayout pattern="${PATTERN}" />
</Console>
```

- Useful in development and containerized environments.
- Ensures logs are visible in runtime console (CloudHub, Docker, etc.)

4. Async Logging

Improves performance by logging events asynchronously.

```
<Async name="AsyncLogger" includeLocation="false" bufferSize="262144">
  <AppenderRef ref="RollingFile" />
  <AppenderRef ref="Console" />
</Async>
```

- Events are processed in the background using a disruptor queue.
- Reduces IO latency and improves application throughput.
- **bufferSize** should be a power of 2 (e.g., 262144).

5. Log Patterns

Defines the format of each log line.

Pattern	Description
%d{yyyy-MM-dd HH:mm:ss.SSS}	Date and time
%-5level	Log level (left padded to 5 chars)
[%t]	Thread name
%c{1.}	Logger name (short class name)
%msg	Log message
%n	New line
%X{key}	MDC/Context data (e.g. correlationId)

Example Pattern

```
%d{yyyy-MM-dd HH:mm:ss.SSS}
%-5level [%t] %c{1.} - %msg
%X{correlationId}%n
```

Sample Log Output

```
2024-05-21 10:15:30.123 INFO  [http-nio-8081-exec-3]
c.m.p.api.OrderAPI - Order created successfully
corrId=12345-abcde
```

6. Environment-wise Configuration

Use system properties or variables to control log level and paths.

Environment	Log Level	LOG_PATH	Description
Development	DEBUG	logs/dev	Detailed logs for debugging
Test / QA	INFO	logs/test	Info level for testing
Staging / UAT	INFO	logs/stage	Monitor with controlled logs
Production	WARN	logs/prod	Minimal logs, warnings & errors
Disaster Recovery	ERROR	logs/dr	Only errors for critical issues

Pass LOG_PATH and log level using JVM system properties or environment variables in each environment.
Example: -DLOG_PATH=/opt/mule/logs/prod -Dlog4j2.level=WARN

7. Recommended log4j2.xml (Complete Example)

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" monitorInterval="60">
    <Properties>
        <Property name="LOG_PATH">${sys:LOG_PATH:-logs}</Property>
        <Property name="APP_NAME">mule-application</Property>
        <Property name="PATTERN">%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level [%t] %c{1.} - %msg %X{correlationId}%n</Property>
    </Properties>
    <Appenders>
        <RollingFile name="RollingFile" fileName="${LOG_PATH}/${APP_NAME}.log"
                     filePattern="${LOG_PATH}/archive/${APP_NAME}-%d{yyyy-MM-dd}-%i.log.gz">
            <PatternLayout pattern="${PATTERN}" />
            <Policies>
              <TimeBasedTriggeringPolicy interval="1" modulate="true" />
              <SizeBasedTriggeringPolicy size="100 MB" />
            </Policies>
            <DefaultRolloverStrategy max="30" fileIndex="min" />
        </RollingFile>
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout pattern="${PATTERN}" />
        </Console>
        <Async name="AsyncLogger" includeLocation="false" bufferSize="262144">
            <AppenderRef ref="RollingFile" />
            <AppenderRef ref="Console" />
        </Async>
    </Appenders>
    <Loggers>
        <Root level="INFO">
            <AppenderRef ref="AsyncLogger" />
        </Root>
        <!-- Example: set package-specific level -->
        <Logger name="com.mulesoft" level="DEBUG" additivity="false">
            <AppenderRef ref="AsyncLogger" />
        </Logger>
    </Loggers>
</Configuration>
```

Key Points

- Async logger improves performance.
- RollingFile handles log rotation.
- Console appender shows logs in runtime.
- Change log level per environment.
- MonitorInterval enables live reload.

Log Level Reference

Level	Purpose
TRACE	Very detailed tracing
DEBUG	Detailed debugging information
INFO	General operational information
WARN	Something unexpected happened
ERROR	A serious problem occurred
FATAL	Very severe error, app may crash

BEST PRACTICES

- Use Async logging in production.
- Use RollingFile to manage disk space.
- Avoid DEBUG/TRACE in production.
- Include correlationId in logs.
- Use meaningful log messages.
- Keep log patterns consistent.
- Monitor log file sizes and retention.
- Use centralized log management (e.g., Splunk, ELK).
- Review and rotate logs regularly.

TIP

Good logging is the key to fast issue resolution and system reliability.

MODULE 14 | 14.9

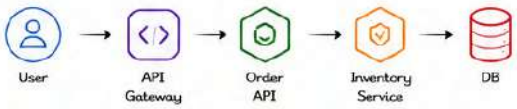
DISTRIBUTED TRACING (RECOMMENDED)

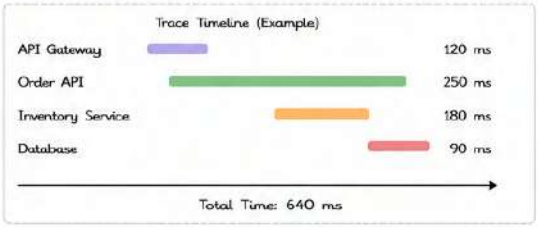
MuleSoft from salesforce

Distributed tracing helps track a request as it flows through multiple systems and services.
It provides end-to-end visibility, helps in faster troubleshooting and performance optimization.

1. END-TO-END REQUEST TRACKING

Trace a single request across multiple components and visualize the complete journey with timings.





2. CORRELATION ID vs TRACE ID

Understand the difference and when to use each.

Feature	Correlation ID	Trace ID
Purpose	Track a request across logs and systems	Track a request across distributed services
Scope	Application / System level	Distributed system (End-to-End)
Generated By	Application / Middleware	Tracing System (e.g., OpenTelemetry)
Format	Simple ID (UUID)	Trace ID (16/32 bytes hex string)
Propagated In	Headers / Logs (e.g., X-Correlation-ID)	Headers (traceparent as per W3C spec)
Use Case	Log correlation, simple request tracking	Performance monitoring, bottleneck analysis
Example	X-Correlation-ID: 9f7b2c...	traceparent: 00-4bf92f... -00f067aa0ba902b7-00

Use Correlation ID for log correlation and Trace ID for end-to-end distributed tracing.

3. MULTIPLE API FLOW TRACKING

Track a request flowing through multiple APIs and downstream systems.

Step	Service / API	Operation	Start Time	Duration	Status
1	Experience API	POST /orders	10:15:10.100	120 ms	OK
2	Process API	POST /orders/validate	10:15:10.220	200 ms	OK
3	System API	GET /inventory	10:15:10.420	150 ms	OK
4	System API	POST /payment	10:15:10.570	230 ms	OK
5	External Service	POST /notification	10:15:10.800	90 ms	OK

Full visibility of each hop with timing and status.

4. MICROSERVICES MONITORING

Monitor health, latency, errors and throughput of microservices.

Monitor Key Metrics
- Request Rate
- Response Time (P50, P95, P99)
- Error Rate
- Throughput
- Latency by Dependency
- Service Health
- SLA Compliance

Helps identify slow services, failures and impacted dependencies quickly.

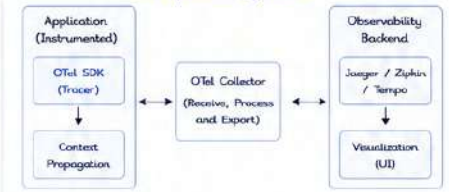
5. OPENTELEMETRY CONCEPTS

OpenTelemetry is an open standard for collecting telemetry data (traces, metrics, logs) from applications.

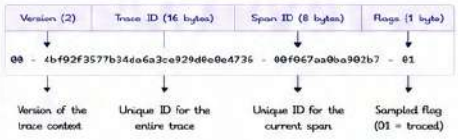
Core Building Blocks

- **Tracer** – Creates and manages spans and traces
- **Span** – Represents a single unit of work
- **Trace** – Collection of spans forming a request journey
- **Context Propagation** – Carries Trace ID across service boundaries
- **Exporters** – Send telemetry data to backends (Jaeger, Zipkin, OTel Collector etc.)

How OpenTelemetry Works



W3C Trace Context (traceparent header)



Example: traceparent Header

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-
             00f067aa0ba902b7-01

# 00   -> version
# 4bf92f3577b34da6a3ce929d0e0e4736 -> Trace ID
# 00f067aa0ba902b7 -> Span ID
# 01   -> Flags (sampled)
```

Benefits

- End-to-end visibility across systems
- Faster root cause analysis
- Identify performance bottlenecks
- Better troubleshooting
- Improved system reliability

BEST PRACTICES

- Propagate Trace Context in all requests.
- Use OpenTelemetry instrumentation in all services.
- Collect logs with Trace ID for correlation.
- Sample traces in production (e.g., 10%)
- Monitor key metrics continuously.

COMMON BACKENDS

- Jaeger – Distributed tracing system
- Zipkin – Open source tracing system
- Tempo – Scalable tracing backend
- OTel Collector – Vendor neutral collector

QUICK TIPS

- Use Correlation ID for logs.
- Use Trace ID for distributed tracing.
- Always visualize traces for complex flows.
- Combine Traces + Metrics + Logs for full observability.

KEY TAKEAWAY

Distributed tracing gives you end-to-end visibility of requests across microservices, helping you detect issues faster and deliver reliable systems.

MODULE 14 | 14.10

LOGGING BEST PRACTICES

MuleSoft from salesforce

Effective logging is critical for monitoring, troubleshooting and ensuring performance of Mule applications. Follow these best practices to keep logs useful, secure and consistent across environments.

1. NEVER LOG PASSWORDS OR TOKENS

- Sensitive data like passwords, tokens, API keys, credit card numbers must never be logged.
- Mask or remove sensitive data before logging.
- Protect your users and comply with security standards.

BAD (Never do this)

```
{
  "user": "john.doe",
  "password": "P@ssw0rd123",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

GOOD (Do this instead)

```
{
  "user": "john.doe",
  "password": "****",
  "token": "****"
}
```

2. STRUCTURED JSON LOGGING

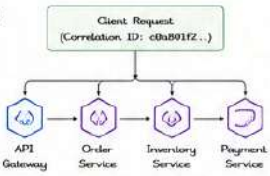
- Use structured JSON logs for consistency and better parsing.
- Makes logs easy to search, filter and analyze.
- Essential for log aggregation tools (ELK, Splunk, Datadog etc.).
- Keep fields simple, flat and meaningful.

Example JSON Log

```
{
  "timestamp": "2024-05-21T10:15:30.123Z",
  "level": "INFO",
  "logger": "com.mule.myapi.order",
  "message": "Order processed successfully",
  "correlationId": "c0a801f2-4b7a-4c6b-bb3e-
                    9f6c1c2a7e11",
  "traceId": "d4e8b9c0a1f2e3d4",
  "spanId": "a1b2c3d4e5f6a7b8",
  "service": "order-service",
  "environment": "prod",
  "status": "SUCCESS",
  "durationMs": 145,
  "httpMethod": "POST",
  "endpoint": "/api/orders"
}
```

3. USE CORRELATION IDS EVERYWHERE

- Always generate/propagate Correlation ID for every request.
- Use Trace ID and Span ID for distributed tracing.
- Include these IDs in every log line.
- Helps track a request across multiple systems and services.



Service	Correlation ID	Trace ID	Span ID
API Gateway	c0a801f2..	d4e8b9c0a1f2e3d4	a1b2c3d4e5f6a7b8
Order Service	c0a801f2..	d4e8b9c0a1f2e3d4	b1c2d3e4f5a6b7c8
Inventory Service	c0a801f2..	d4e8b9c0a1f2e3d4	c1d2e3f4a5b6c7d8
Payment Service	c0a801f2..	d4e8b9c0a1f2e3d4	d1e2f3a4b5c6d7e8

4. AVOID LOGGING LARGE PAYLOADS

- Large request/response payloads increase log size and cost.
- Log only necessary metadata.
- If needed, log a truncated version or payload hash.
- Store full payloads securely in object storage with a reference.

BAD (Logs entire payload)

```
{
  "message": "Request received",
  "payload": { ... huge json ... }
}
```

GOOD (Log metadata only)

```
{
  "message": "Request received",
  "payloadSize": "2456 bytes",
  "payloadHash": "a94a8fe5ccb19ba...",
  "payloadTruncated": true
}
```

5. ERROR LOGGING STANDARDS

- Log errors with proper level (ERROR).
- Always include error message, error code, stack trace and context.
- Do not log and throw generic exceptions.
- Provide meaningful messages for faster root cause analysis.

Example Error Log

```
{
  "timestamp": "2024-05-21T10:20:45.3212",
  "level": "ERROR",
  "logger": "com.mule.myapi.order",
  "message": "Failed to process order",
  "errorCode": "ODER_PROCESSING_FAILED",
  "exception": "java.lang.IllegalStateException",
  "stackTrace": "java.lang.IllegalStateException:...",
  "correlationId": "c0a801f2-4b7a-4c6b-bb3e-9f6c1c2a7e11",
  "traceId": "d4e8b9c0a1f2e3d4",
  "spanId": "b1c2d3e4f5a6b7c8",
  "service": "order-service",
  "environment": "prod"
}
```

6. PRODUCTION LOGGING CHECKLIST

- [x] Use appropriate log levels (TRACE, DEBUG, INFO, WARN, ERROR, FATAL).
- [x] Use structured JSON logging.
- [x] Include Correlation ID, Trace ID and Span ID in every log.
- [x] Never log passwords, tokens, secrets or PII.
- [x] Avoid logging large payloads. Use metadata or hash.
- [x] Log meaningful messages and error codes.
- [x] Include stack traces for errors (not for business exceptions).
- [x] Use Async Logging for performance.
- [x] Roll logs with size & time based policies.
- [x] Separate log files by environment and log level if needed.
- [x] Mask sensitive data if logging is unavoidable.
- [x] Monitor log volume and set alerts for errors.
- [x] Ensure logs are stored and backed up securely.

LOG LEVEL REFERENCE (Quick Guide)

Level	Purpose	Examples
TRACE	Very detailed tracing	Request/response details, flow steps
DEBUG	Debugging information	Variable values, method entry/exit
INFO	General operational information	Request received, processed successfully
WARN	Something unexpected not critical	Retry attempts, minor failures
ERROR	A serious problem	Business or system failures
FATAL	Very severe error, app may crash	OutOfMemoryError, system down

ADDITIONAL TIPS

- Ensure time is logged in UTC format.
- Use consistent field names across all services.
- Review and clean old logs regularly.
- Protect log files with proper access controls.

COMMON LOG FIELDS

timestamp	When the log was created
level	Severity of the log
logger	Source class or component
message	Log message
correlationId	Request Correlation ID
traceId	Distributed Trace ID
spanId	Span ID in the trace
service	Service/Application name
environment	Environment (dev/test/prod)
status	SUCCESS / FAILURE / ERROR
durationMs	Time taken in milliseconds
userId	User identifier (if available)

Good logging is not just about writing logs, it's about writing the right logs.
Follow these best practices to build observable, secure and production-ready Mule applications.

MODULE 15 — 15.1 PROCESSING A 5 GB CSV FILE

MuleSoft from salesforce

Processing huge files like 5 GB CSV is a common enterprise use case. The key is to stream the file, process in batches and avoid loading the entire file into memory.

1. CHALLENGE

- File Size: 5 GB (millions of rows) with multiple columns.
- Goal: Read, validate, transform and load data to a system (Salesforce / Database).
- Constraints:
  - → Limited memory
  - → Avoid timeouts
  - → Ensure scalability & reliability
  - → Handle partial failures

2. WHY NORMAL PROCESSING FAILS

- ✗ Read entire file into memory → OutOfMemoryError.
- ✗ Large payload in memory increases GC pressure.
- ✗ Long running transactions → timeouts.
- ✗ Single commit at the end → data loss risk.
- ✗ High CPU and memory consumption.
- ✗ Poor performance and low scalability.

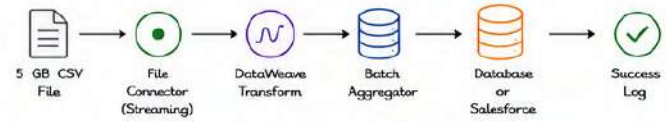
Result:
Application crash, timeouts, high memory usage, unreliable processing.

3. STREAMING SOLUTION

- ✓ Read the file in streaming mode (non-repeatable stream).
- ✓ Process one record at a time (or in small batches).
- ✓ Transform and write in chunks.
- ✓ Commit frequently.
- ✓ Low memory footprint.
- ✓ Highly scalable and reliable.

Key: Never load the entire 5 GB file into memory

4. BATCH JOB ARCHITECTURE



Streaming Read	Transform	Batch Aggregator	Target System	Logs & Monitoring
• Read line by line • No full file load	• Validate • Cleanse • Map fields	• Group records • Commit in batches • Example: 1000 records	• Bulk Insert / Upsert • Database / Salesforce (Bulk API)	• Success / Failure logs • Metrics

5. MULE FLOW OVERVIEW

- **Listener** — Trigger the batch job (Scheduler / HTTP).
- **File (Streaming)** — Read the CSV file in streaming mode.
- **DataWeave Transform** — Transform and validate each record.
- **Batch Aggregator** — Collect records in batches (e.g. 1000).
- **Database / Salesforce** — Bulk insert / upsert the batch.
- **Logger / Monitoring** — Log success, failure, and metrics.

6. DATAWEAVE STREAMING EXAMPLE

```
%dw 2.0
input payload application/csv
output application/json stream=true
---
payload map (row, index) -> {
    id: row["ID"],
    name: row["Name"],
    email: row["Email"],
    amount: (row["Amount"] default "0") as Number
}
```

Notes:
- stream=true ensures output is streamed.
- Each row is processed one by one.

7. BATCH JOB CONFIGURATION

Scheduler	Run the job at required frequency
Batch Size	1000 (tune based on system capacity)
Commit Size	1000
File Read Buffer Size	16 KB / 32 KB
Max Concurrency	1 (increase carefully if target supports it)
Transaction Type	XA (if multiple systems), otherwise LOCAL
Retry Count	3
Error Handling	On Error Continue with logging
Timeout	Set appropriate timeouts for components

Typical Batch Size Guidelines
- 500 – 2000 → Database
- 1000 – 5000 → Salesforce Bulk API
- Always test and tune based on payload size and system capacity.

Batch Aggregator Example

```
<batch:job name="batchJob"
           jobId="csvBatchJob"
           maxFailedRecords="-1">
  <batch:step name="dbStep"
          acceptExpression="#[true]"/>
    <batch:aggregator size="1000"
          timeout="60000"/>
</batch:job>
(Size = 1000 records per batch)
```

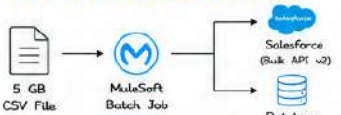
8. MEMORY OPTIMIZATION TECHNIQUES

- ✓ Use streaming file read (non-repeatable stream).
- ✓ Process records one by one (avoid collections).
- ✓ Use batch aggregator to commit in chunks.
- ✓ Avoid storing large data in variables.
- ✓ Use DataWeave streaming (stream=true).
- ✓ Set appropriate JVM heap (e.g. -Xmx2g or as needed).
- ✓ Enable G1GC for better garbage collection.
- ✓ Close resources properly (file, DB connections).
- ✓ Avoid logging entire payloads.

Recommended JVM Settings (Example)

```
-Xms1g -Xmx2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200
```

9. REAL-TIME ENTERPRISE EXAMPLE



Use Case:
Daily customer data load from external system to Salesforce and internal database.

Benefits:
- ✓ Handles millions of records efficiently.
- ✓ Low memory usage.
- ✓ Reliable and fault tolerant.
- ✓ Scalable for larger files.

10. INTERVIEW TIPS

- Always mention streaming first for large files.
- Explain why normal read fails (memory issue).
- Talk about batch aggregator and commit size.
- Discuss error handling and retry strategy.
- Mention logging, monitoring and alerting.
- Explain how you tested with large files.
- Talk about performance tuning and scalability.
- Mention Bulk API for Salesforce.

11. COMMON MISTAKES

- ✗ Reading entire file into memory.
- ✗ Using large batch size without testing.
- ✗ Not handling failures and partial data.
- ✗ Logging entire payloads.
- ✗ No monitoring and alerting.
- ✗ Not tuning JVM and buffer sizes.

12. BEST PRACTICES CHECKLIST

- ☑ Use streaming.
- ☑ Choose optimal batch size.
- ☑ Commit frequently.
- ☑ Handle errors and retries.
- ☑ Use Bulk API for Salesforce.
- ☑ Monitor memory and performance.
- ☑ Log important metrics.
- ☑ Test with production like data.

13. KEY TAKEAWAYS

- ★ Never load 5 GB file fully into memory.
- ★ Use streaming + batch processing.
- ★ Tune batch size and commit size.
- ★ Use proper error handling and logging.
- ★ Monitor memory and performance.
- ★ Test and optimize for your environment.

Remember

Stream → Process → Batch → Commit

= Scalable & Reliable Processing

The key to processing huge files is streaming, batching and smart resource management. This ensures scalability, reliability and high performance in production.

MODULE 15 — 15.2 HANDLING MILLIONS OF SALESFORCE RECORDS

MuleSoft from salesforce

Processing millions of records in Salesforce requires the right API, parallelism, batching and retry strategy. MuleSoft provides powerful tools to load and upsert large volumes of data efficiently and reliably.

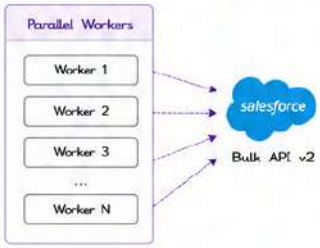
1. BULK API v2 (Recommended)

- Built for high volume data operations.
- Handles millions of records.
- Faster, simpler and more reliable.
- Uses Jobs, Batches and Streams.
- Supports parallel processing.
- Higher daily limits compared to REST API.

Key Points
- Max 150 million records / 24 hours.
- Supports CSV data.
- Automatic batching by Salesforce.
- Best for Insert, Upsert, Update, Delete.

2. PARALLEL PROCESSING

Use multiple parallel threads to upload batches simultaneously.



Benefits
- Reduces total processing time.
- Better utilization of system resources.
- Handles millions of records faster.

Note: Do not exceed Salesforce daily limits.

3. BATCH JOBS (How Salesforce Processes)

- Max → Container for all batches.
- Batch → Group of records.
- Salesforce processes each batch independently.
- Batch size should be optimized (e.g., 10,000 – 50,000 records).
- Monitor job and batch status until completion.

Example	
Total Records	5,000,000
Batch Size	10,000
Total Batches	500
Parallel Workers	5
Batches / Worker	100
Processing Time	Significantly reduced

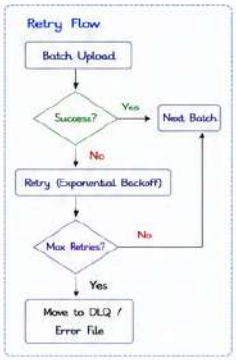
Best Practices
- Keep batch size optimal.
- Use parallel workers.
- Monitor and handle failures.

4. UPSERT vs INSERT

Feature	Insert	Upsert
Definition	Creates new records only.	Creates new or updates existing records.
Duplicate Handling	Fails if duplicate exists.	Updates record if external ID matches.
External ID	Not required.	Required (External ID field).
Use Case	When data is always new.	When data may already exist.
Performance	Slightly faster.	Slightly slower (due to match check).

5. RETRY STRATEGY

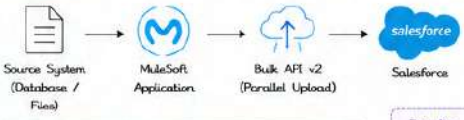
- Network issues, timeouts or locks may cause failures.
- Use automatic retry with exponential backoff.
- Retry only failed batches.
- Download results and reprocess failed records.
- Implement maximum retry attempts to avoid infinite loops.



6. REAL PROJECT EXAMPLE

Scenario: Load 10 Million Customer Records to Salesforce

Architecture



Solution Approach
- Extract data from source in chunks.
- Transform and generate CSV (Streaming).
- Split data into batches (e.g., 20,000 records).
- Upload batches in parallel using Bulk API v2.
- Monitor job, handle failures and retry.
- Log results and send summary report.

Results
- 10 Million records loaded in ~2 hours.
- 99.9% success rate.
- Failures retried automatically.
- Fully scalable and reliable solution.

BEST PRACTICES CHECKLIST

- [x] Use Bulk API v2 for large volume operations.
- [x] Choose optimal batch size (10k – 50k).
- [x] Use parallel processing with multiple workers.
- [x] Monitor job and batch status continuously.
- [x] Handle failures and retry intelligently.
- [x] Use Upsert with External ID for idempotency.
- [x] Do not exceed Salesforce API limits.
- [x] Log and track all operations.
- [x] Validate data before load (data quality).

RECOMMENDED CONFIGURATIONS

Parameter	Recommendation
Batch Size	10,000 – 50,000
Parallel Workers	5 – 20 (based on system capacity)
Max Concurrency	5 – 10
Retry Attempts	3 – 5
Retry Backoff	Exponential (2s, 4s, 8s, 16s...)
Job Timeout	30 minutes – 2 hours
Monitoring Interval	1 – 5 minutes

COMMON CHALLENGES & SOLUTIONS

Too many batches failing
→ Validate data, reduce batch size.

Lock errors
→ Reduce concurrency, use retry.

Timeout issues
→ Increase timeouts, optimize batch size.

API Limit exceeded
→ Reduce workers, schedule during off-peak hours.

Duplicate records
→ Use Upsert with External ID.

INTERVIEW TIPS

- Explain why Bulk API v2 is better than REST for large data.
- How you achieve parallel processing in MuleSoft.
- How you handle failures and retries.
- Difference between Insert and Upsert.
- How you monitor jobs and track results.

KEY TAKEAWAYS

- Use Bulk API v2 + Parallel Processing for millions of records.
- Optimize batch size, concurrency and retry strategy.
- Upsert with External ID for safe and idempotent loads.
- Monitor, handle failures and keep solution scalable.
- Follow best practices to build reliable enterprise integrations.

MODULE 15 15.3

RETRY FAILED RECORDS

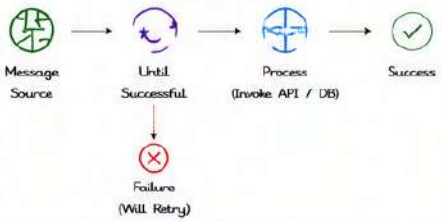
MuleSoft from salesforce

In real-time integrations, failures are inevitable. A robust retry mechanism ensures temporary issues are resolved automatically and only truly bad data is moved to a Dead Letter Queue (DLQ).

1. UNTIL SUCCESSFUL SCOPE

Keeps retrying until the operation is successful or the max attempts are reached.

Mule Flow Example



Key Points

- Retries the entire scope.
- Useful for transient errors (network, timeouts, 5xx).
- Stops when successful or max attempts reached.

2. RETRY POLICIES

Configure how and when Mule should retry.

Policy	Description	Use Case
Simple Retry	Fixed delay between attempts.	Minor delays, occasional failures.
Exponential Backoff Retry	Delay increases exponentially.	APIs / systems under heavy load.
Custom Retry	Custom logic to decide retry.	Business based retry conditions.

Example: Exponential Backoff (XML)

```
<error-handler>
  <on-error-propagate type="ANY">
    <until-successful maxRetries="5" millisBetweenRetries="1000">
      <exponential-backoff-factor>2.0</exponential-backoff-factor>
      <max-millis-between-retries>30000</max-millis-between-retries>
      <!-- Your operation here -->
    </until-successful>
  </on-error-propagate>
</error-handler>
```

Wait Time Sequence: 1s, 2s, 4s, 8s, 16s...

3. DEAD LETTER QUEUE (DLQ)

Messages that fail even after all retry attempts are moved to DLQ for later analysis and reprocessing.

DLQ Should Store

- Original payload
- Error message
- Stack trace
- Failed endpoint
- Retry count
- Timestamp
- Correlation/Message ID

DLQ Options

- DB Table
- File / SFTP
- JMS / Anypoint MQ
- Object Store

4. EXPONENTIAL BACKOFF

Increases the wait time between retries to reduce load on failing systems.

Attempt 1	Attempt 2	Attempt 3	Attempt 4	Attempt 5
1 sec	2 sec	4 sec	8 sec	16 sec

Example

Initial Interval	= 1 sec
Backoff Factor	= 2
Max Interval	= 30 sec
Max Retries	= 5

Benefits

- Reduces system pressure
- Avoids immediate re-failures
- Better success rate over time
- Industry best practice

5. POISON MESSAGES

Messages that will never succeed due to bad data, business rule violations, etc.

Examples

- Invalid data format
- Missing mandatory fields
- Business rule violation
- Permanent downstream rejection (e.g., 400 Bad Request)

Handling Strategy

- Identify using error type / error message.
- Set max retry attempts.
- Move to DLQ after retries exhausted.
- Alert operations team.
- Fix data and reprocess from DLQ.

How to Detect?

- Specific error codes (e.g., 400, 422)
- Validation failures
- Known permanent failures
- Use custom error mapping

6. BEST PRACTICES

Use Until Successful Wisely
- Use only for transient failures.
- Do not retry for business validation errors.

Set Proper Retry Limits
- Configure max attempts and max wait time.
- Avoid infinite retry loops.

Implement Exponential Backoff
- Prevents system overload.
- Gives time for system recovery.

Always Use DLQ
- Never lose failed messages.
- Store enough context for debugging.

Monitor & Alert
- Monitor retry count, DLQ size.
- Alert when DLQ is not empty.

Reprocess Safely
- Fix data and reprocess from DLQ.
- Ensure idempotent operations.

SAMPLE ERROR HANDLER (XML)

```
<error-handler>
  <on-error-propagate type="HTTP:CONNECTIVITY, HTTP:TIMEOUT">
  <until-successful maxRetries="5" millisBetweenRetries="1000">
    <exponetical-backoff-factor>2.0</exponential-backoff-factor>
    <max-millis-between-retries>30000</max-millis-between-retries>
    <!-- Your operation here -->
  </until-successful>
  <on-error-continue type="ANY">
    <logger level="ERROR" message="Failed after retries: #[error.description]"/>
    <flow-ref name="writeToDLQ"/>
  </on-error-continue>
  </on-error-propagate>
</error-handler>
```

RETRY STRATEGY FLOW

INTERVIEW TIPS

- Explain difference between transient and permanent failures.
- Why exponential backoff is better than fixed retry?
- How would you handle poison messages?
- Where will you store DLQ and why?
- How will you reprocess messages from DLQ?
- How do you ensure no data loss during failures?

KEY TAKEAWAY

A good retry strategy = Faster recovery + No data loss + Happy downstream systems + Easy troubleshooting.

MODULE 15 **15.4** # DATABASE CONNECTION FAILURES

MuleSoft from salesforce

Database connectivity issues are common in enterprise systems.
A robust integration must handle failures gracefully and recover automatically without data loss.

1. DB CONNECTIVITY ISSUES

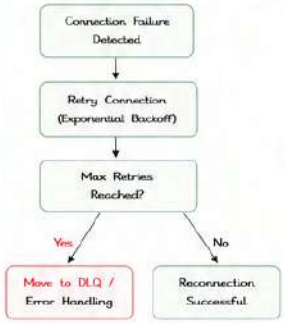
Common reasons for database connection failures:

- Database server down or unreachable
- Network issues (latency, packet loss, firewall)
- Connection pool exhausted
- Invalid credentials or expired password
- Database restart or maintenance
- DNS resolution failures
- Too many concurrent connections
- Query timeout or long running transactions
- Database resource exhaustion (CPU, Memory, Locks)

Symptoms

- Connection timeout
- Socket timeout
- Connection refused
- SQLTransientConnectionException
- Communications link failure
- Too many connections

2. RECONNECTION STRATEGY



Best Practices

- Use automatic reconnection with retry.
- Use exponential backoff to avoid retry storm.
- Configure max retries based on business needs.
- Log and alert after max retries.
- Use idempotent operations to avoid duplicates.

3. CONNECTION POOLING

Connection pooling improves performance and handles failures better.

Parameter	Description	Recommended Value
Max Pool Size	Maximum number of connections in the pool	10 – 50 (based on DB capacity)
Min Pool Size	Minimum number of connections kept in pool	2 – 5
Idle Timeout	Close idle connections after configured time	30 – 60 seconds
Acquire Timeout	Time to wait for connection from pool	5 – 30 seconds
Validation Query	Query to validate connection before using	SELECT 1 (or DB specific)
Test While Idle	Validate idle connections	true

Note: Proper pooling prevents connection leaks and improves scalability.

4. TRANSACTION HANDLING

Ensure data consistency even when failures occur.

Best Practices

- Use local transactions for single DB operations.
- Use XA transactions for multiple resources (DB + JMS, DB + MQ).
- Always rollback on failure.
- Keep transactions short and focused.
- Avoid long running transactions.
- Handle exceptions and rollback properly.

Example Flow

5. TIMEOUT CONFIGURATION

Proper timeout settings prevent hanging requests and free up resources.

Timeout Type	Description	Recommended Value
Connection Timeout	Time to establish connection to DB	5 – 10 seconds
Socket Timeout	Time to wait for data from DB	30 – 120 seconds
Query Timeout	Max time a query can run	30 – 300 seconds (based on query)
Transaction Timeout	Max time a transaction can remain open	60 – 300 seconds
Idle Timeout	Close idle connections	30 – 60 seconds

6. RECOVERY STRATEGY

1. **Detect Failure** – Monitor connection and query failures using error handlers and logs.
2. **Retry with Backoff** – Retry using exponential backoff until success or max retries.
3. **Failover (If Available)** – Use DB failover / replication if primary DB is down.
4. **Move to DLQ** – If all retries fail, store the message in DLQ for manual reprocessing.
5. **Alert & Monitor** – Notify support team and monitor for resolution.

Tools & Techniques

- **DB Health Check (Scheduler)** – Periodic query (e.g., SELECT 1) to verify DB availability.
- **Circuit Breaker Pattern** – Stop calling DB when failures cross the threshold.
- **DLQ for Failed Records** – Avoid data loss and allow reprocessing after recovery.
- **Monitoring & Alerting** – Use Anypoint Monitoring, Custom Alerts, Slack/Email.

SAMPLE CONFIGURATION (DB CONFIG)

```
<db:config name="DB_Config" doc:name="Database Config">
  <db:connection-pooling-profile ...>
    <db:pooling-profile maxPoolSize="20" minPoolSize="5"
        idleTimeout="60000" acquireTimeout="10000"
        validationQuery="SELECT 1" testWhileIdle="true"/>
    <db:reconnection frequency="5000" count="5"/>
    <db:connection validateConnections="true"
        validateFrequency="60000"/>
  </db:connection-pooling-profile>
  <db:transaction-settings action="ALWAYS_BEGIN"
      timeout="120"/>
</db:config>
```

EXAMPLE ON-ERROR-HANDLER (RECONNECT & RETRY)

```
<error-handler>
  <on-error-propagate type="DB:CONNECTIVITY">
    <until-successful maxRetries="5" millisBetweenRetries="2000"
        exponential-backoff-factor="2.0"
        maxMillisBetweenRetries="30000">
      <logger level="WARN"
          message="DB connection failed. Retrying..." />
      <flow-ref name="Reprocess_Flow" />
    </until-successful>
    <on-error-continue>
      <set-payload value="#[payload]" />
      <flow-ref name="Send_To_DLQ" />
    </on-error-continue>
  </on-error-propagate>
</error-handler>
```

INTERVIEW TIPS

- Explain common DB connection issues and their impact.
- How would you handle transient vs permanent failures?
- Why connection pooling is important?
- How to avoid connection leaks?
- How would you ensure data consistency during failures?
- Difference between local and XA transactions?
- How would you configure retries with exponential backoff?

KEY TAKEAWAY

Handle failures gracefully, retry intelligently, use pooling efficiently, and ensure data consistency. This makes your integration reliable and production-ready.

MODULE 15 15.5

API TIMEOUT HANDLING

MuleSoft from salesforce

APIs may be slow or unresponsive due to network issues, high load or downstream failures.
Proper timeout handling ensures your application remains responsive and resilient.

1. HTTP TIMEOUT (Connection Timeout)

Time taken to establish a connection with the server.
If the connection is not established within the timeout, the request fails.

Example Use Cases
- Server down
- Network issues
- DNS resolution delay
- Firewall or routing issues

Config (HTTP Request)
```
<http:request ... >
    <http:request-connection
      connectionTimeout="5000"/>
</http:request>
```
Recommended: 3s – 10s

2. READ TIMEOUT (Socket Timeout)

Time to wait for data after the connection is established.
If no data is received within the timeout, the request fails.

Example Use Cases
- Slow server processing
- Database queries taking long
- Large payload generation delay
- Network latency while reading data

Config (HTTP Request)
```
<http:request ... >
    <http:request-connection
      socketTimeout="30000"/>
</http:request>
```
Recommended: 10s – 60s

3. RESPONSE TIMEOUT (Overall Timeout)

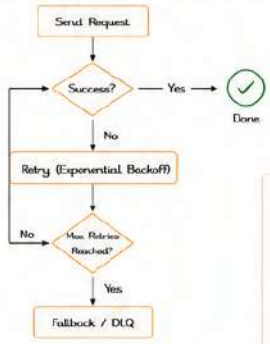
Total time allowed for the entire request-response cycle.
If the response is not received within this time, the request fails.

Example Use Cases
- End-to-end SLA
- Prevent hanging requests
- Protect resources
- User experience control

Config (HTTP Request)
```
<http:request ... >
    <http:request-connection
      responseTimeout="60000"/>
</http:request>
```
Recommended: 30s – 120s

4. RETRY LOGIC

Retries help overcome transient failures.



Exponential Backoff Example
- Attempt 1 → wait 1 sec
- Attempt 2 → wait 2 sec
- Attempt 3 → wait 4 sec
- Attempt 4 → wait 8 sec
- ...
- Max Attempt = 5

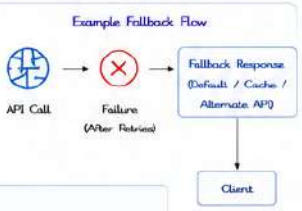
Mule (XML) - Retry Policy
```
<error-handler>
    <retry-policy maxRetries="5"
                  initialInterval="1000"
                  maxInterval="16000"
                  backOffMultiplier="2" />
</error-handler>
```

5. FALLBACK RESPONSE

When all retries fail, return a meaningful response or route to an alternative flow.

Fallback Strategies
- Return default/meaningful response
- Use Cache or Static Data
- Call alternative system
- Route to DLQ for manual reprocessing
- Notify & Alert

Example Fallback Flow



Example: Return Default Response
```
%dw 2.0
output application/json
---
{
  "status": "SERVICE_UNAVAILABLE",
  "message": "We are experiencing issues. Please try again later.",
  "code": "503",
  "timestamp": now()
}
```

6. BEST PRACTICES

- Set appropriate timeouts based on the backend SLA.
- Use separate timeouts: Connection, Socket(Read) and Response.
- Implement retry only for idempotent and safe operations.
- Use exponential backoff to avoid retry storms.
- Always provide meaningful fallback responses.
- Log and monitor timeout events.
- Use circuit breaker pattern for repeated failures.
- Test timeout scenarios in lower environments.
- Avoid infinite retries.
- Tune timeouts based on real performance metrics.

TIMEOUT RECOMMENDATIONS (Guideline)

Timeout Type	Description	Recommended Range
Connection Timeout	Time to establish connection	3s - 10s
Read (Socket) Timeout	Time to wait for data after connection	10s - 60s
Response Timeout	Total time for entire request-response	30s - 120s

COMMON TIMEOUT ERRORS

- java.net.ConnectException: Connection timed out
- java.net.SocketTimeoutException: Read timed out
- org.mule.transport.http.HttpRequestTimeoutException
- 504 Gateway Timeout (from upstream gateway)
- 503 Service Unavailable (due to timeout)

SAMPLE: COMPLETE TIMEOUT CONFIGURATION

```
<http:request config-ref="HTTP_Request_Configuration" path="/api/orders"
method="POST">t-connection
      connectionTimeout="5000"
      socketTimeout="30000"
      responseTimeout="60000" />
</http:request>
```

Global Config Example
```
<http:request-configuration name="HTTP_Request_Configuration">
   <http:request-connection
      connectionTimeout="5000"
      socketTimeout="30000"
      responseTimeout="60000" />
</http:request-configuration>
```

ERROR HANDLING FLOW (Overview)

INTERVIEW TIPS

- What is the difference between connection, read and response timeout?
- How do you implement retry with exponential backoff in MuleSoft?
- When should we not retry an API call?
- How do you handle non-idempotent operations?
- What is the advantage of fallback response?
- How do you prevent retry storms?
- How do you monitor and alert on timeout failures?

KEY TAKEAWAY

Proper timeout configuration + smart retry + meaningful fallback = Resilient & user-friendly integrations.

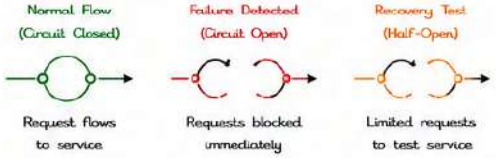
MODULE 15 **15.6** # CIRCUIT BREAKER PATTERN

MuleSoft from salesforce

The Circuit Breaker pattern prevents an application from repeatedly trying to execute an operation that is likely to fail. It helps in building resilient and failure-tolerant integrations.

1. WHAT IS CIRCUIT BREAKER?

Circuit Breaker acts like an electrical circuit breaker. When failures occur frequently, it "opens the circuit" and stops requests to the failing service for a period of time.



Key Benefits

- Prevents cascading failures across systems.
- Reduces load on failing services.
- Improves system stability and user experience.
- Provides time for the failing service to recover.

2. CIRCUIT BREAKER STATES

CLOSED STATE

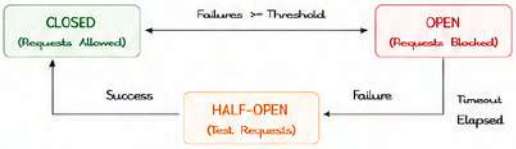
- Normal operation.
- All requests are allowed to pass to the service.
- Circuit monitors failures.
- If failures exceed threshold → OPEN.

OPEN STATE

- Circuit is open.
- All requests fail fast without calling the service.
- Wait for a configured timeout.
- After timeout → move to HALF-OPEN.

HALF-OPEN STATE

- Allows a limited number of test requests.
- If test succeeds → close the circuit.
- If test fails → open the circuit again and wait for timeout.

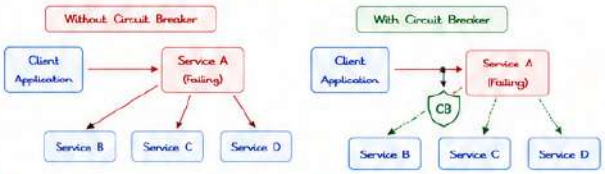


3. STATE DETAILS

State	When it Happens	What Happens	Next Transition
CLOSED	System is healthy	Requests go to target service. Failures are counted.	If failures >= Threshold → OPEN
OPEN	Failures exceed threshold	All requests are rejected immediately (fail fast).	After wait duration → HALF-OPEN
HALF-OPEN	After wait duration in OPEN state	A limited number of requests are allowed to test the service.	Success → CLOSED Failure → OPEN

Fail Fast: Do not call the service when circuit is OPEN. Return an error or fallback.

4. HOW IT PREVENTS CASCADING FAILURES



- ✗ Service A is down or slow.
- ✗ All requests keep hitting it.
- ✗ Threads, connections and resources get exhausted.
- ✗ Service B, C, D also start failing.
- ✗ Total system failure (cascade)

- ✓ Circuit opens when Service A fails.
- ✓ Requests are blocked (fail fast).
- ✓ No unnecessary load on Service A.
- ✓ Other services remain healthy.
- ✓ System stays stable.

5. CIRCUIT BREAKER CONFIGURATION (MULESOFT EXAMPLE)

Sample Policy (XML)

```
<circuitBreaker:config name="CB_Config">
  <circuitBreaker:basic
    failureThreshold="5"
    recoveryTimeout="60000"
    successThreshold="2"
    maxRequestCount="1"
    enabled="true"/>
</circuitBreaker:config>
```

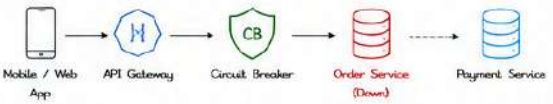
- failureThreshold: Number of failures in CLOSED state before opening the circuit.
- recoveryTimeout: Time in milliseconds the circuit stays in OPEN state.
- successThreshold: Number of successful requests in HALF-OPEN state to close the circuit.
- maxRequestCount: Number of test requests allowed in HALF-OPEN state.
- enabled: Enable or disable the circuit breaker.

How to Use

1. Configure policy in global elements.
2. Reference the policy in an HTTP Request, DB, or custom operation.
3. Mule runtime automatically manages the states.

6. ENTERPRISE EXAMPLE

Scenario: E-commerce Order Processing



Normal (Closed)

- Order service is healthy.
- All order requests go through.
- Circuit is CLOSED.

Failure Occurs (Open)

- Order service goes down.
- After 5 failures, circuit opens.
- Requests fail fast, not sent to Order Service.

Recovery (Half-Open)

- After 60 sec, circuit allows 1 test request.
- If successful → circuit closes.
- If fails → stays open and wait again.

Benefits

- ✓ Users get quick response.
- ✓ System resources are protected.
- ✓ Other services like Payment remain available.
- ✓ Better overall system resilience.

BEST PRACTICES

- ✓ Set appropriate failure threshold (not too low, not too high).
- ✓ Use reasonable recovery timeout.
- ✓ Keep max request count low in HALF-OPEN state.
- ✓ Always provide meaningful fallback or error message when OPEN.
- ✓ Monitor circuit breaker metrics (failures, state changes, retries).
- ✓ Combine with retries and timeouts for better resilience.
- ✓ Test failure scenarios in lower environments.

INTERVIEW TIPS

- Explain the 3 states clearly with transitions.
- Why is fail fast important in OPEN state?
- How does Circuit Breaker help in microservices / SOA?
- Difference between Retry and Circuit Breaker?
- What happens if recovery timeout is too short or too long?
- How would you configure Circuit Breaker in MuleSoft?

Circuit Breaker = Fail Fast + Protect Resources + Recover Gracefully

Build resilient integrations, prevent cascades, and keep your systems stable!

MODULE 15 15.7

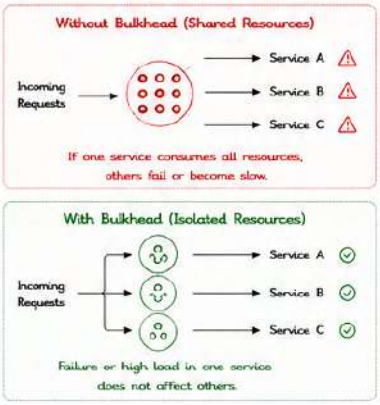
BULKHEAD PATTERN

MuleSoft from salesforce

Isolate resources into separate compartments (bulkheads) so a failure in one part does not affect other parts of the system.

1. RESOURCE ISOLATION

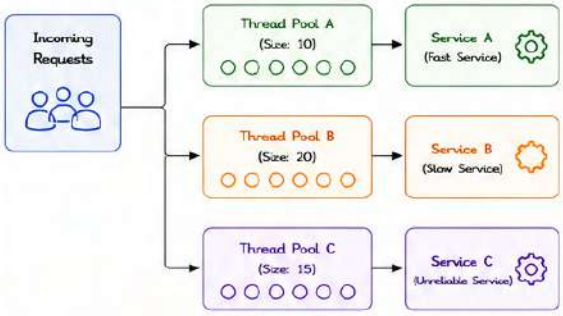
The Bulkhead Pattern isolates resources so that failures, slow operations or high load in one component do not impact others. Each "compartment" has its own resources.



Benefits
- Resource isolation
- Improved stability
- Better fault tolerance
- Predictable performance
- Easier capacity management

2. THREAD POOLS

Assign dedicated thread pools (or connection pools, queues, etc.) to isolate workloads.

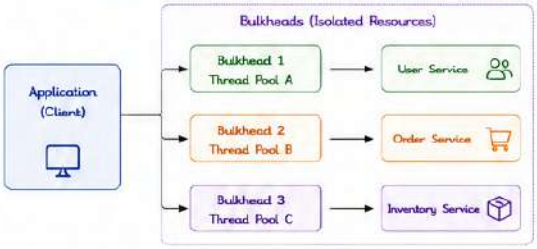


Key Point

Each service has its own quota of threads. If one pool is exhausted, other pools continue to operate normally.

3. INDEPENDENT SERVICES

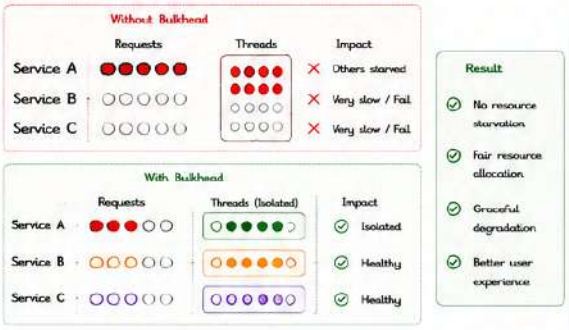
Bulkheads ensure each downstream dependency is called through its own isolated resources.



A problem in User Service does not impact Order or Inventory Service. Each dependency is independent and isolated.

4. PREVENT RESOURCE STARVATION

Without bulkheads, one component can consume all resources, causing thread starvation and system-wide failures.



Result
- No resource starvation
- Fair resource allocation
- Graceful degradation
- Better user experience

5. HIGH AVAILABILITY

Bulkheads improve availability by containing failures and enabling graceful degradation.

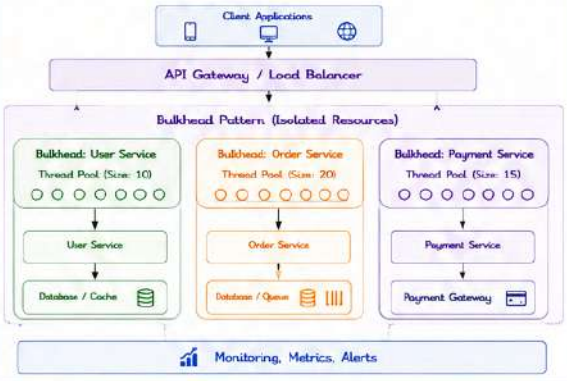
- **Fault Isolation** — Failures are contained within a bulkhead.
- **Graceful Degradation** — Only the failing component is affected.
- **Better Resilience** — Other services continue to serve requests.
- **Easier Recovery** — Individual compartments recover independently.

Example Scenario

If Inventory Service is down:
- Inventory bulkhead is exhausted
- Order and User services continue to work
- System remains available

> "Bulkheads keep the ship afloat even if one compartment is leaking."

6. EXAMPLE ARCHITECTURE



KEY TAKEAWAYS

- Bulkhead Pattern isolates resources into separate compartments.
- Use dedicated thread pools/connection pools for each dependency.
- Prevents a single point of failure from impacting the entire system.
- Improves stability, resilience and overall system performance.
- Essential for building scalable, highly available enterprise systems.

BEST PRACTICES

- Define bulkheads per dependency or service.
- Set appropriate limits for thread pool sizes.
- Monitor usage and tune limits based on metrics.
- Combine with Circuit Breaker for better resilience.
- Test failure scenarios to validate isolation.

MODULE 15 15.8

IDEMPOTENCY

MuleSoft from salesforce

Idempotency ensures that repeated requests with the same intent have the same effect as a single request.
It helps prevent duplicate processing and ensures data consistency in distributed systems.

1. WHAT IS IDEMPOTENCY?

An operation is idempotent if applying it multiple times produces the same result as applying it once.

Idempotent Operation (Same Result)
Create Order (idempotent)

Request (1st time)	→	Order Created
Same Request (Repeat)	→	No Change (Already Created)

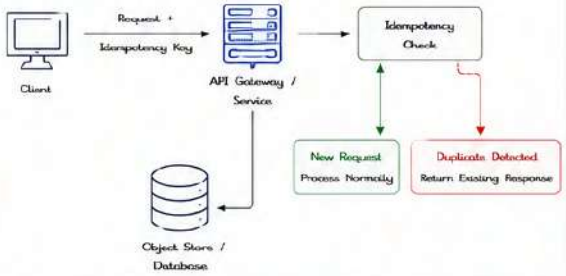
Non-Idempotent Operation (Different Result)
Charge Card (non-idempotent)

Request (1st time)	→	Charged $100
Same Request (Repeat)	→	Charged $200 (Duplicate Charge)

Benefits
- Prevents duplicate side effects
- Handles retries safely
- Improves reliability and user trust

2. DUPLICATE REQUEST PREVENTION

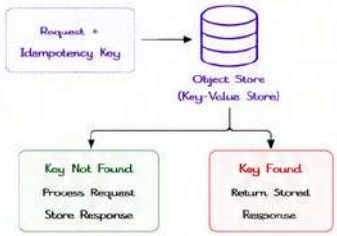
Idempotency ensures that duplicate requests are detected and handled safely.



Key Idea: Use a unique key to identify the request and its outcome.

3. OBJECT STORE

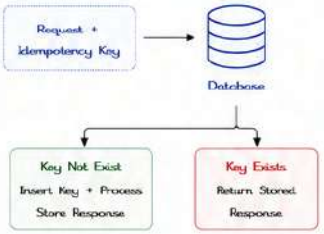
Store the response against the idempotency key in a fast key-value store (e.g., Redis, DynamoDB).



Best for
- High throughput systems
- Fast lookup and response
- Short TTL (e.g., 24–72 hours)

4. DATABASE CHECK

Store idempotency keys and status in a database with a unique constraint.



Best for
- Relational databases (ACID compliance)
- Durable storage
- Complex queries & auditing

5. IDEMPOTENCY KEYS

A unique key is generated by the client and sent with the request.

Guidelines
- Key must be unique per request intent.
- Use UUIDv4 or a sufficiently random value.
- Include key in request header (e.g., Idempotency-Key).
- Store key with request hash/metadata for safety.
- Set an expiration (TTL) to clean up old keys.

Example Request

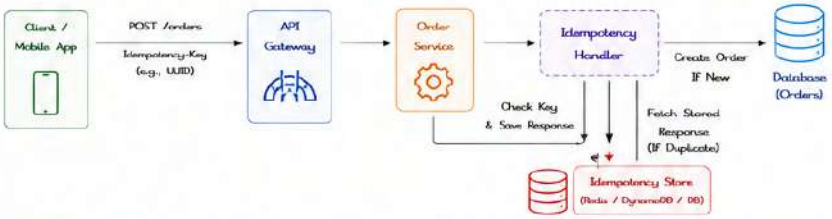
```
POST /orders HTTP/1.1
Host: api.example.com
Content-Type: application/json
Idempotency-Key: 8f4c2a9b-4b0a-4b1f-8a5d-9c0e2d3f7b1a
{
  "item": "Laptop",
  "amount": 1200,
  "currency": "USD"
}
```

Example Response (Duplicate Request)

```
{
  "status": "success",
  "message": "Duplicate request. Returning original response.",
  "data": { "orderId": "ORD-12345", "amount": 1200 }
}
```

6. ENTERPRISE EXAMPLE

Scenario: E-commerce Order Creation



Flow
1. Client sends request with Idempotency-Key.
2. API Gateway forwards to Order Service.
3. Idempotency Handler checks the key.
4. If key exists → return stored response.
5. If key not exists → create order, store response with key.
6. Return response to client.

Result: Duplicate requests return the same response without creating multiple orders.

KEY TAKEAWAYS

- Idempotency ensures safety in retries and network failures.
- Always use a unique key to identify request intent.
- Store the result of the first request and reuse it.
- Choose the right storage (Object Store or Database).
- Idempotency is essential for reliable, scalable integrations.

BEST PRACTICES

- Generate keys on the client side.
- Validate and enforce key presence.
- Use appropriate TTL for stored keys.
- Store request hash to detect payload changes.
- Monitor duplicate rates and key store size.
- Clean up expired keys periodically.

COMMON USE CASES

- Order creation
- Payment processing
- Subscription signup
- Inventory reservation
- Any POST/PUT operations with side effects

Idempotency turns "at-least-once" delivery into "exactly-once" effect.
It's a cornerstone for building reliable, fault-tolerant systems.

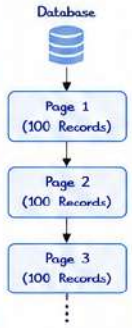
MODULE 15 15.9

PAGINATION

MuleSoft from salesforce

Pagination is a technique used to retrieve large datasets in smaller chunks instead of loading everything at once. It improves API performance, reduces memory usage, and enables scalable enterprise integrations.

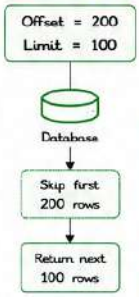
1. WHAT IS PAGINATION?



Key Points

- Retrieves data page by page
- Improves response time
- Reduces memory usage
- Supports millions of records
- Used in APIs and databases

2. OFFSET PAGINATION



Example (API)

```
GET /customers?
offset=200&limit=100
```

SQL Example

```
SELECT *
FROM CUSTOMER
LIMIT 100 OFFSET 200;
```

Advantages

- Easy implementation
- Good for small datasets

Disadvantages

- Slow for millions of rows
- Database scans skipped records

3. LIMIT PAGINATION

Returns only a fixed number of records.

Example (API)

```
GET /orders?limit=50
```

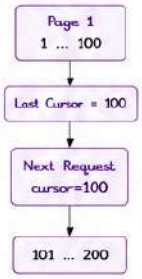
Example Output

Records Returned
1
2
3
...
50

Best Use

- Dashboard
- Reports
- Simple APIs

4. CURSOR PAGINATION



Example (API)

```
GET /orders?cursor=100
```

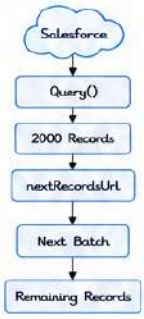
How it works

Page 1 returns records 1–100 and provides the last cursor (100). Next request uses that cursor to get the next set (101–200).

Advantages

- Extremely fast
- No skipped rows
- Best for millions of records
- Enterprise recommendation

5. SALESFORCE PAGINATION



Key Concepts

- Query Locator
- nextRecordsUrl
- queryMore()
- Bulk API v2
- Bulk Query

Best Practice

Use Bulk API v2 for millions of Salesforce records.

6. DATABASE PAGINATION

Common Examples

Database	Example
MySQL	LIMIT 100 OFFSET 200
PostgreSQL	LIMIT 100 OFFSET 200
Oracle	OFFSET 200 ROWS FETCH NEXT 100 ROWS ONLY
SQL Server	OFFSET 200 ROWS FETCH NEXT 100 ROWS ONLY

Note

Always create indexes on filtering columns to improve pagination performance.

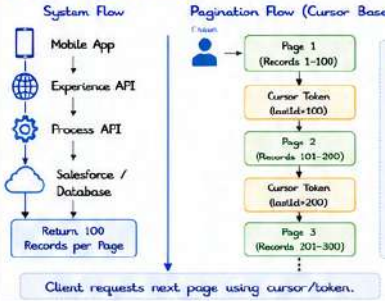
7. PERFORMANCE TIPS

- Prefer Cursor Pagination
- Use Database Indexes
- Return Required Columns Only
- Avoid SELECT *
- Keep Page Size Configurable
- Cache Frequently Accessed Pages
- Use Streaming for Huge Files
- Avoid Large OFFSET values

8. PAGINATION COMPARISON

Type	Speed	Large Data	Easy	Enterprise	Best Use
Offset Pagination	Medium	✗ No	✓ Yes	Limited	Small datasets
Limit Pagination	Medium	✗ No	✓ Yes	Limited	Simple APIs
Cursor Pagination	Excellent	✓ Yes	Medium	Recommended	Large datasets
Salesforce QueryMore	Excellent	✓ Yes	✓ Yes	Recommended	Salesforce data
Bulk API v2	Excellent	✓ Millions	✓ Yes	Best	Millions of records

9. ENTERPRISE EXAMPLE



How it works

Client requests the first page. API returns records and a cursor token. Client sends the cursor to get the next page.

10. BEST PRACTICES

- Use Cursor Pagination whenever possible
- Keep page size between 100–1000
- Index database columns
- Validate pagination parameters
- Avoid huge OFFSET values
- Log pagination metrics
- Secure pagination parameters
- Return total count only when required

11. INTERVIEW QUESTIONS

- What is Pagination?
- Difference between Offset and Cursor Pagination?
- Which pagination performs best?
- How does Salesforce QueryMore work?
- Why avoid OFFSET on large datasets?
- What is nextRecordsUrl?
- When should Bulk API v2 be used?

12. KEY TAKEAWAYS

- Pagination improves API scalability.
- Cursor Pagination is the preferred enterprise solution.
- Salesforce uses Query Locator and nextRecordsUrl.
- Proper indexing significantly improves pagination performance.
- Pagination minimizes memory usage and accelerates response times.

Efficient Pagination = Faster APIs + Lower Memory Usage + Better User Experience + Enterprise Scalability

MODULE 15 15.10

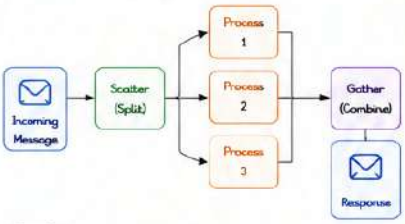
PARALLEL PROCESSING

MuleSoft from salesforce

Parallel Processing allows multiple tasks to run concurrently, improving performance, reducing response time and maximizing system resources.

1. SCATTER-GATHER

Splits a message into multiple parts (scatter), processes them in parallel, and then combines the results (gather).

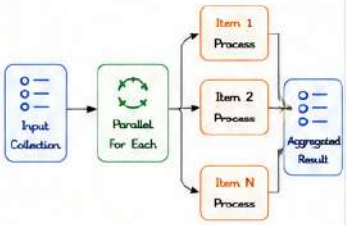


Benefits

- Improves performance for independent tasks
- Reduces overall processing time
- Ideal when order of results is not critical

2. PARALLEL FOR EACH

Iterates over a collection and processes each item in parallel.

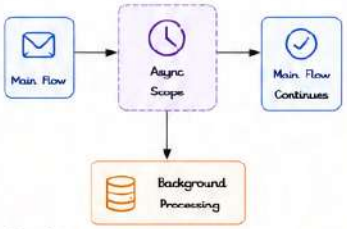


Benefits

- Processes multiple items concurrently
- Great for large collections
- Significant reduction in total processing time

3. ASYNC SCOPE

Executes operations asynchronously without blocking the main flow.

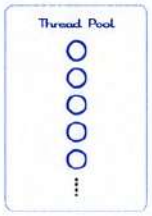


Benefits

- Non-blocking execution
- Improves throughput
- Ideal for notifications, logging, audit, etc.

4. THREAD MANAGEMENT

Mule uses thread pools to manage parallel execution. Proper configuration is critical for performance and stability.



Key Points

- Threads are reused from the pool
- Avoid creating too many threads
- Configure pool size based on system capacity
- Monitor CPU, Memory and Queue size

Configuration (example)

```
<thread-pool name="custom-pool"
   coreThreads="10"
   maxThreads="50"
   queueCapacity="1000"
   keepAliveTime="60" />
```

5. COMMON USE CASES

E-Commerce
Process order, inventory update, payment and notification in parallel

Logistics
Track shipment, update status, send email & SMS in parallel

Banking
Validate account, check balance, fraud check, send alerts

System Integrations
Call multiple systems/APIs concurrently

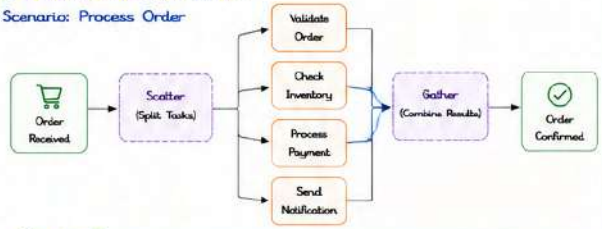
Notifications
Send email, SMS, push notification in parallel

6. BEST PRACTICES

- Use Scatter-Gather for independent operations.
- Use Parallel For Each for large collections.
- Use Async Scope for non-critical, fire-and-forget tasks.
- Configure thread pools appropriately.
- Avoid blocking operations inside parallel flows.
- Handle errors in each parallel route.
- Use timeouts to prevent hanging threads.
- Monitor application performance and resource utilization.
- Test under production-like load.

7. EXAMPLE ARCHITECTURE

Scenario: Process Order



How it works

1. Incoming order is scattered into independent tasks.
2. All tasks execute in parallel.
3. Results are gathered and combined.
4. Final response is returned to the client.
5. Total processing time is reduced.

Why Parallel Processing?

- Reduces overall response time
- Increases system throughput
- Better resource utilization
- Improves user experience

Key Components

Scatter-Gather
Split and aggregate messages

Parallel For Each
Parallel iteration over collection items

Async Scope
Run in background without blocking

Thread Pools
Manage and reuse threads efficiently

Parallel Processing = Concurrency + Performance + Scalability

Use the right pattern, manage threads wisely, and design for reliability.

MODULE 15 | 15.11

LARGE FILE STREAMING

MuleSoft from salesforce

Streaming allows Mule applications to process large files and payloads without loading the entire content into memory. This improves performance, reduces memory usage and enables reliable processing of large data.

1. REPEATABLE STREAMS

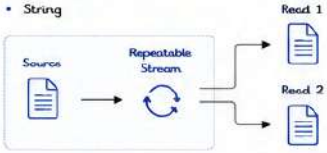
The stream can be read multiple times.

Characteristics

- Can be re-read any number of times
- Underlying data is cached or stored
- More memory usage (depends on size)

Examples

- File connector with repeatable=true (default)
- Byte Array
- String



2. NON-REPEATABLE STREAMS

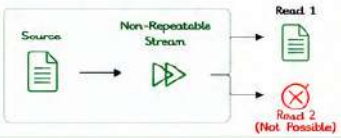
The stream can be read only once.

Characteristics

- Can be read only one time
- Data is consumed as it is read
- Very low memory usage (streaming)

Examples

- File connector with repeatable=false
- HTTP Request (streaming response)
- JMS, MQ, SFTP streaming



3. FILE CONNECTOR STREAMING

Read or write large files as a stream to avoid loading entire file in memory.

Configuration

```
<file:read
  path="/data/largefile.csv"
  outputMimeType="text/csv"
  repeatable="false"
  streamingStrategy="AUTO" />
```

Key Points

- repeatable=false enables streaming.
- Streaming strategy AUTO or ALWAYS.
- Works with very large files efficiently.
- Ideal for ETL and data migration.

4. HTTP STREAMING

Stream large HTTP request/response payloads instead of buffering entire content.

Examples

- Streaming request (upload large file)

```
<http:request method="POST"
  url="https://api.example.com/upload"
  responseTimeout="60000"
  streamingStrategy="ALWAYS" />
```

- Streaming response (download large file)

```
<http:request method="GET"
  url="https://api.example.com/largefile"
  streamingStrategy="ALWAYS"
  responseTimeout="60000" />
```

Benefits

- Handles large payloads without memory issues.
- Faster time to first byte.
- Better scalability and throughput.

5. DATAWEAVE STREAMING

DataWeave can stream large payloads using readUrl, read, and streaming operators.

Example: readUrl (Stream CSV)

```
%dw 2.0
output application/json
---
(readUrl("/data/largefile.csv", "application/csv",
{streaming: true}))
  map (row) -> {
    id: row.id as Number,
    name: row.name,
    amount: row.amount as Number
  }
```

Key Points

- readUrl with streaming:true reads line by line.
- Avoids loading entire file in memory.
- Use map, filter, reduce for streaming transformation.
- Works well for CSV, JSON (NDJSON), XML.

6. MEMORY OPTIMIZATION

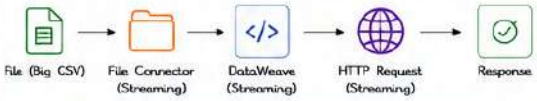
Best practices to reduce memory usage while processing large files.

- Use non-repeatable streams (repeatable=false).
- Enable streamingStrategy (AUTO or ALWAYS).
- Use DataWeave streaming functions.
- Avoid storing payloads in variables unnecessarily.
- Process data in chunks when possible.
- Close resources (file, http) properly.
- Use Object Store or external storage for intermediate data.
- Monitor heap usage and tune JVM.
- Avoid converting streams to String/Bytes.

7. STREAM TYPE COMPARISON

Type	Repeatable	Can Read Multiple Times?	Memory Usage	Use Case
Repeatable Stream	✓	Yes	Higher (depends on size)	Small/Medium files, Multiple processors
Non-Repeatable Stream	✗	No	Very Low	Large files, High performance flows
HTTP Streaming	✗	No	Very Low	Large downloads/ uploads
File Streaming	✗	No (if repeatable=false)	Very Low	Read/Write large files
DataWeave Streaming	✗	No	Very Low	Transform large payloads

8. LARGE FILE PROCESSING FLOW (EXAMPLE)



How it works

1. File is read as a non-repeatable stream.
2. DataWeave processes line by line (streaming).
3. HTTP request sends data in streaming mode.
4. Response is also streamed back.
5. Very low memory footprint.

9. USE CASES

- **ETL / Data Migration** — Process large datasets efficiently.
- **File Upload / Download** — Handle very large files over HTTP.
- **Log Processing** — Stream and analyze large log files.
- **Real-time Integrations** — Stream data between systems.

10. BEST PRACTICES

- Always prefer streaming for large files (> 10 MB).
- Use repeatable=false unless re-read is required.
- Enable proper streamingStrategy in connectors.
- Use DataWeave streaming functions.
- Avoid converting streams to objects or strings.
- Process and discard data as soon as possible.
- Monitor heap, CPU and GC performance.
- Test with production-like large data volumes.

11. KEY TAKEAWAYS

- Streaming prevents memory overload.
- Non-repeatable streams are best for large files.
- File, HTTP and DataWeave support streaming.
- Proper configuration ensures high performance.
- Streaming enables scalable and reliable integrations.

Stream, don't load! Use streaming to handle large data with maximum efficiency.

Lower memory usage • Higher throughput • Better scalability

MODULE 15 15.12

BATCH AGGREGATOR

MuleSoft from salesforce

The Batch Aggregator collects messages and groups them into batches based on a defined batch size and/or timeout. Once the batch is complete, it is released for further processing. This improves performance and reduces system overhead.

1. BATCH AGGREGATOR

Aggregates incoming messages into a batch and releases the batch when conditions are met.

Benefits

- Improves throughput
- Reduces number of transactions
- Efficient processing of large volumes
- Better resource utilization

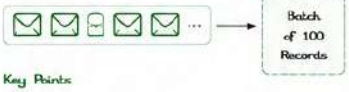
2. BATCH SIZE

Defines the number of messages to collect before releasing the batch.

Example

Batch Size = 100

When 100 messages are collected, the batch is released.



Key Points:

- Trigger based on number of messages.
- Ideal for high volume processing

3. COMMIT SIZE

Defines the number of records to be committed in each transaction when processing the batch.

Example

Commit Size = 50

If batch size is 100, records will be committed in 2 transactions of 50 each.

Key Points

- Helps avoid large transactions.
- Improves performance and reduces locks.

4. AGGREGATION LOGIC

Defines how incoming messages are aggregated into a single collection (list/payload).

Example: Collect payloads into a list

```
%dw 2.0
output application/java
---
payload map (item) -> item
```

DataWeave Logic

Other Options

- Custom aggregation using DataWeave
- Distinct aggregation
- Group by fields
- Filter and aggregate

5. BULK DATABASE INSERT

Insert aggregated records into database in bulk to improve performance.

Example (MySQL)

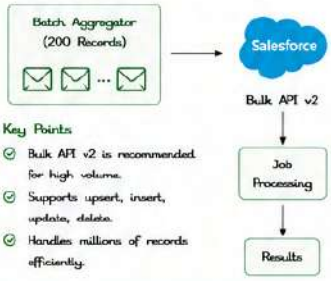
```
INSERT INTO CUSTOMER (ID, NAME, EMAIL)
VALUES (?, ?, ?), (?, ?, ?), (?, ?, ?), ... ;
```

Benefits

- Reduces number of database calls
- Higher throughput and performance

6. SALESFORCE BULK PROCESSING

Use Salesforce Bulk API v1 or Bulk API 2.0 to process large volumes efficiently.



Key Points

- Bulk API v2 is recommended for high volume.
- Supports upsert, insert, update, delete.
- Handles millions of records efficiently.

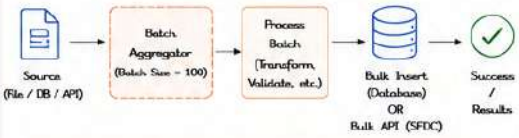
7. CONFIGURATION EXAMPLE (BATCH AGGREGATOR)

```
<batch:job name="batchAggregatorJob" maxFailedRecords="0">
    <batch:step name="step1">
        <batch:input>
            <file:inbound-endpoint path="/data/in"
                                   responseTimeout="10000"/>
        </batch:input>
        <batch:process-aggregator>
            <batch:aggregator batchSize="100"
                              timeOut="60000"
                              aggregationStrategy="#[payload]"
                />
        </batch:process-aggregator>
        <batch:step-commit commitSize="50"/>
    </batch:step>
</batch:job>
```

Parameters

- batchSize : Number of messages in batch.
- timeOut : Release batch after timeout.
- commitSize : Records committed per transaction.
- aggregationStrategy : How messages are aggregated.

8. BATCH PROCESSING FLOW



Flow Explanation

1. Messages are read from the source.
2. Batch Aggregator collects messages.
3. When batch size or timeout is reached, batch is released.
4. Batch is processed and inserted using bulk operation.
5. Results are returned / logged.

9. KEY CONCEPTS

Concept	Description
Batch Aggregator	Groups messages into a batch.
Batch Size	Number of messages to collect before releasing.
Commit Size	Number of records committed per transaction.
Aggregation Logic	Defines how messages are combined.
Bulk Insert	Insert batch into DB in one call.
Salesforce Bulk	Use Bulk API v1 or v2 for large data.

10. USE CASES

- Bulk data loading into database
- Data migration and synchronization
- E-commerce order processing
- Log processing and archiving
- Salesforce data integration

11. BEST PRACTICES

- Choose appropriate batch size based on payload and system capacity
- Use commit size to avoid large transactions.
- Use Bulk Insert / Bulk API for high volume data.
- Handle errors and retries at record level.
- Monitor memory usage and tune batch size accordingly.
- Use streaming where possible for large files.
- Log batch results and failures for auditing.

12. QUICK SUMMARY

- Batch Aggregator improves performance by grouping messages.
- Batch Size controls when the batch is released.
- Commit Size controls transaction size.
- Use bulk operations for database and Salesforce.
- Proper configuration ensures scalability and reliability.

KEY TAKEAWAY

Batch Aggregator + Bulk Operations
= Higher Throughput + Better Performance
+ Lower System Load + Scalable Integrations

MODULE 15 15.13

MESSAGE QUEUES

MuleSoft from salesforce

Message Queues enable systems to communicate asynchroncouly by storing messages until the consumer is ready to process them. They provide decoupling, reliability, scalability and fault tolerance in enterprise integrations.

1. ANYPOINT MQ

- Managed message broker by MuleSoft.
- Built on ActiveMQ Artemis.
- Fully managed and cloud native.
- High availability and resilience.
- Tight integration with Anypoint Platform.

Architecture

Features

- Queues, Topics
- Persistent messaging
- HA and auto-scaling
- Dead Letter Queue
- Monitoring and metrics
- Secure (TLS, IAM)

2. ACTIVEMQ

- Open source message broker.
- Supports multiple protocols (JMS, AMQP, MQTT, STOMP).
- Reliable, flexible and widely used.
- Supports Queues and Topics.

Architecture

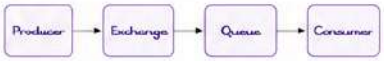
Features

- JMS 1.1 / 2.0 support
- Message persistence
- Clustering and failover
- DLQ support
- Message groups
- Web console

3. RABBITMQ

- Open source message broker.
- Implements AMQP protocol.
- Lightweight, high performance.
- Flexible routing with Exchanges.

Architecture



Features

- AMQP 0-9-1 support
- Routing (Direct, Topic, Fanout, Headers)
- Acknowledgements
- Message durability
- DLQ support
- Management UI

4. AWS SQS

- Fully managed message queuing service.
- Highly scalable, reliable and secure.
- Decouples and scales microservices.

Types

Features

- Unlimited throughput (Standard)
- Long polling, Visibility timeout
- Dead Letter Queue support
- Encryption
- CloudWatch metrics
- IAM security

5. AZURE SERVICE BUS

- Fully managed enterprise messaging service.
- Supports Queues and Topics (Pub/Sub).
- Reliable, scalable and secure.

Entities

Features

- Sessions, Transactions
- Dead-lettering
- Auto-forwarding
- Message filtering
- Duplicate detection
- Azure Monitor integration

6. KAFKA BASICS

- Distributed event streaming platform.
- High throughput, low latency.
- Designed for real-time data pipelines.

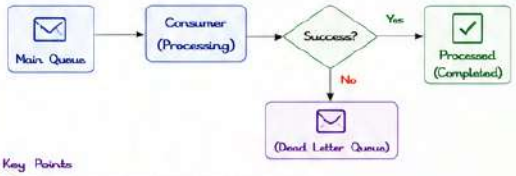
Core Concepts

Features

- High throughput
- Durable and scalable
- Consumer groups
- Fault tolerant
- Partitioning
- Stream processing

7. DLQ (DEAD LETTER QUEUE)

Stores messages that cannot be processed successfully after multiple retry attempts.

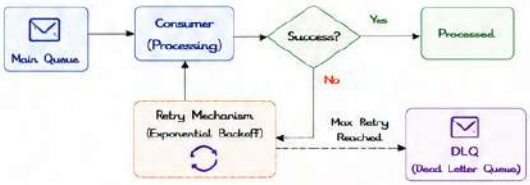


Key Points

1. Prevents bad messages from blocking the queue.
2. Helps in error analysis and reprocessing.
3. Messages can be manually fixed and re-queued.
4. Configured with max redelivery / retry count.

8. RETRY FLOW

Retry mechanism attempts to process failed messages before sending them to DLQ.



Retry Strategies

- Immediate Retry
- Fixed Interval Retry
- Exponential Backoff
- Maximum Retry Count

9. MESSAGE QUEUE COMPARISON

Platform	Type	Protocols	Persistence	Scalability	DLQ	Use Cases
Anypoint MQ	Managed Broker (Artemis)	JMS, AMQP, MQTT, STOMP	Yes	Auto-scale (Cloud)	Yes	Mule integrations, Enterprise apps
ActiveMQ	Open Source Broker	JMS, AMQP, STOMP, MQTT	Yes	High	Yes	General purpose messaging
RabbitMQ	Open Source Broker	AMQP	Yes	High	Yes	Microservices, Flexible routing
AWS SQS	Managed Service	HTTPS API	Yes	Very High	Yes	Cloud applications, Decoupling
Azure Service Bus	Managed Service	AMQP, HTTPS, SMP	Yes	Very High	Yes	Enterprise messaging, Hybrid solutions
Kafka	Streaming Platform	Kafka Protocol	Yes	Very High	(DLQ via App logic)	Event streaming, Real-time analytics

10. COMMON USE CASES

- Asynchronous processing
- Decoupling systems
- Load balancing
- Event-driven architectures
- Reliable message delivery
- Microservices communication
- Real-time data pipelines

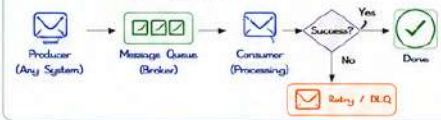
11. BEST PRACTICES

- Choose queues based on use case.
- Use persistent messages.
- Always configure DLQ.
- Implement retry with backoff.
- Set appropriate visibility timeout.
- Monitor queue metrics and alerts.
- Use idempotent consumers.
- Keep messages small.
- Secure queues with encryption.

12. KEY TAKEAWAYS

- Message Queues provide reliable asynchronous communication.
- DLQ and Retry mechanisms ensure fault tolerance.
- Choose the right broker based on performance and features.
- Proper configuration improves reliability and scalability.

TYPICAL FLOW



Protocol Legend

- JMS – Java Message Service
- AMQP – Advanced Message Queuing Protocol
- MQTT – IoT Protocol
- STOMP – Streaming Protocol
- SMP – Service Messaging Protocol

Right Message Queue + Proper Configuration + DLQ + Retry = Reliable, Scalable and Resilient Integrations

MODULE 15 15.14

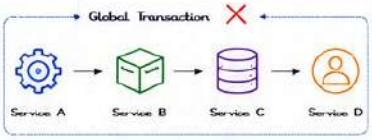
SAGA PATTERN

MuleSoft from salesforce

The Saga Pattern is a way to manage data consistency across multiple microservices without using traditional distributed transactions. It breaks a large transaction into a series of local transactions. If one fails, previous successful steps are compensated using compensating transactions.

1. DISTRIBUTED TRANSACTIONS

In microservices, each service has its own database. Traditional ACID transactions across services are not possible.



Problem: If Service C fails after A and B succeed, we cannot rollback changes in A and B automatically.

Solution: Use SAGA Pattern.

2. COMPENSATION TRANSACTIONS

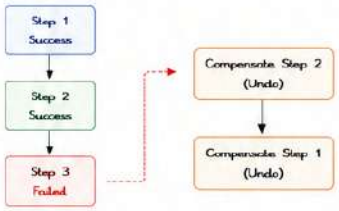
Each forward action has a compensating action that undoes its effect.

Service	Forward Action	Compensating Action
Order Service	Create Order	Cancel Order
Inventory Service	Reserve Stock	Release Stock
Payment Service	Charge Payment	Refund Payment
Shipping Service	Create Shipment	Cancel Shipment

Compensating transactions are executed in reverse order of completed steps.

3. ROLLBACK FLOW

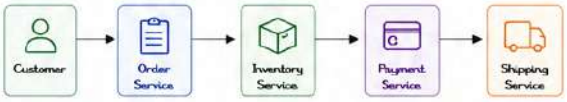
If any step fails, the system triggers compensating transactions for all previously completed steps.



System returns to a consistent state.

4. ORDER PROCESSING EXAMPLE (SAGA FLOW)

Scenario: Place an order in an e-commerce system.



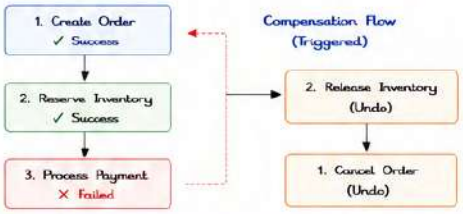
Forward Flow (Happy Path)
1. Create Order
2. Reserve Inventory
3. Process Payment
4. Create Shipment
5. Order Confirmed

Compensation Flow (If any step fails)
4. Cancel Shipment
3. Refund Payment
2. Release Inventory
1. Cancel Order

Compensation flow runs in reverse order of successful steps.

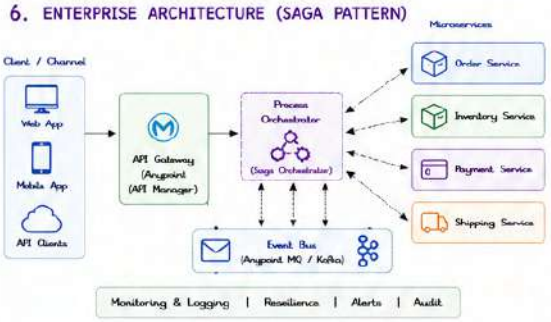
5. PAYMENT FAILURE HANDLING

If payment fails at step 3, previously successful steps are compensated.

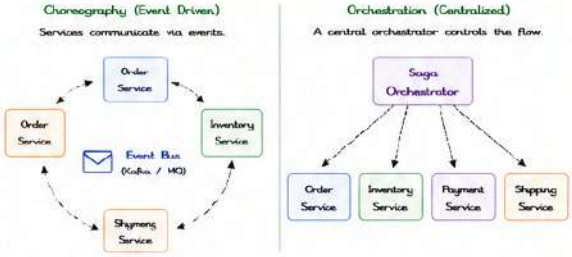


System is rolled back to original consistent state.

6. ENTERPRISE ARCHITECTURE (SAGA PATTERN)



7. SAGA TYPES



Orchestration gives more control, Choreography gives more scalability.

8. KEY BENEFITS

- Maintains data consistency across microservices.
- No need for distributed transactions (2PC).
- High availability and fault tolerance.
- Scalable and flexible.
- Works with eventual consistency.
- Suitable for cloud-native architectures.

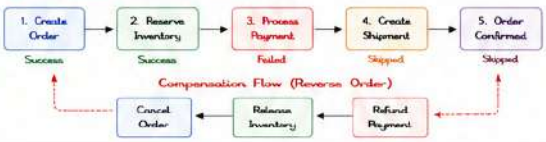
9. BEST PRACTICES

- Design compensating transactions carefully.
- Make compensating actions idempotent.
- Use reliable messaging (MQ/Kafka).
- Log and monitor each saga step.
- Handle timeouts and failures gracefully.
- Use correlation IDs to track saga flows.
- Implement retry with exponential backoff.

10. COMMON USE CASES

- Order processing in e-commerce
- Booking systems (Flight, Hotel)
- Banking & financial operations
- Insurance claim processing
- Subscription management

SAGA ROLLBACK EXAMPLE (DETAILED FLOW)



SAGA FLOW SUMMARY

- Execute steps one by one.
- If all succeed → Transaction completed.
- If any step fails → Trigger compensation.
- Compensate in reverse order.
- Ensure system returns to consistent state.

KEY TAKEAWAY

Saga Pattern ensures data consistency in distributed systems using local transactions, compensations, and reliable messaging.

Saga Pattern = Local Transactions + Compensations + Reliable Messaging = Consistency in Distributed Systems

MODULE 15 | 15.15

EVENT-DRIVEN ARCHITECTURE

MuleSoft from salesforce

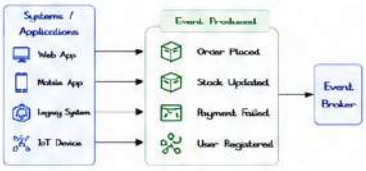
Event-Driven Architecture (EDA) allows systems to communicate by producing, detecting, and responding to events. It enables decoupled, scalable, resilient and real-time integrations.

1. EVENT PRODUCERS

Producers generate events when something happens in a system.

Examples

- Order Service creates "Order Placed" event
- Inventory Service creates "Stock Updated" event
- Payment Service creates "Payment Failed" event



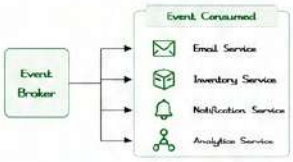
Producers are not aware of who will consume the event.

2. EVENT CONSUMERS

Consumers listen for events and take appropriate actions.

Examples

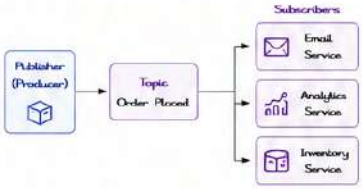
- Send email when order is placed
- Update inventory when stock updated
- Notify admin when payment failed



Consumers are decoupled from producers. They only care about the events.

3. PUBLISH / SUBSCRIBE

The Publish/Subscribe pattern allows multiple consumers to subscribe to events.



- Publisher publishes events to a topic.
- Subscribers subscribe to topics of interest.
- Broker delivers events to all subscribers.

4. EVENT BROKER

Event Broker receives events from producers and delivers them to consumers.

Responsibilities

- Accept events from producers
- Store events (durability)
- Route events to subscribers
- Ensure reliability and ordering
- Handle scaling and fault tolerance

Key Features

- Decoupling
- Scalability
- Reliability
- Durability
- High Throughput

Broker acts as the central nervous system of EDA.

5. KAFKA

Apache Kafka is a distributed event streaming platform.

Key Concepts

- Topic : A category/feed of events
- Partition : Parallelism and scalability
- Producer : Publishes events to topics
- Consumer Group : Group of consumers
- Offset : Position of event in a partition
- Retention : How long events are stored

Key Benefits

- High throughput and low latency
- Scalable and distributed
- Fault tolerant and durable
- Supports real-time streaming
- Ecosystem with Kafka Connect, Streams, etc.

6. ANYPOINT MQ (MULESOFT)

Fully managed message broker by MuleSoft.

Key Features

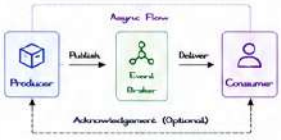
- Managed and cloud-native
- Persistent queues and topics
- High availability and auto scaling
- Message persistence and redelivery
- Dead-letter queues (DLQ)
- Secure (TLS, IAM, encryption)
- Easy integration with Anypoint Platform

7. ASYNC INTEGRATIONS

EDA enables asynchronous communication. Producer does not wait for consumer.

Benefits

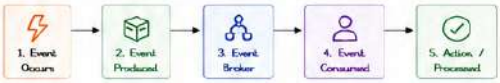
- Improved performance
- Better scalability
- Fault isolation
- Resilient to downtime
- Eventual consistency



Common Patterns

- Fire and Forget
- Request / Reply (Async)
- Competing Consumers
- Event Sourcing
- CQRS

8. EDA FLOW OVERVIEW



1. An event occurs in a system.
2. Producer creates and publishes the event.
3. Event Broker receives and stores/routes the event.
4. Consumers receive the event.
5. Consumers process the event and trigger actions.

9. KAFKA vs ANYPOINT MQ

Feature	Kafka	Anypoint MQ
Type	Open Source	Fully Managed (MuleSoft)
Deployment	Self-managed / Cloud	Managed (Cloud)
Protocol	Kafka Protocol	AMQP 1.0
Persistence	Configurable (Retention)	Persistent Queues / Topics
Scalability	Very High	Auto Scaling
Use Case	Event Streaming, Big Data	Enterprise Integration, APIs
Ecosystem	Kafka Connect, Streams	Anypoint Platform, DLQ, Monitoring

10. COMMON USE CASES

- Order processing and fulfillment
- Payment and transaction processing
- Inventory and stock updates
- User activity and notifications
- Log aggregation and monitoring
- IoT data ingestion and processing
- Real-time analytics and dashboards

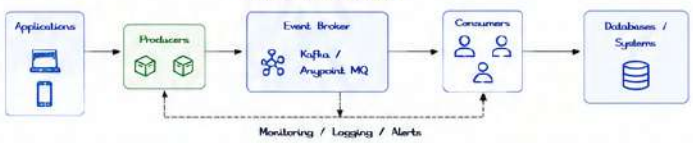
11. BEST PRACTICES

- Design event contracts carefully (schema, versioning).
- Use idempotent consumers.
- Handle failures and retries with DLQ.
- Monitor queues/topics and consumer lag.
- Use correlation IDs for tracing.
- Keep events small and relevant.
- Secure events in transit and at rest.
- Document events and flows.

12. KEY TAKEAWAYS

- EDA promotes decoupling and scalability.
- Events are the heart of communication.
- Use the right broker for the right use case.
- Handle failures, retries and monitoring properly.
- EDA enables real-time, resilient enterprise systems.

SAMPLE ARCHITECTURE



Right Events + Right Broker + Right Patterns = Scalable, Resilient and Real-time Enterprise Integrations

MODULE 15 | 15.16

END-TO-END ENTERPRISE ORDER PROCESSING SCENARIO

MuleSoft from salesforce

A complete real-time project demonstrating integration of Experience API, Process API, System APIs with Salesforce, SAP, Database, Kafka, SQS and best practices for enterprise-grade solutions.

1. BUSINESS SCENARIO

A customer places an order on the Web/Mobile App. The order is processed in real-time across multiple enterprise systems.

- Validate & enrich customer data
- Check product availability in SAP
- Create order in Salesforce
- Update inventory in SAP
- Send order confirmation (Email/SMS)
- All events are published to Kafka for analytics
- Handle failures with retry, error queues and alerts

ENTERPRISE SYSTEMS

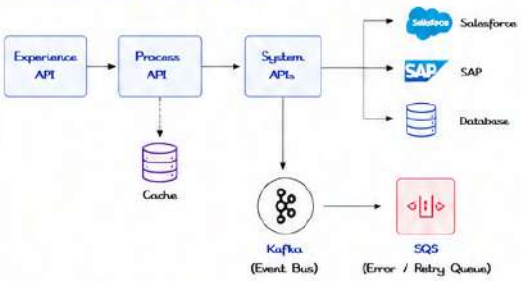
2. API LAYER

Experience API (Front-End Layer)
- Receives order from Web/Mobile
- Validates request
- Secures & forwards to Process API

Process API (Orchestration Layer)
- Orchestrates the end-to-end flow
- Calls System APIs
- Handles business rules & events

System APIs (Integration Layer)
- Connects to Salesforce, SAP, Database, Kafka, SQS
- Performs CRUD operations
- Handles transformation & mapping

3. INTEGRATION ARCHITECTURE



Key Flow

1. Experience API receives order.
2. Process API validates & orchestrates.
3. System APIs integrate with backend systems.
4. Events published to Kafka topics.
5. Failures go to SQS for retry & alerts.
6. Success messages trigger confirmations.

4. DATA FLOW

1. Order placed by customer.
2. Experience API validates & stores request.
3. Process API enriches customer, validates rules.
4. System API checks inventory in SAP.
5. Create order record in Salesforce.
6. Update inventory in SAP.
7. Persist order details in Database.
8. Publish events to Kafka.
9. Send confirmation via Email/SMS.

5. EVENT STREAMING (KAFKA)

Topic	Description
order.created	Published when order is placed
order.validated	Published after validation
inventory.updated	Published after inventory update
order.completed	Published when order succeeds
order.failed	Published when order fails

Benefits
- Decouples systems
- Real-time analytics
- Scalable and reliable

6. ERROR HANDLING & RETRY

Retry Strategy
- Max Retry Count
- Exponential Backoff
- Idempotent APIs
- Dead Letter Queue
- Alert & Notify

7. LOGGING & MONITORING

Logging
- Correlation ID for end-to-end tracing
- Log request / response
- Log errors with stack trace
- Log business events

Monitoring
- API performance & throughput
- Error rates & success rates
- Queue depth (Kafka, SQS)
- System health dashboards
- Alerts & Notifications

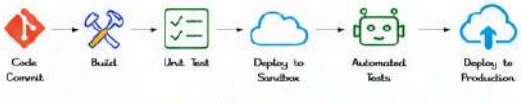
SECURITY

- OAuth 2.0 / JWT for API security
- TLS 1.2+ for all communications
- Role based access control (RBAC)
- Secure properties & secrets
- Input validation & data masking
- API rate limiting & throttling
- Audit logs & compliance

9. SYSTEMS INVOLVED

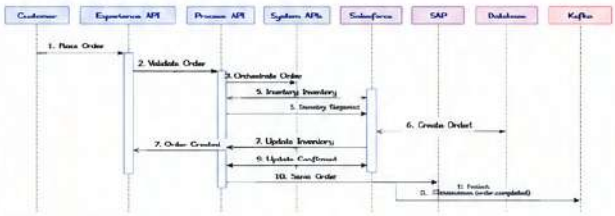
System	Role
Salesforce	Customer & Order Management
SAP	Inventory & Product Management
Database (MySQL / Oracle)	Order Persistence
Kafka	Event Streaming Platform
Amazon SQS	Error Queue & Retry Mechanism
Email/SMS Service	Customer Notifications

10. CI/CD PIPELINE



Tools Used

11. SEQUENCE DIAGRAM (HIGH LEVEL)



12. TECHNOLOGY STACK

MuleSoft
- Anypoint Platform
- Experience / Process / System APIs
- DataWeave
- Anypoint MQ / Connectors

Event & Messaging
- Kafka
- Amazon SQS

Back-end Systems
- Salesforce
- SAP
- Database

Other Services
- Email / SMS
- AWS CloudWatch
- Anypoint Monitoring
- Secrets Manager

KEY TAKEAWAY This end-to-end project demonstrates real-time order processing using modern integration patterns, robust error handling, secure APIs, event-driven architecture, and DevOps best practices.

MODULE 15 15.18

PRODUCTION TROUBLESHOOTING GUIDE

MuleSoft from salesforce

Use this guide to quickly identify, analyze and resolve common production issues in Mule applications.
Follow a structured approach: Detect → Diagnose → Analyze → Resolve → Validate → Monitor.

1. HIGH CPU

Symptoms
- CPU usage consistently > 80%
- High response time, timeouts
- System feels sluggish

Common Causes
- Tight loops / inefficient code
- Heavy DataWeave transformations
- Excessive logging
- Thread starvation / deadlocks

Troubleshooting Steps
1. Check CPU usage (top/htop, JMX)
2. Analyze thread dump for blocking threads
3. Review DataWeave profiling
4. Check logs for retry storms or loops
5. Optimize code, increase worker count if needed

2. MEMORY ISSUES

Symptoms
- Memory usage steadily increasing
- GC frequency high
- Application slow over time

Common Causes
- Memory leaks
- Large payloads / collections
- Caching without eviction

Troubleshooting Steps
1. Monitor heap usage (JMX, VisualVM)
2. Check heap dump for leak analysis
3. Review object retention & caches
4. Optimize DataWeave & payload size
5. Tune JVM (Xms, Xmx) and GC

3. OUTOFMEMORYERROR

Symptoms
- java.lang.OutOfMemoryError
- Application crashes or restarts
- GC overhead limit exceeded

Common Causes
- Heap size too small
- Memory leak
- Large in-memory data processing

Troubleshooting Steps
1. Check logs for OOM stack trace
2. Get heap dump (-XX:+HeapDumpOnOutOfMemoryError)
3. Analyze dump (MAT / VisualVM)
4. Fix leak or reduce payload size
5. Increase heap & tune GC

4. SLOW APIS

Symptoms
- High response time
- Users report slowness
- SLAs not met

Common Causes
- Backend system slow
- Inefficient queries / transformations
- Network latency
- Thread pool exhaustion

Troubleshooting Steps
1. Check end-to-end response time (APM)
2. Identify slow component (flow / call / DB)
3. Analyze downstream latency
4. Optimize queries, caching, payloads
5. Scale workers / tune thread pools

5. QUEUE BUILD-UP

Symptoms
- Queue size continuously increasing
- Messages delayed
- Consumers lagging

Common Causes
- Consumers slower than producers
- Downstream system slow/unavailable
- Retry storms
- Insufficient worker threads

Troubleshooting Steps
1. Check queue size (JMX, MQ Console)
2. Compare producer vs consumer rate
3. Check downstream system health
4. Tune batch size / concurrency
5. Scale consumers or workers

6. TIMEOUT ISSUES

Symptoms
- Read / connection / response timeouts
- Failed requests
- Intermittent errors

Common Causes
- Backend slow/unreachable
- Network issues
- Timeout values too low
- Thread starvation

Troubleshooting Steps
1. Check logs for timeout exceptions
2. Verify backend system health
3. Test connectivity & latency
4. Increase timeout values (if appropriate)
5. Optimize performance / reduce payload

7. WORKER CRASHES

Symptoms
- Workers stopping unexpectedly
- Application auto-restarts
- Errors in logs / alerts

Common Causes
- Unhandled exceptions
- Memory leaks / OOM
- Invalid configurations
- Dependency failures

Troubleshooting Steps
1. Check logs around crash time
2. Review stack trace & error cause
3. Check system resources (CPU, Memory)
4. Validate recent deployments / configs
5. Fix issue & monitor restarts

8. LOG ANALYSIS

Key Things to Look For
- Error stack traces
- Exception messages
- Timestamps & correlation IDs
- External system responses
- Retry attempts & failures

Useful Log Fields

Field	Purpose
timestamp	When the event occurred
correlationId	Trace end-to-end flow
thread	Identify thread issues
logger	Source of the log
message	Actual log message
level	INFO / WARN / ERROR

9. ROOT CAUSE ANALYSIS

RCA Approach
1. **Identify the Problem** — What is the issue and impact?
2. **Collect Data** — Logs, metrics, metrics, dumps, alerts
3. **Analyze** — Find patterns, errors, bottlenecks
4. **Determine Root Cause** — Use 5 Whys / Fishbone / Timeline
5. **Implement Fix** — Resolve underlying cause
6. **Validate & Monitor** — Confirm fix and monitor closely

5 Whys Example

Problem: Orders are failing
↓ Why? Timeout
↓ Why? Backend slow
↓ Why? DB overloaded
↓ Why? Missing index
↓ Why? Change not tested
↓ Root Cause: Missing DB index in release

10. QUICK DIAGNOSTIC CHECKLIST

Area	Checklist
System	CPU, Memory, Disk, Network, JVM, GC
Application	Errors in logs, recent deployments, configs
APIs	Response times, error rates, timeouts
Queues	Queue size, consumer lag, retry queues
Databases	DB connections, slow queries, locks
External Systems	Availability, latency, error rates
Security	Auth failures, token expiry, cert issues

11. TOOLS & COMMANDS

Tool / Command	Use
JMX / Anypoint Monitoring	CPU, Memory, Threads, Queues
Thread Dump (jstack)	Analyze thread issues
Heap Dump (jmap)	Memory leak analysis
VisualVM / MAT	Heap & CPU analysis
APM (New Relic, Dynatrace)	End-to-end performance
MQ Console (Kafka, SQS, MQ)	Queue depth, lag, metrics
OS Tools (top, iostat, netstat)	System resource usage

12. BEST PRACTICES

- Set correct JVM heap & GC settings
- Implement proper logging with correlation IDs
- Use circuit breakers and bulkheads
- Monitor key metrics and set alerts
- Load test before production releases
- Keep configurations externalized
- Design for resiliency and graceful degradation
- Regularly review and clean up logs
- Document runbooks for common issues
- Conduct post-incident reviews

KEY TAKEAWAY

Monitor proactively, detect early, analyze systematically, fix permanently.
A structured approach reduces downtime and improves system reliability.


MuleSoft
from salesforce
Thank You!
For reading and learning with us.
We hope this MuleSoft Anypoint Platform Components Guide – Volume 3 has helped you deepen your knowledge and enhance your integration expertise.
Keep practicing, keep building, and keep innovating.
You're now one step closer to becoming a MuleSoft integration expert!
Thank you for being a part of this journey!
Your curiosity and commitment drive your growth and inspire continuous learning.
Dataweave Handbook
coming soon